Laporan Tugas Besatr 1

IF3070 Dasar Inteligensi Artifisial 2026/2027

Optimasi Penataan Kendaraan pada Geladak Kapal Menggunakan Algoritma Local Search

Kelompok 21 - Antum Intelligence

NIM	Nama
18224001	Nisrina Zakiyah
18224017	Naomi Azzahra
18224083	Antania Hanjani Yustika Putri

Dosen Pengampu: Ir. Windy Gambetta, MBA

Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

DAFTAR ISI

DAFTAR ISI **2**

DAFTAR TABEL **4**

DAFTAR GAMBAR **5**

1. Deskripsi Persoalan **7**

1.1 Latar Belakang **7**

1.2 Variabel Permasalahan **8**

1.3 Objective Function **8**

1.4 Constraints **9**

1.5 Representasi Permasalahan **10**

1.5.1 Representasi Geladak Kapal 10

1.5.2 Representasi Kendaraan 11

1.5.3 Representasi State 11

1.6 Initial State **12**

1.7 Neighbor **12**

1.7.1 Mengubah orientasi kendaraan 12

1.7.2 Memindahkan kendaraan ke koordinat lain 13

1.7.3 Menukar dua kendaraan 13

2. Pembahasan **17**

2.1 Implementasi Algoritma Local Search **17**

2.1.1 Struktur Program dan Fungsi Umum **17**

2.1.2 Hill-Climbing Steepest Ascent **24**

2.1.3 Simulated Annealing **28**

2.1.4 Genetic Algorithm **33**

2.1.5 Visualisasi hasil **43**

2.2 Hasil Eksperimen **43**

2.2.1 Konfigurasi Data Uji..... 43
2.2.2 Hill-Climbing Steepest Ascent..... 45
2.2.3 Simulated Annealing..... 48
2.2.4 Genetic Algorithm..... 54
2.3 Analisis Eksperimen..... 77
2.3.1 Kedekatan terhadap Global Optima..... 77
2.3.2 Perbandingan Hasil Antar Algoritma..... 79
Xx..... 79
2.3.3 Perbandingan Durasi Pencarian..... 79
2.3.4 Konsistensi Hasil..... 80
2.3.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm..... 81
3. Kesimpulan dan Saran..... 82
3.1 Kesimpulan..... 82
3.2 Saran..... 83
4. Pembagian Tugas..... 84
REFERENSI..... 85
Lampiran A. Form Penggunaan Generative AI..... 86
Lampiran B. Lampiran Lainnya..... 88

DAFTAR TABEL

Tabel 2.1 fungsi umum program.....17

Tabel 2.1.2 fungsi HC Steepest Ascent..... 25

Tabel 2.1.3 fungsi Simulated Annealing.....30

Tabel 2.1.4 fungsi Genetic Algorithm..... 36

Tabel 2.2.2 Hasil Percobaan HC Steepest Ascent.....45

Tabel 2.2.3 Hasil Percobaan Simulated Annealing..... 49

Tabel 2.3.2 Perbandingan Hasil Antar Algoritma..... 79

Tabel 3.1 Perbandingan durasi, kontrol populasi vs kontrol iterasi (dari rata-rata 3 seed).....80

Tabel 4 Pembagian Tugas..... 84

DAFTAR GAMBAR

Gambar hc-sa_run1. Initial dan Final state untuk run dengan Seed 21..... 46
Gambar hc-sa_run2. Initial dan Final state untuk run dengan Seed 42..... 46
Gambar hc-sa_run3. Initial dan Final state untuk run dengan Seed 63..... 46
Gambar hc-objective_run1..... 47
Gambar hc-objective_run2..... 47
Gambar hc-objective_run3..... 48
Gambar SA-1. State awal run 1, seed 21, objective –112680..... 50
Gambar SA-2. State akhir run 1, 17 kendaraan, objective 1170..... 50
Gambar SA-3. State awal run 2, seed 42, objective –56060..... 51
Gambar SA-4. State akhir run 2, 18 kendaraan, objective 1240..... 51
Gambar SA-5. State awal run 3, seed 63, objective –37110..... 52
Gambar SA-6. State akhir run 3, 17 kendaraan, objective 1210..... 52
Gambar SA-7. History objective dan probabilitas penerimaan run 1..... 53
Gambar SA-8. History objective dan probabilitas penerimaan run 2..... 53
Gambar SA-9. History objective dan probabilitas penerimaan run 3..... 54
Gambar Initial State G-A1-1. Seed 5, pop 30, iter 50. Value : -48170..... 56
Gambar End State G-A1-1. Seed 5, pop 30, iter 50. Value : 1140..... 56
Gambar Initial State G-A1-2. Seed 6, pop 30, iter 50. Value : -25260..... 57
Gambar End State G-A1-2. Seed 6, pop 30, iter 50. Value : 1020..... 57
Gambar Initial State G-A1-3. Seed 7, pop 30, iter 50. Value : -27220..... 58
Gambar End State G-A1-3. Seed 7, pop 30, iter 0. Value : 860..... 58
Gambar Initial State G-A2-1. Seed 5, pop 30, iter 100. Value : -48170..... 59
Gambar End State G-A2-1. Seed 5, pop 30, iter 100. Value : 1210..... 59
Gambar Initial State G-A2-2. Seed 6, pop 30, iter 100. Value : -25260..... 60
Gambar End State G-A2-2. Seed 6, pop 30, iter 100. Value : 1100..... 60

Gambar Initial State G-A2-3. Seed 7, pop 30, iter 100. Value : -27220.....61
Gambar End State G-A2-3. Seed 7, pop 30, iter 100. Value : 920.....61
Gambar Initial State G-A3-1. Seed 5, pop 30, iter 200. Value : -48170.....62
Gambar End State G-A3-1. Seed 5, pop 30, iter 200. Value : 1210.....62
Gambar Initial State G-A3-2. Seed 6, pop 30, iter 200. Value : -25260.....63
Gambar End State G-A3-2. Seed 6, pop 30, iter 200. Value : 1100.....63
Gambar Initial State G-A3-3. Seed 7, pop 30, iter 200. Value : -27220.....64
Gambar End State G-A3-3. Seed 7, pop 30, iter 200. Value : 1000.....64
Gambar Initial State G-B1-1. Seed 5, pop 20, iter 100. Value : -51090.....65
Gambar End State G-B1-1. Seed 5, pop 20, iter 100. Value : 1110.....65
Gambar Initial State G-B1-3. Seed 7, pop 20, iter 100. Value : -27220.....67
Gambar End State G-B1-3. Seed 7, pop 20, iter 100. Value : 930.....67
Gambar Initial State G-B2-1. Seed 5, pop 50, iter 100. Value : -48170.....68
Gambar End State G-B2-1. Seed 5, pop 50, iter 100. Value : 1160.....68
Gambar Initial State G-B2-2. Seed 6, pop 50, iter 100. Value : -25260.....69
Gambar End State G-B2-2. Seed 6, pop 50, iter 100. Value : 1180.....69
Gambar Initial State G-B2-3. Seed 7, pop 50, iter 100. Value : -27220.....70
Gambar End State G-B2-3. Seed 7, pop 50, iter 100. Value : 1210.....70
Gambar Initial State G-B3-1. Seed 5, pop 100, iter 100. Value : -48170.....71
Gambar End State G-B3-1. Seed 5, pop 100, iter 100. Value : 1010.....71
Gambar Initial State G-B3-2. Seed 6, pop 100, iter 100. Value : -22330.....72
Gambar End State G-B3-2. Seed 6, pop 100, iter 100. Value : 1110.....72
Gambar Initial State G-B3-3. Seed 7, pop 100, iter 100. Value : -22250.....73
Gambar End State G-B3-3. Seed 7, pop 100, iter 100. Value : 1160.....73
Gambar G-A1-1. Nilai objective max dan rata-rata populasi.....74
Gambar G-A1-2. Nilai objective max dan rata-rata populasi.....74

Gambar G-A1-3. Nilai objective max dan rata-rata populasi.....75
Gambar G-A2-1. Nilai objective max dan rata-rata populasi.....75
Gambar G-A2-2. Nilai objective max dan rata-rata populasi.....75
Gambar G-A2-3. Nilai objective max dan rata-rata populasi.....76
Gambar G-A3-1. Nilai objective max dan rata-rata populasi.....76
Gambar G-A3-2. Nilai objective max dan rata-rata populasi.....77
Gambar G-A3-3. Nilai objective max dan rata-rata populasi.....77

1. Deskripsi Persoalan

1.1 Latar Belakang

PT Intim Line merupakan perusahaan yang mengoperasikan kapal kargo pengangkut kendaraan. Proses bisnis utama perusahaan dimulai dari menerima pengiriman dengan *dealer* kendaraan beberapa waktu sebelum pengiriman dilakukan. Daftar yang diterima berupa daftar kendaraan dengan atribut dimensi (p*l), berat, tarif pengiriman dan deadline, setiap data berbeda-beda.

Pada Status Quo, penempatan kendaraan pada geladak kapal dilakukan secara manual ditempat. Artinya, penggunaan ruang pada geladak kapal sepenuhnya bergantung pada staf operasional. Akibatnya, ruang geledak dimanfaatkan secara maksimal dan optimal dan kendaraan dengan nilai paling menguntungkan untuk perusahaan tidak dapat diprioritaskan secara sistematis.

Secara sistematis, jika PT Intim Line berniat untuk digitalisasi proses pemasukkan kendaraan pada geledak terdapat kompleksitas dalam pemrosesan data. Setiap kendaraan memiliki banyak pilihan dan jumlah kombinasi yang turut tumbuh dengan bertambahnya kendaraan. Ditambah dengan batasan teknis pada dunia nyata seperti kapasitas berat kapal dan kapasitas berat kapal. Sehingga yang dicari adalah susunan yang paling menguntungkan untuk PT Intim Line secara finansial namun tetap berada pada batasan teknis dunia nyata.

Karakteristik permasalahan diatas sesuai dengan algoritma *local search.* Pada masalah yang membutuhkan susunan akhir yang terbaik, algoritma solusi cukup menyimpan state saat ini lalu memperbaikinya secara bertahap. Pendekatan ini efektif secara penghematan memori dan mampu menghasilkan solusi yang layak dalam waktu singkat tanpa menelusuri seluruh ruang pencarian. Pada laporan ini dibandingkan tiga algoritma dari kategori *local search* yaitu *Steepest Ascent Hill Climbing, Simulated Annealing,* dan *Genetic Algorithm.* Eksperimen Bertujuan untuk memaksimalkan total tarif pengiriman dengan tetap memenuhi seluruh batasan, serta menentukan algoritma yang paling sesuai berdasarkan nilai objective, waktu eksekusi dan konsistensi hasil.

1.2 Variabel Permasalahan

Entitas	Atribut	Keterangan
Vehicle	id	Identifier kendaraan
Vehicle	Dimensions (w, l)	Ukuran kendaraan dalam satuan grid
Vehicle	Orientation	Horizontal atau Vertical
Vehicle	ShippingFee	Tarif pengiriman (juta rupiah)
Vehicle	Weight	Berat kendaraan (unit)
Vehicle	ETA	Estimasi waktu kedatangan (sisa hari)
Ship	Dimensions (w, l)	Ukuran geladak kapal dalam satuan grid
Ship	MaxCapacity	Kapasitas angkut maksimum (unit)

1.3 Objective Function

Tujuan utama optimasi adalah memaksimalkan total *fee* kendaraan yang dimuat. Setiap state menyimpan seluruh kendaraan, tetapi kendaraan dengan posisi None dianggap tidak diangkut. Nilai objective dihitung menggunakan rumus berikut:

$$f(s) = \sum fee_i - 1000 \times (P_{kapasitas} + P_{overlap} + P_{keluar}), \text{untuk i dengan s != None}$$

$P_{kapasitas}$ menyatakan kelebihan berat muatan terhadap kapasitas kapal, $P_{overlap}$ menghitung jumlah penempatan tambahan pada sel yang sudah terisi. Misalnya, jika tiga kendaraan menempati satu sel, nilai *overlap* pada sel tersebut adalah 2. Selain itu, P_{keluar} menghitung sel kendaraan yang berada di luar batas geladak. Sel di luar kapal tidak dihitung lagi sebagai *overlap*. Ketiga penalti bernilai 0 pada *state* yang valid, sehingga *objective* = *total fee*.

Bobot penalti ditetapkan sebesar 1000. Ia lebih besar daripada *fee* per kendaraan dalam data uji, yaitu 20 sampai 100, tetapi tidak menjamin semua susunan valid selalu memiliki nilai *objective* lebih tinggi daripada susunan yang melanggar.

Terdapat juga fungsi *objective* alternatif yang menilai solusi berdasarkan jumlah kendaraan yang diangkut, ini menggunakan penalti yang sama dengan *objective* utama, tetapi mengganti total fee dengan jumlah kendaraan:

$$g(s) = \sum (\text{kendaraan dengan posisi != None}) - 1000 \times (P_{kapasitas} + P_{overlap} + P_{keluar})$$

Perbedaan kedua fungsi tersebut terlihat jelas ketika dua susunan sama-sama valid. Contohnya, susunan pertama memuat dua kendaraan dengan total *fee* = 80, sedangkan susunan kedua memuat satu kendaraan dengan *fee* = 100. *Objective* utama memilih susunan kedua karena ***fee*-nya lebih besar**, *Objective* alternatif memilih susunan pertama karena j**umlah kendaraannya lebih banyak**.

Kedua objective menggunakan fungsi hitung_penalti() yang sama untuk menghitung pelanggaran kapasitas, overlap, dan batas geladak. Bagian utama implementasinya adalah:

```
def objective_function(state, daftar_kendaraan, kapal):
   """Hitung total fee kendaraan dikurangi penalti pelanggaran."""
   total_fee = hitung_fee(state, daftar_kendaraan)
   return total_fee - hitung_penalti(state, daftar_kendaraan, kapal)


def objective_function_jumlah(state, daftar_kendaraan, kapal):
   """Hitung jumlah kendaraan yang diangkut dikurangi penalti."""
   jumlah_kendaraan = sum(posisi is not None for posisi in state)
   return jumlah_kendaraan - hitung_penalti(state, daftar_kendaraan,
kapal)
```

Fungsi alternatif diuji pada contoh *state* kosong, satu dan dua kendaraan valid, kasus *overlap*, kendaraan keluar kapal, dan kelebihan kapasitas. Untuk dua kendaraan 1 x 1 yang berada pada sel yang sama, nilainya 2 - 1000 = -998. Kendaraan tunggal yang menempati satu sel di luar lapal memperoleh nilai 1 - 1000 = -999. Dua kendaraan dengan total berat = 4 pada kapal berkapasitas = 3 juga memperoleh nilai -998.

1.4 Constraints

Terdapat tiga *constraint* yang diterapkan pada permasalahan penataan kendaraan di geladak kapal seperti yang dicantumkan pada spesifikasi.

1. Kendaraan tidak boleh saling bertumpuk (*no overlapping*).

Pelanggaran *constraint* terjadi ketika satu sel di geladak ditempati oleh dua atau lebih kendaraan. Pengecekan *constraint* ditangani oleh **`cek_pelanggaran_overlap()`**, yang menghitung jumlah kejadian penumpukan sel yang sudah terisi. Misalnya, jika tiga kendaraan menempati satuperc sel, maka nilai pelanggarannya adalah 2.

2. Seluruh sel kendaraan yang diangkut harus berada di dalam batas geladak.

   Pelanggaran *constraint* terjadi ketika suatu kendaraan terangkut memiliki *body* yang berada di luar geladak. Pengecekan *constraint* dilakukan oleh **`cek_pelanggaran_keluar_kapal(),`** yang menghitung jumlah sel ditempati kendaraaan berada di luar geladak.

3. Total berat kendaraan yang diangkut tidak boleh melebihi kapasitas maksimal kapal.

   Pelanggaran *constraint* terjadi ketika total berat kendaraan terangkut melebihi kapasitas maksimal kapal. Pengecekan *constraint* ditangani oleh **`cek_pelanggaran_kapasitas()`** yang menghitung kelebihan berat kendaraan terangkut terhadap kapasitas maksimal kapal.

Ketiga *constraint* akan ditangani oleh penalti. Fungsi **`hitung_penalti()`** akan menjumlahkan nilai dari masing-masing pelanggaran lalu mengalikannya dengan bobot 1000. Hasilnya nanti akan digunakan pada **`objective_function()`** untuk mengurangi total *fee*. Bobot 1000 dipilih karena nilainya jauh lebih besar dari *fee* per kendaraan sehingga pelanggaran dinilai lebih merugikan. Pada *state* yang valid, ketiga pelanggaran akan bernilai 0.

1.5 Representasi Permasalahan

1.5.1 Representasi Geladak Kapal

Kapal pengangkut kendaraan direpresentasikan oleh objek dari kelas Kapal yang memiliki tiga atribut yaitu dp (panjang geladak yang searah sumbu x), dl (lebar geladak yang searah sumbu y) dan kapasitas (berat maksimum yang dapat diangkut). Geladak kapal dimodelkan sebagai grid dua dimensi berukuran dp x dl, dengan masing-masing sel diwakili oleh satu titik koordinat. Titik (0,0) terletak pada

pojok kiri atas geladak. Suatu titik (x,y) dapat dikatakan dalam geladak jika memenuhi syarat **`0 ⩽ x < dp`** dan **`0 ⩽ y < dl`**, sedangkan di luar rentang tersebut dianggap berada di luar geladak.

1.5.2 Representasi Kendaraan

Kendaraan direpresentasikan oleh objek dari kelas Kendaraan yang memiliki beberapa atribut yaitu id (pengenal unik kendaraan), p (panjang kendaraan), l (lebar kendaraan), berat, fee, dan ETA (estimasi waktu kedatangan). Kendaraan berbentuk segi empat dengan ukuran p x l. Pada orientasi horizontal, kendaraan menempati sebanyak p sel pada sumbu x dan l sel pada sumbu y, sedangkan untuk orientasi vertikal sebaliknya. Bagian tubuh dari suatu kendaraan akan menempati beberapa sel berbeda dalam geladak. Himpunan sel ini akan dihitung oleh fungsi program.

1.5.3 Representasi State

State adalah satu penataan kendaraan lengkap, berupa *list* yang panjangnya sama dengan jumlah kendaraan, dan urutannya mengikuti daftar kendaraan. Setiap elemen pada *state* berisi salah satu dari **`None`** atau **`(x, y, arah)`**. **`None`** merupakan kondisi di mana kendaraan tidak terangkut kapal. **`(x, y, arah)`** mendandakan kendaraan tersebut terangkut kapal, dengan ujung kiri atas kendaraan berada pada titik **`(x, y)`** dan kendaraan diangkut dalam orientasi **`arah`**

Contoh state untuk tiga kendaraan:

```
state= [(0,0,"H"), None, (5,2,"V")]
```

Artinya:

- Kendaraan 1 diangkut kapal, posisi ujung kiri atas kendaraan di (0,0), orientasi kendaraan horizontal
- Kendaraan 2 tidak diangkut kapal

- Kendaraan 3 diangkut kapal, posisi ujung kiri atas kendaraan di (5,2), orientasi kendaraan vertikal

1.6 Initial State

Untuk menyelesaikan permasalahan penataan kendaraan di geladak kapal, kami menggunakan *initial state* yang dibuat secara acak sesuai dengan ketentuan spesifikasi. Dibandingkan dengan algoritma seperti *first fit* atau *best fit* yang mungkin terkesan sudah tertata sejak awal dan membuat algoritma *searching* memulai pencarian dekat *local maximum*, *random initial state* memungkinkan kami untuk mengeksplorasi *initial state* yang beragam setiap kali algoritma *searching* dijalankan. Dengan adanya variasi ini, membuat adanya kemungkinan bagi algoritma *searching* untuk menjangkau ruang pencarian berbeda.

Pembuatan *initial state* akan ditangani oleh **`random_initial_state()`** yang akan membuat keputusan terangkut atau tidaknya setiap kendaraan. Algoritma ini memberi peluang 50% untuk setiap kendaraan tidak terangkut (representasi: None). Kendaraan yang terangkut akan diberi posisi (x,y,arah) di mana x dan y merupakan letak kendaraan di kapal, serta arah merupakan orientasi acak antara horizontal/vertikal. Pembuatan *initial state* tidak memeriksa constraint, sehingga *state* awal sangat mungkin melanggar *constraint* yang ada.

1.7 Neighbor

Pencarian *neighbor* akan ditangani oleh **`get_neighbors()`** dengan tiga jenis perpindahan yang diberikan pada spesifikasi

1.7.1 Mengubah orientasi kendaraan

Orientasi dari kendaraan yang terangkut akan diubah dari horizontal (“H”) menjadi vertikal (“V”) atau dari vertikal (“V”) menjadi horizontal (“H”). Namun, hal ini tidak berlaku pada kendaraan dengan dimensi 1:1 karena memutar orientasinya tidak akan mengubah state.

1.7.2 Memindahkan kendaraan ke koordinat lain

Perpindahan ini akan menangani beberapa kasus. Kasus pertama, yakni memindahkan koordinat dari kendaraan terangkut ke koordinat lain yang ada di dalam geladak dengan orientasi tetap. Kasus kedua, yakni memberikan koordinat pada kendaraan yang belum terangkut sekaligus arahnya yaitu horizontal. Kasus ketiga, yakni memindahkan kendaraan yang terangkut keluar geladak sehingga representasi koordinatnya akan None.

1.7.3 Menukar dua kendaraan

Dua kendaraan akan ditukar posisinya. Hal ini berlaku untuk dua kendaraan terangkut, atau satu terangkut dan satu tidak. Bagi dua kendaraan yang sama-sama tidak terangkut tidak akan ditukar karena hasilnya akan sama.

```
def get_neighbors(state, daftar_kendaraan, kapal):
    '''
    Menghasilkan semua Neighbor dari current state
    Neighbor didapatkan dengan cara berikut:
    1. Mengubah arah 1 Kendaraan (V to H, H to V)
        Tapi tidak berlaku untuk kendaraan dimensi 1:1 karena akan sama
saja
    2. Memindahkan 1 Kendaraan ke koordinat lain
        Pemindahan koordinat yang ditangani:
        a. Pindahkan 1 Kendaraan di dalam kapal ke koordinat lain di
dalam kapal juga
        b. Pindahlan 1 Kendaraan di luar kapal ke koordinat dalam kapal
        c. Pindahkan 1 Kendaraan di dalam kapal ke luar (None)
    3. Menukar posisi 2 Kendaraan
        Penukaran yang ditangani:
        - Tukar 2 Kendaraan yang sama2 di dalam Kapal
        - Tukar 1 Kendaraan di dalam kapal dan 1 di luar
        2 Kendaraan di luar kapal ga ditukar karena akan sama saja

    Parameter:
```

```
    - state (current)
    - daftar_kendaraan: list objek Kendaraan
    - kapal

    Return:
    - list of state: semua neighbor
    '''
    daftar_neighbor = []
    n = len(state)

    # Mengubah arah
    for i, posisi in enumerate(state):
        # Skip kendaraan yang gapunya posisi (gaaada arah)
        if posisi is None:
            continue
        x, y, arah = posisi

        # Skip Kendaraan 1:1 (karena akan sama saja)
        k = daftar_kendaraan[i]
        if k.p == k.l:
            continue

        if arah == "V":
            arah_baru = "H"
        else:
            arah_baru = "V"
        if not is_muat(k, x, y, arah_baru, kapal):
            continue
        new_state = state[:]
        new_state[i] = (x, y, arah_baru)
        daftar_neighbor.append(new_state)

    # 2a. Pindahkan 1 Kendaraan di dalam kapal ke koordinat lain di dalam
kapal juga
```

```
    for i, posisi in enumerate(state):
        if posisi is None:
            continue
        x, y, arah = posisi
        k = daftar_kendaraan[i]
        ukuran_x, ukuran_y = ukuran_kendaraan(k, arah)

        # Hanya pindahkan ke titik yang membuat seluruh badannya tetap di
dalam Kapal
        for x_baru in range(kapal.dp - ukuran_x+1):
            for y_baru in range(kapal.dl - ukuran_y+1):
                if (x_baru,y_baru) == (x,y):
                    continue
                new_state = state[:]
                new_state[i] = (x_baru, y_baru, arah)
                daftar_neighbor.append(new_state)

    # 2b. Pindahlan 1 Kendaraan di luar kapal ke koordinat dalam kapal
    for i, posisi in enumerate(state):
        if posisi is not None:
            continue
        k = daftar_kendaraan[i]

        for arah in ["H", "V"]:
            ukuran_x, ukuran_y = ukuran_kendaraan(k, arah)
            for x in range(kapal.dp - ukuran_x +1):
                for y in range(kapal.dl - ukuran_y +1):
                    new_state = state[:]
                    new_state[i] = (x, y, arah)
                    daftar_neighbor.append(new_state)

    # 2c. Memindahkan 1 Kendaraan di dalam ke luar (None)
    for i, posisi in enumerate(state):
        if posisi is None:
```

```
            continue

        new_state = state[:]
        new_state[i] = None
        daftar_neighbor.append(new_state)

    # Menukar posisi 2 Kendaraan
    for i in range(n):
        for j in range(i+1, n):
            #Skip kalau 2 2 nya di luar
            if state[i] is None and state[j] is None:
                continue

            k1 = daftar_kendaraan[i]
            k2 = daftar_kendaraan[j]
            #K2 pindah ke posisi lama K1 kalau posisi K1 bukan None
            if state[i] is not None:
                x1,y1,arah1 = state[i]
                if not is_muat(k2, x1, y1, arah1, kapal):
                    continue
            #K1 pindah ke posisi lama K2 kalau posisi K2 bukan None
            if state[j] is not None:
                x2,y2,arah2 = state[j]
                if not is_muat(k1, x2, y2, arah2, kapal):
                    continue

            new_state = state[:]
            new_state[i], new_state[j] = new_state[j], new_state[i]
            daftar_neighbor.append(new_state)

    return daftar_neighbor
```

2. Pembahasan

2.1 Implementasi Algoritma Local Search

2.1.1 Struktur Program dan Fungsi Umum

Tabel 2.1 fungsi umum program

Fungsi/Kelas	Deskripsi/Tujuan	Parameter	Return
`load_data()`	Memuat data uji (data.json). Data kendaraan dan kapal diubah menjadi objek Kendaraan dan objek Kapal yang bisa diproses oleh program.	`Path` (lokasi file json)	Kapal, data_kendaraan
`random_initial_state()`	Membuat *initial state* acak yang berisi posisi masing-masing kendaraan. Tiap kendaraan berpeluang 50% tidak diangkut.	`Daftar_kendaraan, kapal`	List state per kendaran
`get_neighbors()`	Membuat semua kemungkinan *neighbor* dari state sekarang. Dilakukan dengan mengubah orientasi satu kendaraan, memindahkan koordinat satu kendaraan, dan menukar dua kendaraan.	`State, daftar_kendaraan, kapal`	List state tetangga
`objective_function()`	Menghitung nilai *objective* utama suatu state berdasarkan total *fee* kendaraan yang terangkut dikurang penalti pelanggaran.	`State, daftar_kendaraan, kapal`	Nilai objective
`objective_function_jumlah()`	Menghitung nilai *objective* alternatif berdasarkan jumlah	`State, daftar_kend`	Nilai objective

	kendaraan yang diangkut, dikurangi penalti pelanggaran kapasitas, *overlap*, dan kendaraan yang keluar dari batas geladak	`araan, kapal`	
`hitung_fee()`	Menjumlahkan fee dari kendaraan yang terangkut	`State, daftar_kendaraan`	Total fee
`hitung_penalti()`	Menghitung penalti pelanggaran dengan menjumlahkan poin dari masing-masing pelanggaran lalu diberi bobot.	`State, daftar_kendaraan, kapal`	Total penalti
`cek_pelanggaran_kapasitas()`	Menghitung kelebihan berat kendaraan yang diangkut terhadap kapasitas maksimal kapal.	`State, daftar_kendaraan, kapal`	Kelebihan berat
`cek_pelanggaran_overlap()`	Menghitung berapa banyak kejadian di mana dua atau lebih kendaraan menempati posisi yang sama pada geladak kapal.	`State, daftar_kendaraan, kapal`	Jumlah koordinat overlap
`cek_pelanggaran_keluar_kapal()`	Menghitung jumlah koordinat kendaraan terangkut yang berada di luar batas geladak kapal.	`State, daftar_kendaraan, kapal`	Jumlah koordinat di luar geladak
`sel_terisi()`	Menghasilkan daftar koordinat yang ditempati oleh satu kendaraan	`Kendaraan, posisi`	List koordinat
`ukuran_kendaraan()`	Helper penghitung ukuran kendaraan (terhadap geladak) berdasarkan arah	`Kendaraan, arah`	Panjang/lebar

`is_muat()`	Helper yang mengecek apakah kendaraan dengan posisi tertentu muat disisipkan di kapal	`Kendaraan, posisi (x,y,arah), kapal`	True/False

Source code utils.py

```
import json
import random
from models import Kapal, Kendaraan

def load_data(path):
    with open(path, 'r') as file:
        data = json.load(file)
    kp  = data['kapal']
    kapal = Kapal(kp['dl'], kp['dp'], kp['kapasitas'])

    d_kendaraan = []
    for kd in data["kendaraan"]:
        kendaraan = Kendaraan(kd['id'], kd['p'], kd['l'], kd['berat'],
kd['fee'], kd['ETA'])
        d_kendaraan.append(kendaraan)

    return kapal, d_kendaraan

def random_initial_state(daftar_kendaraan, kapal):
    '''
    Membuat state awal secara acak

    Parameter:
    - daftar_kendaraan: list objek Kendaraan
    - kapal

    Return:
```

```
    - state: list posisi (None atau (x, y, arah)) untuk tiap kendaraan
    '''
    state = []
    for kendaraan in daftar_kendaraan:
        if random.random() < 0.5: # Tiap kendaraan punya peluang 50%
tidak diangkut (None)
            state.append(None)
        else:
            arah = random.choice(["V", "H"])
            ukuran_x, ukuran_y = ukuran_kendaraan(kendaraan, arah)
            x = random.randint(0, kapal.dp - ukuran_x)
            y = random.randint(0, kapal.dl - ukuran_y)
            state.append((x, y, arah))
    return state

def get_neighbors(state, daftar_kendaraan, kapal):
    '''
    Menghasilkan semua Neighbor dari current state
    Neighbor didapatkan dengan cara berikut:
    1. Mengubah arah 1 Kendaraan (V to H, H to V)
        Tapi tidak berlaku untuk kendaraan dimensi 1:1 karena akan sama
saja
    2. Memindahkan 1 Kendaraan ke koordinat lain
        Pemindahan koordinat yang ditangani:
        a. Pindahkan 1 Kendaraan di dalam kapal ke koordinat lain di
dalam kapal juga
        b. Pindahlan 1 Kendaraan di luar kapal ke koordinat dalam kapal
        c. Pindahkan 1 Kendaraan di dalam kapal ke luar (None)
    3. Menukar posisi 2 Kendaraan
        Penukaran yang ditangani:
        - Tukar 2 Kendaraan yang sama2 di dalam Kapal
        - Tukar 1 Kendaraan di dalam kapal dan 1 di luar
        2 Kendaraan di luar kapal ga ditukar karena akan sama saja
```

```
    Parameter:
    - state (current)
    - daftar_kendaraan: list objek Kendaraan
    - kapal

    Return:
    - list of state: semua neighbor
    '''
    daftar_neighbor = []
    n = len(state)

    # Mengubah arah
    for i, posisi in enumerate(state):
        # Skip kendaraan yang gapunya posisi (gaaada arah)
        if posisi is None:
            continue
        x, y, arah = posisi

        # Skip Kendaraan 1:1 (karena akan sama saja)
        k = daftar_kendaraan[i]
        if k.p == k.l:
            continue

        if arah == "V":
            arah_baru = "H"
        else:
            arah_baru = "V"
        if not is_muat(k, x, y, arah_baru, kapal):
            continue
        new_state = state[:]
        new_state[i] = (x, y, arah_baru)
        daftar_neighbor.append(new_state)

    # 2a. Pindahkan 1 Kendaraan di dalam kapal ke koordinat lain di dalam
```

```
kapal juga
    for i, posisi in enumerate(state):
        if posisi is None:
            continue
        x, y, arah = posisi
        k = daftar_kendaraan[i]
        ukuran_x, ukuran_y = ukuran_kendaraan(k, arah)

        # Hanya pindahkan ke titik yang membuat seluruh badannya tetap di
dalam Kapal
        for x_baru in range(kapal.dp - ukuran_x+1):
            for y_baru in range(kapal.dl - ukuran_y+1):
                if (x_baru,y_baru) == (x,y):
                    continue
                new_state = state[:]
                new_state[i] = (x_baru, y_baru, arah)
                daftar_neighbor.append(new_state)

    # 2b. Pindahlan 1 Kendaraan di luar kapal ke koordinat dalam kapal
    for i, posisi in enumerate(state):
        if posisi is not None:
            continue
        k = daftar_kendaraan[i]

        for arah in ["H", "V"]:
            ukuran_x, ukuran_y = ukuran_kendaraan(k, arah)
            for x in range(kapal.dp - ukuran_x +1):
                for y in range(kapal.dl - ukuran_y +1):
                    new_state = state[:]
                    new_state[i] = (x, y, arah)
                    daftar_neighbor.append(new_state)

    # 2c. Memindahkan 1 Kendaraan di dalam ke luar (None)
    for i, posisi in enumerate(state):
```

```
        if posisi is None:
            continue

        new_state = state[:]
        new_state[i] = None
        daftar_neighbor.append(new_state)

    # Menukar posisi 2 Kendaraan
    for i in range(n):
        for j in range(i+1, n):
            #Skip kalau 2 2 nya di luar
            if state[i] is None and state[j] is None:
                continue

            k1 = daftar_kendaraan[i]
            k2 = daftar_kendaraan[j]
            #K2 pindah ke posisi lama K1 kalau posisi K1 bukan None
            if state[i] is not None:
                x1,y1,arah1 = state[i]
                if not is_muat(k2, x1, y1, arah1, kapal):
                    continue
            #K1 pindah ke posisi lama K2 kalau posisi K2 bukan None
            if state[j] is not None:
                x2,y2,arah2 = state[j]
                if not is_muat(k1, x2, y2, arah2, kapal):
                    continue

            new_state = state[:]
            new_state[i], new_state[j] = new_state[j], new_state[i]
            daftar_neighbor.append(new_state)

    return daftar_neighbor

def ukuran_kendaraan(k, arah):
```

```
    '''
    Menghitung ukuran berdasarkan arah kendaraan
    '''
    if arah == "H":
        return k.p, k.l
    elif arah == "V":
        return k.l, k.p

def is_muat(k, x, y, arah, kapal):
    '''
    Cek apakah kendaraan dengan posisi x,y,arah muat di kapal
    '''
    ukuran_x, ukuran_y = ukuran_kendaraan(k, arah)
    if (x + ukuran_x <= kapal.dp) and (y + ukuran_y <= kapal.dl):
        return True
    return False
```

2.1.2 Hill-Climbing Steepest Ascent

Hill Climbing Steepest Ascent adalah salah satu varian *Hill Climbing* standar, yaitu algoritma *local search* yang memanfaatkan fungsi heuristik. Algoritma dimulai dari *initial state lengkap* dan menuju *goal state* dengan cara berpindah ke *neighbor* yang lebih baik.

Implementasi *Hill Climbing Steepest Ascent* terdapat pada kelas **`HC_steepest`** di **`src/algorithms/hc_steepest.py`.** Berikut merupakan tahap pencarian *Steepest Ascent* dalam Permasalahan Penataan Kendaraan di Geladak Kapal:

1. Algoritma akan menerima *initial state* yang dibentuk oleh **`random_initial_state()`** berupa list posisi dari masing-masing kendaraan. Kendaraan bisa tidak diangkut (None) ataupun terangkut dengan representasi (x, y, arah). Karena dibentuk secara acak, state awal dapat terjadi *overlap,* kendaraan di luar batas geladak, atau melebihi kapasitas

geladak. *State* ini akan dinilai menggunakan **`objective_function()`** yang menghitung total fee kendaraan terangkut dikurangi penalti pelanggaran.

2. Pada setiap iterasi, algoritma akan mencari seluruh *neighbor* dari *current state* meggunakan **`get_neighbors()`** yang bekerja dengan mengubah orientasi satu kendaraan, memindahkan satu kendaraan ke koordinat lain, memasukkan atau mengeluarkan kendaraan dari geladak, dan menukar dua kendaraan. Setiap *neighbor* akan dievaluasi menggunakan **`objective_function()`**, lalu dipilih satu *neighbor* dengan nilai *objective* tertinggi. Jika nilai *neighbor* terpilih lebih besar dari nilai *current state*, maka algoritma akan berpindah ke *neighbor* tersebut.
3. Setiap perpindahan akan dicatat pada *history* berupa *dictionary* yang berisi nomor iterasi dan nilai *objective* pada state yang baru.
4. Iterasi akan terus berlanjut hingga menemukan kondisi terminasi di mana *neighbor* terpilih memiliki nilai lebih kecil atau sama dengan nilai *current state*. Pada kondisi ini, pencarian berhenti dan *current state* saat itu akan dikembalikan sebagai *final state.*

Tabel 2.1.2 fungsi HC Steepest Ascent

Fungsi/Kelas	Deskripsi
`HC_steeest.__init_()`	Menyimpan *initial state* dan menginisialisasi pencatatan *history.*
`HC_steeest.run()`	Menjalankan proses *searching* sampai tidak ada *neighbor* yang lebih baik, lalu mencatat *state* akhir, nilai *objective* akhir, *durasi*, *iterasi,* dan *history.*

Bagian inti pemilihan dan penerimaan kandidat pada run() adalah:

```
            # Cari neighbor yang punya value terbaik
            highest_value = float("-inf")
            best_neighbor = None
```

```
            for n in neighbors:
                neighbor_value = objective_function(n,
self.daftar_kendaraan, self.kapal)
                if neighbor_value > highest_value:
                    highest_value = neighbor_value
                    best_neighbor = n

            # Tidak menerima jika value neighbor terpilih ⩽ current value,
terminate
            if highest_value ⩽ current_value:
                break

            # Pindah ke best neighbor, perbarui current value
            current_state = best_neighbor
            current_value = highest_value
```

Source code HC_steepest.py

```
import time
from objective_function import objective_function
from utils import get_neighbors

class HC_steepest:
    '''
    Hill Climbing Steepest Ascent
    1x iterasi:
    - Periksa semua neighbor, lalu pilih neighbor dengan value tertinggi
    - Jika value neighbor terpilih > value state sekarang, pindah.
    - Jika value neighbor terpilih ⩽ value state sekarang, terminate dan
kembalikan solusi.
    '''
    def __init__(self, initial_state, daftar_kendaraan, kapal):
        self.initial_state = initial_state
        self.daftar_kendaraan = daftar_kendaraan
```

```
        self.kapal = kapal
        self.iteration_count = 0
        self.duration = 0
        self.final_state = None
        self.final_value = None
        self.history = []

    def run(self):
        # Inisialisasi
        self.history = []
        self.iteration_count = 0
        waktu_mulai = time.time()

        # Memulai dengan initial state
        current_state = self.initial_state[:]
        current_value = objective_function(current_state,
self.daftar_kendaraan, self.kapal)
        self.history.append({
            "iterasi": self.iteration_count,
            "value": current_value
        })

        while True:
            # Cari semua suksesor/neighbor
            neighbors = get_neighbors(current_state, self.daftar_kendaraan,
self.kapal)
            if not neighbors:
                break

            # Cari neighbor yang punya value terbaik
            highest_value = float("-inf")
            best_neighbor = None
            for n in neighbors:
                neighbor_value = objective_function(n,
```

```
self.daftar_kendaraan, self.kapal)
                if neighbor_value > highest_value:
                    highest_value = neighbor_value
                    best_neighbor = n

            # Tidak menerima jika value neighbor terpilih ⩽ current value,
terminate
            if highest_value ⩽ current_value:
                break

            # Pindah ke best neighbor, perbarui current value
            current_state = best_neighbor
            current_value = highest_value
            self.iteration_count += 1
            self.history.append({
                "iterasi": self.iteration_count,
                "value": current_value
            })

        self.final_state = current_state
        self.final_value = current_value
        self.duration = time.time() - waktu_mulai
```

2.1.3 Simulated Annealing

Simulated Annealing adalah salah satu algoritma *local search* yang memilih satu *neighbor* secara *random* pada setiap iterasi. Satu neighbor dipilih secara acak menggunakan **random.choice()** tanpa mempertimbangkan nilai objectivenya. Jika neighbor yang terpilih memiliki nilai objective lebih rendah daripada state saat ini, neighbor tersebut masih dapat diterima dengan probabilitas $p = e^{(\frac{\Delta E}{T})}$, sehingga pencarian dapat berpindah dari susunan yang kurang baik.

Implementasi SA terdapat pada kelas **SimulatedAnnealing** di **src/algorithms/simulated_annealing.py.** *State* awal dibuat oleh

random_initial_state() pada **utils.py** dan dipanggil melalui **main.py**. Setiap elemen state berada pada indeks yang sama dengan kendaraan pada **daftar_kendaraan**. **None** berarti kendaraan pada indeks tersebut tidak dimuat, sedangkan tuple **(x, y, arah)** menunjukkan posisi dan orientasinya. Karena objective dan generator neighbor menggunakan indeks ini, urutan state dan **daftar_kendaraan** harus selalu konsisten.

Fungsi **get_neighbors()** menghasilkan kandidat melalui perubahan orientasi, perpindahan kendaraan di dalam kapal, pemasukan kendaraan ke kapal, pengeluaran kendaraan dari kapal, dan pertukaran posisi dua kendaraan. Koordinat kandidat dibatasi agar badan kendaraan tetap berada di dalam kapal, tetapi kandidat dapat tetap mengalami *overlap* dengan kendaraan lain atau menyebabkan kelebihan kapasitas karena kedua kondisi tersebut ditangani melalui penalti objective. Pada setiap iterasi, **get_neighbors()** terlebih dahulu membangkitkan seluruh kandidat neighbor. Setelah itu, SA memilih tepat satu kandidat menggunakan **random.choice()** dan hanya mengevaluasi objective kandidat tersebut. Kandidat yang mengalami *overlap* atau kelebihan kapasitas tidak langsung ditolak, tetapi pelanggaran tersebut dihitung melalui penalti pada *objective function*. Namun, posisi kendaraan yang keluar dari kapal umumnya dicegah sejak proses pembangkitan neighbor melalui pemeriksaan batas menggunakan **is_muat()** dan pembatasan koordinat.

Perubahan nilai dinyatakan sebagai ΔE = f(neighbor) − f(current_state). Karena objective dimaksimalkan, kandidat dengan ΔE >= 0 diterima. Jika ΔE < 0, probabilitas penerimaannya adalah $p = e^{(\frac{\Delta E}{T})}$.

Sebagai contoh, Berdasarkan history **SA_run1**, probabilitas penerimaan kandidat yang lebih buruk tercatat sebesar 0,3657 pada iterasi ke-4. Ketika temperatur semakin rendah, probabilitas dapat menjadi sangat kecil, yaitu 2,48 x 10^{-22} pada iterasi ke-1800 dan 0,0 pada iterasi ke-3000. Hal ini menunjukkan bahwa kandidat dengan *objective* lebih rendah semakin sulit diterima seiring turunnya temperatur.

Temperatur dimulai dari 1000 dan dikalikan dengan 0,998 setelah setiap iterasi, baik kandidat diterima maupun ditolak. Dengan demikian, temperatur pada awal iterasi ke-(k) dapat dituliskan sebagai:

$$T_k = 1000(0{,}998)^k$$

Loop berlanjut selama (T_k > 0,1).

Pencarian berhenti ketika temperatur tidak lagi lebih besar dari 0,1 atau ketika tidak tersedia neighbor. Dengan parameter bawaan T_0 = 1000, cooling_rate = 0,998, dan Tmin = 0,1, serta dengan asumsi *neighbor* selalu tersedia, SA menjalankan 4601 iterasi.

rejected_moves mencatat jumlah neighbor yang memiliki nilai *objective* lebih rendah dan kemudian ditolak oleh proses probabilistik. Nilai ini bukan jumlah *local optimum* dan bukan pula jumlah seluruh kandidat yang lebih buruk, karena kandidat yang lebih buruk kadang-kadang tetap diterima. Selain itu, SA tidak mengevaluasi seluruh *neighbor* untuk menentukan apakah *state* saat ini merupakan *local optimum*.

Tabel 2.1.3 fungsi Simulated Annealing

Fungsi/Kelas	Deskripsi
SimulatedAnnealing.**__init__()**	Menyimpan state awal dan parameter temperature, lalu memvalidasi nilainya
SimulatedAnnealing.**run**	Menjalankan proses searching dan mencatat state akhir, nilai objective, history, durasi, iterasi, dan jumlah penolakan langkah yang lebih buruk

Bagian inti pemilihan dan penerimaan kandidat pada **run()** adalah:

```
            neighbor = random.choice(neighbors)
            neighbor_value = objective_function(neighbor,
self.daftar_kendaraan, self.kapal)
            delta_e = neighbor_value - current_value

            probability = 1.0 if delta_e ⩾ 0 else math.exp(delta_e /
temperature)
```

```
            if delta_e ⩾ 0 or random.random() < probability:
                current_state = neighbor
                current_value = neighbor_value
            else:
                self.rejected_moves += 1

            temperature *= self.cooling_rate
```

Source code Simulated Annealing

```
import math
import random
import time
from objective_function import objective_function
from utils import get_neighbors

class SimulatedAnnealing:
   def __init__(self, initial_state, daftar_kendaraan, kapal,
                initial_temperature=1000, cooling_rate=0.998,
minimum_temperature=0.1):
       if not 0 < minimum_temperature < initial_temperature < math.inf:
           raise ValueError("Temperature harus bernilai positif dan
minimum < initial")
       if not 0 < cooling_rate < 1:
           raise ValueError("Cooling rate harus berada di antara 0 dan
1")

       self.initial_state = initial_state
       self.daftar_kendaraan = daftar_kendaraan
       self.kapal = kapal
       self.initial_temperature = initial_temperature
       self.cooling_rate = cooling_rate
```

```
        self.minimum_temperature = minimum_temperature
        self.iteration_count = 0
        self.duration = 0
        self.final_state = None
        self.final_value = None
        self.history = []
        self.rejected_moves = 0

    def run(self):
        self.iteration_count = 0
        self.rejected_moves = 0
        waktu_mulai = time.perf_counter()

        current_state = self.initial_state[:]
        current_value = objective_function(current_state,
self.daftar_kendaraan, self.kapal)
        temperature = self.initial_temperature
        self.history = [{
            "iterasi": 0,
            "value": current_value,
            "probability": None
        }]

        while temperature > self.minimum_temperature:
            neighbors = get_neighbors(current_state,
self.daftar_kendaraan, self.kapal)
            if not neighbors:
                break

            neighbor = random.choice(neighbors)
            neighbor_value = objective_function(neighbor,
self.daftar_kendaraan, self.kapal)
            delta_e = neighbor_value - current_value
```

```
            probability = 1.0 if delta_e ⩾ 0 else math.exp(delta_e /
temperature)

            if delta_e ⩾ 0 or random.random() < probability:
                current_state = neighbor
                current_value = neighbor_value
            else:
                self.rejected_moves += 1

            self.iteration_count += 1
            self.history.append({
                "iterasi": self.iteration_count,
                "value": current_value,
                "probability": probability
            })
            temperature *= self.cooling_rate

        self.final_state = current_state
        self.final_value = current_value
        self.duration = time.perf_counter() - waktu_mulai
```

2.1.4 Genetic Algorithm

Genetic Algorithm (setelah ini akan ditulis sebagai GA) adalah algoritma optimisasi yang mengembangkan sekumpulan kandidat solusi (**populasi**) dari generasi ke generasi. Setiap kandidat disebut **individu.** GA memilih ***parent*** berdasarkan nilai ***fitness***, menggabungkan gen dari dua *parent* melalui ***crossover***, lalu dalam beberapa waktu tertentu mengubah gen melali **mutasi**. Implementasinya terdapat pada kelas **`Genetic.`**

2.1.4.1 Representasi Kromosom

Setiap individu direpresentasikan sebagai *list* dengan satu gen untuk setiap kendaraan, mengikuti urutan **`daftar_kendaraan`** . Gen bernilai **`None`** jika kendaraan tidak dimuat, atau **`(x, y, arah)`** jika dimuat. **`x`** dan **`y`** adalah posisi

kendaraan di kapal, dan **arah** bernilai **"H"** (Horizontal) atau **"V"** (vertikal). Populasi awal dibuat dengan memanggil **random_intial_state()** untuk setiap individu.

2.1.4.2 Fungsi Fitness

Fitness digunakan untuk menilai kualitas individu, dihitung dengan **objective_funciton()** dari **utils.py** dan dimaksimalkan oleh GA:

$$fitness(s) = total\ fee(s) - 1000\ x\ (P_{kapasitas}(s) + P_{overlap}(s) + P_{keluar}(s))$$

Total_fee adalah jumlah *fee*kendaraan yang dimuat. Penalti kapasitas dihitung dari kelebihan berat diatas kapasitas kapal, penalti *overlap* dari jumlah sel yang ditempati lebih dari satu kendaraan, dan penalti keluar dari jumlah sel kendaraan yang berada di luar batas kapal.

2.1.4.3 Fungsi Seleksi

Parent dipilih dengan *tournament selection*. Pada setiap pemilihan, tiga individu diambil secara acak (atau seluruh individu jika ukuran populasi kurang dari tiga), dan individu dengan *fitness* tertinggi menjadi *parent*.

```
   def tournament(self, population, fitness):
        """Seleki tournament: ambik k individu acak dan kembalikan yg
objectivenya tertinggi"""
        picked = random.sample(range(len(population)),
min(self.tournament_k, len(population)))
        winner = max(picked, key=lambda i: fitness[i])
        return population[winner]
```

2.1.4.3 Fungsi Crossover

Crossover menggabungkan gen dua *parent* menjadi dua anak sehingga bagian solusi yang baik dari masing-masing *parent* dapat bertemu. Dengan peluang **crossover_rate = 0.9**, GA menggunakan uniform *crossover* untuk menghasilkan dua anak. Untuk setiap posisi gen, salah satu parent dipilih dengan

peluang 0,5 dan anak kedua menerima gen dari parent yang satunya. Jika *crossover* tidak terjadi, kedua anak menjadi salinan dari kedua parent.

```
def crossover(self, parent_a, parent_b):
    child_1, child_2 = [], []
    for gene_a, gene_b in zip(parent_a, parent_b):
        if random.random() <0.5:
            child_1.append(gene_a)
            child_2.append(gene_b)
        else:
            child_1.append(gene_b)
            child_2.append(gene_a)
    return child_1, child_2
```

2.1.4.3 Fungsi Mutasi

Mutasi mengubah gen secara acak agar populasi tetap beragam dan tidak terjebak pada solusi lokal. Setiap anak memiliki **`mutation_rate = 0.3`** , untuk dimutasi. Mutasi memiliki satu kendaraan secara acak:

- Jika gen bernilai None, kendaraan dimasukkan ke posisi acak dengan orientasi “H” atau “V”.
- Jika kendaraan sudah dimuat, dipilih secara acak satu dari tiga tindakan: mengubah orientasi, memindahkan posisi, atau mengeluarkan dari kapal.
- Untuk kendaraan dengan panjang dan lebar sama, tindakan rotasi diganti dengan pemindahan posisi.

```
def mutate(self, state):
    new_state = state[:]
    i = random.randrange(len(new_state))
    kendaraan = self.daftar_kendaraan[i]
    posisi = new_state[i]

    if posisi is None:
        arah = random.choice(["H", "V"])
```

```
            x, y = _posisi_acak(kendaraan, self.kapal, arah)
            new_state[i] = (x, y, arah)
            return new_state

        x, y , arah = posisi
        move = random.choice(["rotate", "move", "remove"])

        #handling 1:1 vehicle

        if move == "rotate" and kendaraan.p == kendaraan.l :
            move = "move"

        if move == "rotate" :
            new_state[i] = (x, y, "V" if arah == "H" else "H")
        elif move == "move" :
            x_baru, y_baru = _posisi_acak(kendaraan, self.kapal, arah)
            new_state[i] = (x_baru, y_baru, arah)
        else:
            new_state[i] = None

        return new_state
```

2.1.4.3 Elitism

Sebanyak elite = 2 individu terbaik langsung disalin ke generasi berikutnya tanpa *crossover* atau mutasi. Bertujuan untuk menjamin fitness terbaik tidak turun dari generasi ke generasi akibat operator acak.

```
            order = sorted(range(self.pop_size), key=lambda i: fitness[i],
reverse=True)
            next_pop = [population[i] for i in order [:self.elite]]
```

Tabel 2.1.4 fungsi Genetic Algorithm

Fungsi/Kelas	Deskripsi

Genetic.__init__()	Menyimpan daftar kendaraan, kapal, ukuran populasi, jumlah generasi, dan parameter GA
Genetic.evaluate()	Menghitung fitness setiap individu dalam populasi dengan fungsi `objective_function()`
Genetic.tournament()	Memilih satu parent: mengambil `tournament_k` individu acak, lalu mengambilkan individu dengan nilai fitness tertinggi.
Genetic.crossover()	Menghasilkan dua anak melalui uniform crossover, tiap gen diambil dari parent A atau B dengan peluang 0,5.
Genetic.mutate()	Mengubah satu kendaraan secara acak, dimuat ke posisi acak jika belum dimuat, atau dipindadah, atau dikeluaarkan jika sudaa dimuat
Genetic.run()	Menjalankan seluruh GA, membuat populasi awal, lalu tiap generasi melakukan elitisme, seleksi *parent*,, crossover, mutasi, evaluasi dan pencatatan histori individu terbaik.

Source code Genetic Algorithms

```
import math
import random
import time
from objective_function import objective_function
from utils import random_initial_state

def _ukuran(kendaraan, arah):
    """Ukuran (x, y) kendaraan sesuai arah."""
    if arah == "H":
        return kendaraan.p, kendaraan.l
    return kendaraan.l, kendaraan.p

def _posisi_acak(kendaraan, kapal, arah):
    """Posisi (x, y) acak yang membuat seluruh badan kendaraan tetap di
```

```
kapal"""
    ukuran_x, ukuran_y = _ukuran(kendaraan, arah)
    x = random.randint(0, max(0, kapal.dp-ukuran_x))
    y = random.randint(0, max(0, kapal.dl-ukuran_y))

    return x, y

class Genetic:
    """Geneteic Algorithm (GA) untukk memaksimalkan objective_function.
    Pola pemakaian : ga = Genetic(daftar_kendaraan, kapal, pop_size,
iterations)
    ga.run()
    Param yg bisa diubah :
    - pop_size (jumlah individu (kendaraan))
    - iterations : banyak generasi

    Hyperparam tetap : crossover_rate, mutation_rate, tournament_k, elite

    Atribut hasil (setelah run()):
    - initial_state: individu terbaik di populasi awal
    - final_state: individu terbaik yang pernah ditemukan
    - final_value: objective final_state
    - candidate_peak: nilai maksimum populasi pada generasi terakhir
    - history: list of dict {"iterasi", "value" (max populasi), "avg"
(rata-rata populasi)}
    - duration: lama pencarian (detik)
    """

    def __init__(self, daftar_Kendaraan, kapal, pop_size=50,
iterations=100,
                 crossover_rate = 0.9, mutation_rate=0.3, tournament_k=3,
elite=2):
        self.daftar_kendaraan = daftar_Kendaraan
        self.kapal = kapal
```

```
        self.pop_size = pop_size
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.tournament_k = tournament_k
        self.elite = min(elite, pop_size)

        self.initial_state = None
        self.initial_value = None
        self.final_state = None
        self.final_value = None
        self.candidate_peak = None
        self.history = []
        self.duration = 0
        self.iterations = iterations

    def evaluate(self, population):
        return [objective_function(state, self.daftar_kendaraan,
self.kapal)
                for state in population]

    def tournament(self, population, fitness):
        """Seleki tournament: ambik k individu acak dan kembalikan yg
objectivenya tertinggi"""
        picked = random.sample(range(len(population)),
min(self.tournament_k, len(population)))
        winner = max(picked, key=lambda i: fitness[i])
        return population[winner]

    def crossover(self, parent_a, parent_b):
        """Uniform crossover: tiap gen (1 kendaraan) diambil dari parent
A atau B dengan peluang sama.
        Posisi ke-i selalu kendaraan ke-i. Return 2 anak (masing-masing
berupa state).."""
        child_1, child_2 = [], []
```

```
        for gene_a, gene_b in zip(parent_a, parent_b):
            if random.random() <0.5:
                child_1.append(gene_a)
                child_2.append(gene_b)
            else:
                child_1.append(gene_b)
                child_2.append(gene_a)
        return child_1, child_2

    def mutate(self, state):
        """Mutasi: membuat 1 perubahaan acak pada 1 kendaraan. eg:
        - No vehicle in ship : masukkan ke kapal dengan posisi acak
        - Vehicle avail in ship : ganti arah, pindah posisi, keluarkan
        return state : New state """
        new_state = state[:]
        i = random.randrange(len(new_state))
        kendaraan = self.daftar_kendaraan[i]
        posisi = new_state[i]

        if posisi is None:
            arah = random.choice(["H", "V"])
            x, y = _posisi_acak(kendaraan, self.kapal, arah)
            new_state[i] = (x, y, arah)
            return new_state

        x, y , arah = posisi
        move = random.choice(["rotate", "move", "remove"])

        #handling 1:1 vehicle

        if move == "rotate" and kendaraan.p == kendaraan.l :
            move = "move"

        if move == "rotate" :
```

```
            new_state[i] = (x, y, "V" if arah == "H" else "H")
        elif move == "move" :
            x_baru, y_baru = _posisi_acak(kendaraan, self.kapal, arah)
            new_state[i] = (x_baru, y_baru, arah)
        else:
            new_state[i] = None

        return new_state

    def run(self):
        start = time.perf_counter()

        #population initiation
        population = [
            random_initial_state(self.daftar_kendaraan, self.kapal)
            for _ in range(self.pop_size)
        ]
        fitness = self.evaluate(population)

        best_idx = max(range(self.pop_size), key=lambda i: fitness[i])
        self.initial_state = population[best_idx]
        self.initial_value = fitness[best_idx]
        self.final_state = self.initial_state
        self.final_value = self.initial_value

        self.history = [{"iterasi" : 0,
                         "value" : max(fitness),
                         "avg" : sum(fitness)/self.pop_size}]

        #Genetic Loop

        for generasi in range(1, self.iterations + 1):
            #elitism : return is the best individual just as is
            order = sorted(range(self.pop_size), key=lambda i:
```

```
fitness[i], reverse=True)
            next_pop = [population[i] for i in order [:self.elite]]

            while len(next_pop) < self.pop_size:
                parent_a = self.tournament(population, fitness)
                parent_b = self.tournament(population, fitness)

                if random.random() < self.crossover_rate:
                    child_1, child_2 = self.crossover(parent_a, parent_b)
                else:
                    child_1, child_2 = parent_a[:], parent_b[:]

                for child in (child_1, child_2):
                    if random.random() < self.mutation_rate:
                        child = self.mutate(child)
                    if len(next_pop) < self.pop_size:
                        next_pop.append(child)

            population = next_pop
            fitness = self.evaluate(population)

            #record max and avrg
            gen_best = max(range(self.pop_size), key=lambda i:
fitness[i])
            self.history.append({"iterasi": generasi,
                                 "value": fitness[gen_best],
                                 "avg": sum(fitness)/self.pop_size})

            if fitness[gen_best] > self.final_value:
                self.final_value = fitness[gen_best]
                self.final_state = population[gen_best]

        self.candidate_peak = self.history[-1]["value"]
        self.duration = time.perf_counter() - start
```

2.1.5 Visualisasi hasil

Visualisasi dibuat menggunakan **Matplotlib** pada **src/visualization.py**. Geladak ditampilkan sebagai bidang persegi panjang, sedangkan tiap kendaraan digambar sesuai ukuran, posisi, dan orientasinya. Posisi (x, y) menunjukkan sudut kiri atas kendaraan. Orientasi H menggunakan dimensi (p, l), sedangkan orientasi V menukarnya menjadi (l, p). ID kendaraan ditampilkan pada gambar, dan kendaraan yang tidak dimuat dicantumkan di bawahnya.

State awal dapat berisi kendaraan yang bertumpuk atau melewati batas geladak. Oleh karena itu, batas sumbu mengikuti posisi kendaraan agar pelanggaran tetap terlihat. Garis hitam menandai batas kapal yang sebenarnya. Warna setiap kendaraan ditentukan berdasarkan urutannya dalam data, sehingga kendaraan yang sama dapat dikenali pada gambar awal dan akhir.

Fungsi **plot_hasil()** menggabungkan gambar *state* awal, *state* akhir, dan grafik perkembangan *objective*. Grafik SA mencatat nilai *objective* setelah setiap iterasi, sehingga kurvanya dapat turun ketika kandidat yang lebih buruk diterima. Pada GA, grafik menampilkan nilai maksimum dan rata-rata populasi. Grafik probabilitas ditampilkan untuk *history* yang mencatat peluang penerimaan kandidat.

Pada SA, probabilitas bernilai 1 ketika kandidat memiliki nilai objective yang sama atau lebih baik, dan mengikuti $p = e^{(\frac{\Delta E}{T})}$ ketika nilainya menurun. Karena selisih nilai setiap kandidat berbeda, grafik probabilitas tidak selalu menurun secara mulus walaupun temperatur terus berkurang.

2.2 Hasil Eksperimen

2.2.1 Konfigurasi Data Uji

Data yang digunakan untuk menguji permasalahan penataan kendaraan di geladak kapal terletak pada **`src/data/data.json`**. Data berisi daftar **30 kendaraan** dengan variasi yang beragam. Nantinya, data ini akan dimuat oleh

`load_data()` yang terdapat pada file **`utils.py`**, lalu diubah menjadi satu objek Kapal dan list objek Kendaraan.

```
{   "kapal": {"dl": 10, "dp": 20, "kapasitas": 100},
    "kendaraan" : [
        {"id": "KDR001", "p": 2, "l": 1, "berat": 4, "fee": 50, "ETA": 1},
        {"id": "KDR002", "p": 3, "l": 2, "berat": 5, "fee": 70, "ETA": 3},
        {"id": "KDR003", "p": 4, "l": 3, "berat": 7, "fee": 100, "ETA": 6},
        {"id": "KDR004", "p": 5, "l": 4, "berat": 6, "fee": 80, "ETA": 3},
        {"id": "KDR005", "p": 6, "l": 5, "berat": 5, "fee": 80, "ETA": 1},
        {"id": "KDR006", "p": 3, "l": 1, "berat": 7, "fee": 90, "ETA": 1},
        {"id": "KDR007", "p": 4, "l": 2, "berat": 8, "fee": 80, "ETA": 4},
        {"id": "KDR008", "p": 5, "l": 3, "berat": 4, "fee": 20, "ETA": 3},
        {"id": "KDR009", "p": 6, "l": 4, "berat": 5, "fee": 70, "ETA": 6},
        {"id": "KDR010", "p": 4, "l": 1, "berat": 7, "fee": 60, "ETA": 4},
        {"id": "KDR011", "p": 5, "l": 2, "berat": 8, "fee": 100, "ETA": 2},
        {"id": "KDR012", "p": 6, "l": 3, "berat": 4, "fee": 90, "ETA": 6},
        {"id": "KDR013", "p": 2, "l": 1, "berat": 3, "fee": 60, "ETA": 2},
        {"id": "KDR014", "p": 3, "l": 2, "berat": 5, "fee": 70, "ETA": 2},
        {"id": "KDR015", "p": 4, "l": 3, "berat": 6, "fee": 60, "ETA": 7},
        {"id": "KDR016", "p": 5, "l": 4, "berat": 7, "fee": 90, "ETA": 4},
        {"id": "KDR017", "p": 6, "l": 5, "berat": 9, "fee": 30, "ETA": 5},
        {"id": "KDR018", "p": 3, "l": 1, "berat": 2, "fee": 60, "ETA": 5},
        {"id": "KDR019", "p": 4, "l": 2, "berat": 4, "fee": 20, "ETA": 7},
        {"id": "KDR020", "p": 5, "l": 3, "berat": 6, "fee": 80, "ETA": 3},
        {"id": "KDR021", "p": 6, "l": 4, "berat": 7, "fee": 70, "ETA": 6},
        {"id": "KDR022", "p": 4, "l": 1, "berat": 9, "fee": 40, "ETA": 2},
        {"id": "KDR023", "p": 5, "l": 2, "berat": 8, "fee": 90, "ETA": 1},
        {"id": "KDR024", "p": 6, "l": 3, "berat": 3, "fee": 60, "ETA": 1},
        {"id": "KDR025", "p": 1, "l": 1, "berat": 2, "fee": 40, "ETA": 1},
        {"id": "KDR026", "p": 2, "l": 2, "berat": 5, "fee": 30, "ETA": 1},
        {"id": "KDR027", "p": 3, "l": 3, "berat": 7, "fee": 100, "ETA": 1},
        {"id": "KDR028", "p": 4, "l": 4, "berat": 4, "fee": 60, "ETA": 1},
        {"id": "KDR029", "p": 5, "l": 5, "berat": 6, "fee": 40, "ETA": 1},
        {"id": "KDR030", "p": 6, "l": 6, "berat": 9, "fee": 80, "ETA": 1}
    ]
}
```

2.2.2 Hill-Climbing Steepest Ascent

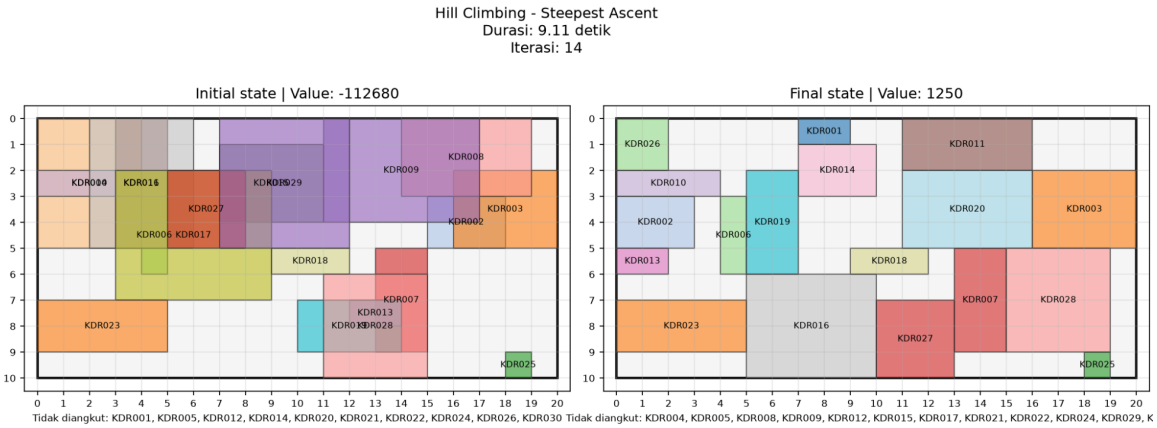
Percobaan dilakukan dengan menggunakan seed 21 pada run 1, seed 42 pada run 2, dan seed 63 pada run 3. Berikut merupakan hasilnya dalam bentuk tabular:

Tabel 2.2.2 Hasil Percobaan HC Steepest Ascent

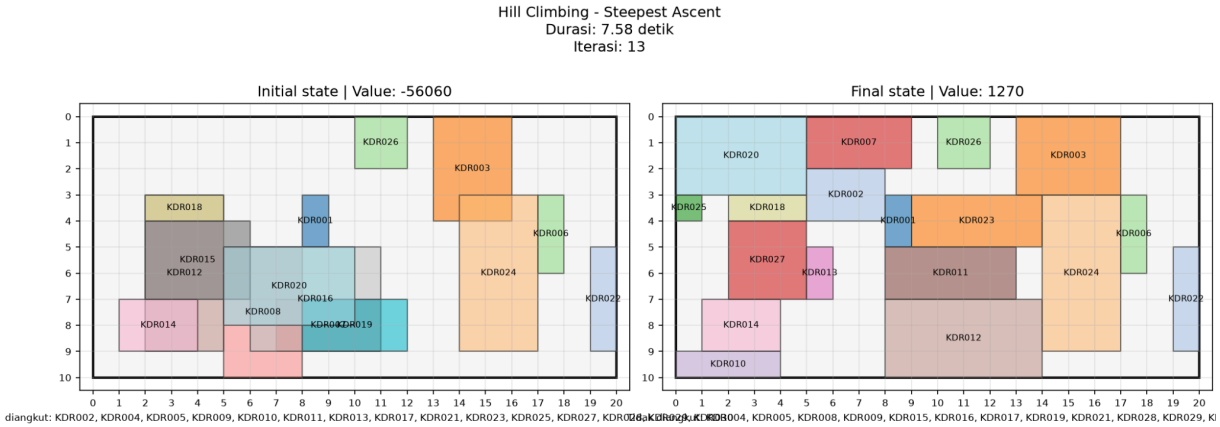
Run	Objective Awal	Objective Akhir	Jumlah Iterasi	Durasi (s)
1	-112680	1250	14	9.11
2	-56060	1270	13	7.58
3	-37110	1310	12	7.14

Ketiga percobaan menghasilkan susunan akhir yang valid di mana tidak ada kondisi di mana kapal mengangkut kendaran hingga melebihi kapasitas, kendaraan overlap atau kendaraan yang keluar dari batas di geladak.

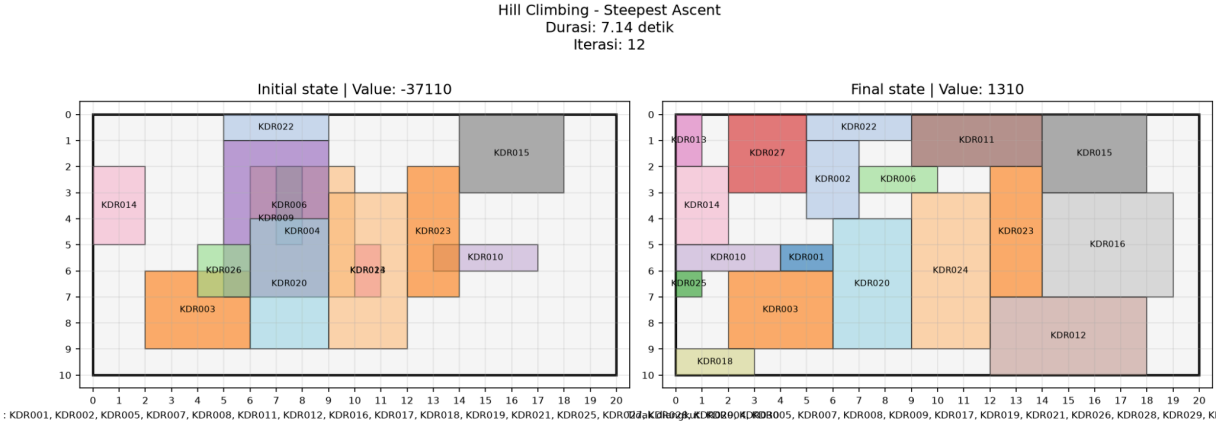
Visualisasi percobaan tersimpan pada **`output/hc_steepest/hc-sa_run1`** hingga **`hc-sa_run3.`** Berikut merupakan perbandingan susunan kendaraan sebelum dan sesudah pencarian untuk ketiga seed.



Gambar hc-sa_run1. Initial dan Final state untuk run dengan Seed 21

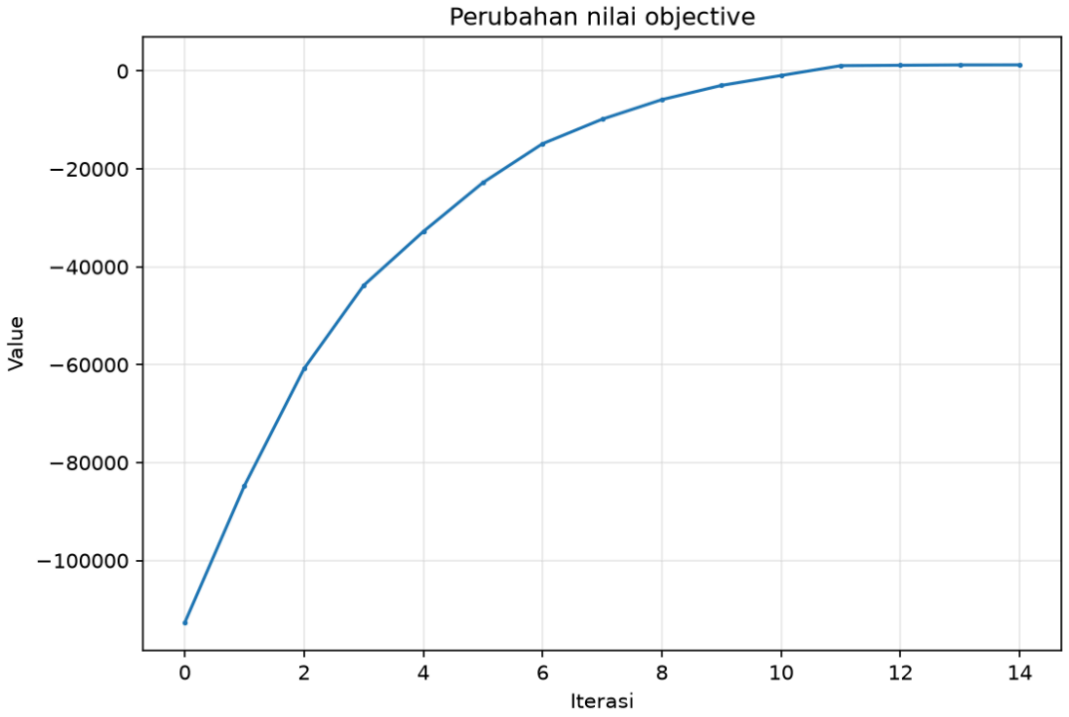


Gambar hc-sa_run2. Initial dan Final state untuk run dengan Seed 42

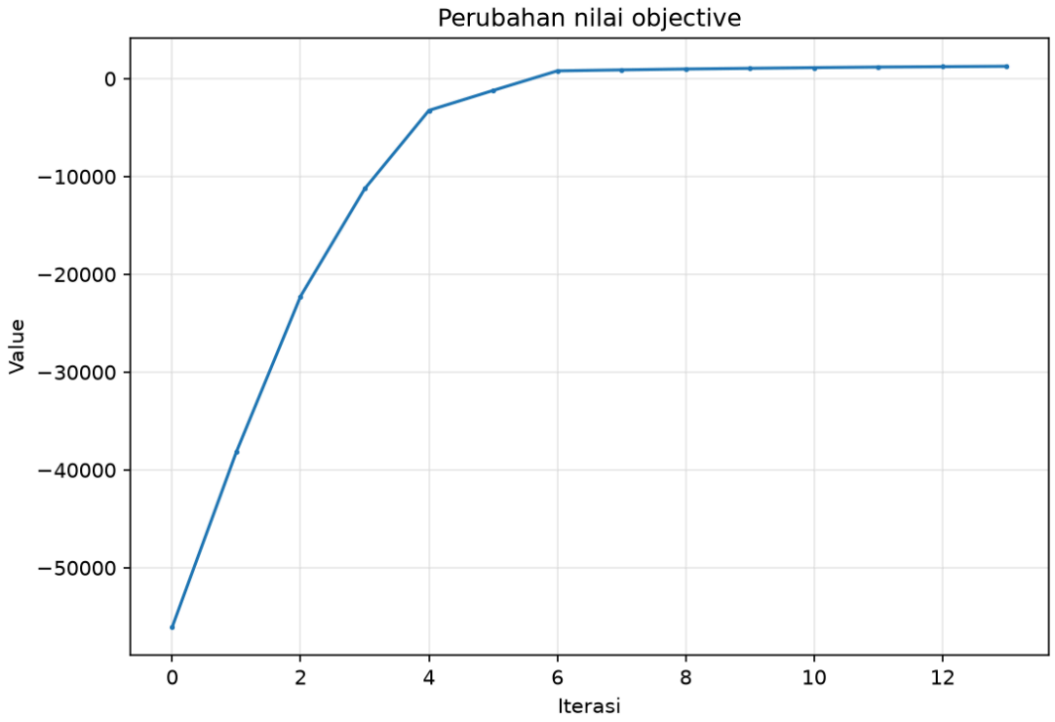


Gambar hc-sa_run3. Initial dan Final state untuk run dengan Seed 63

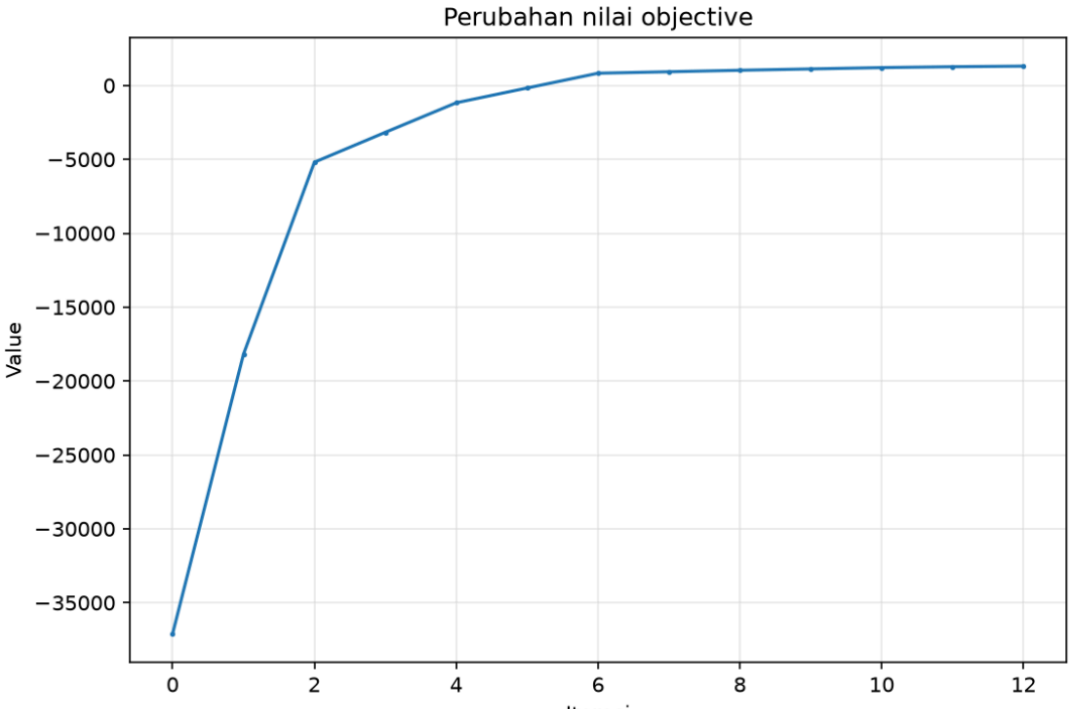
Gambar berikut menunjukkan perubahan nilai *objective function* tiap iterasi pada masing-masing *run*



Gambar hc-objective_run1



Gambar hc-objective_run2



Gambar hc-objective_run3

Pada *history*, nilai *objective function* menunjukkan peningkatan yang monoton pada ketiga run dan tidak pernah turun. Hal ini sesuai dengan karakteristik Steepest Ascent yang hanya menerima *neighbor* dengan nilai *objective* lebih tinggi.

Jika diteliti lebih lanjut, kenaikan terjadi dalam dua fase. Fase pertama, ketika nilai objective masih bernilai negatif dan naik tajam karena Steepest Ascent berusaha menghilangkan setiap pelanggaran. Nilai *objective* baru bernilai positif sejak iterasi ke-6. Lalu pada fase kedua, *state* sudah dianggap lebih valid dan kenaikan nilai *objective* landai karena Steepest Ascent hanya memperbaiki total fee. Kemudian, kurva terlihat mendatar saat tidak ada neighbor yang lebih baik dan kondisi akhirnya diterminasi.

2.2.3 Simulated Annealing

Percobaan SA menggunakan data pada **src/data/data.json**.

- Geladak berukuran 20 x 10 sel dengan kapasitas berat 100.
- Data terdiri atas 30 kendaraan, dari KDR001 sampai KDR030, dengan panjang dan lebar 1 sampai 6 sel, berat 2 sampai 9, fee 20 sampai 100, serta ETA 1 sampai 7.

Semua percobaan menggunakan **initial_temperature = 1000**, **cooling_rate = 0.998**, dan **minimum_temperature = 0.1**. Seed 21, 42, dan 63 masing-masing dipakai pada run 1, run 2, dan run 3 untuk menentukan *state* awal dan *random choice* selama pencarian.

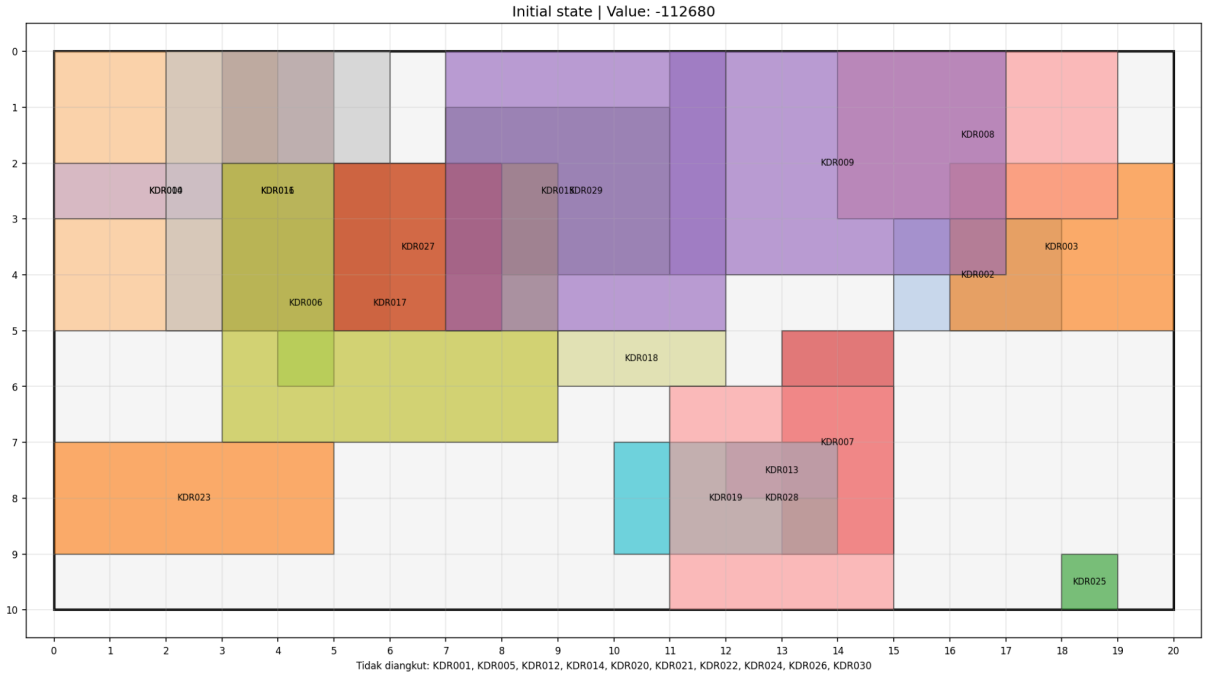
Tabel 2.2.3 Hasil Percobaan Simulated Annealing

Run	Objective Awal	Objective Akhir	Jumlah Iterasi	Durasi (s)	Langkah buruk ditolak
1	-112680	1170	4601	21,1624	4212
2	-56060	1240	4601	20,4528	4292
3	-37110	1210	4601	21,4031	4244

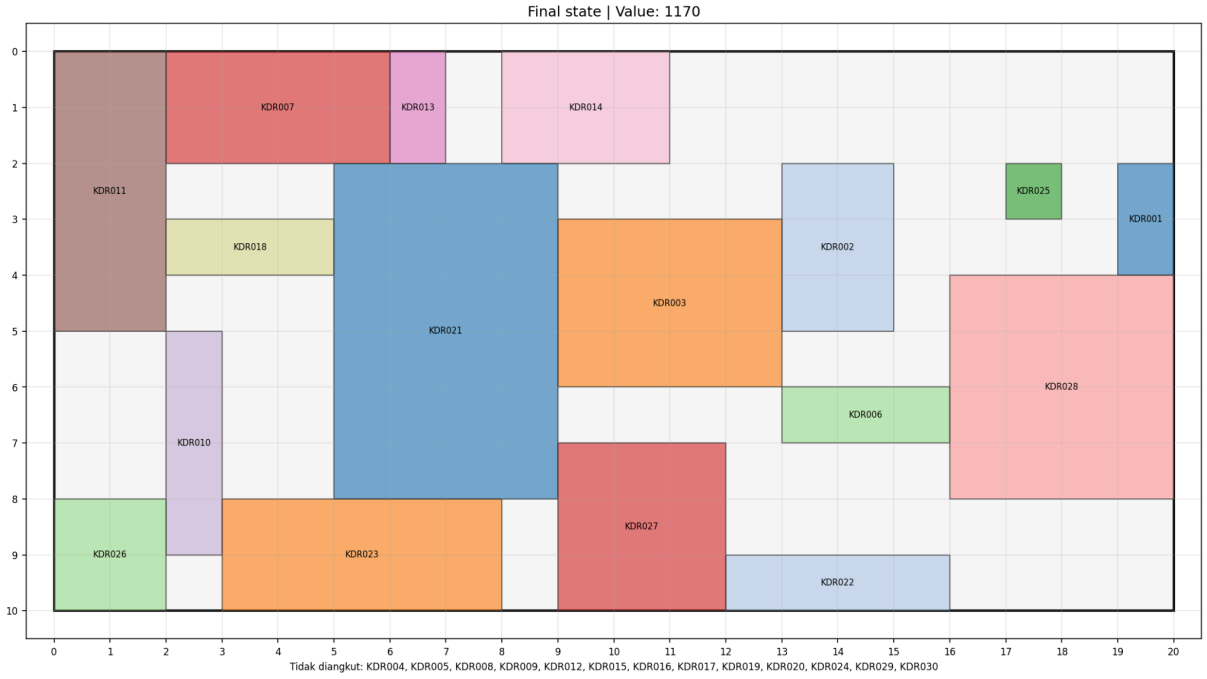
Tabel berikut memuat hasil tiga percobaan SA yang tersimpan pada **output/simulated_annealing/SA_run1** sampai **SA_run3**. Kolom terakhir berarti jumlah kandidat dengan nilai *objective* lebih rendah yang ditolak. Durasi yang dicantumkan merupakan waktu pencarian algoritma pada saat eksperimen dijalankan dan dapat berbeda bergantung pada kondisi sistem.

Ketiga percobaan menghasilkan susunan akhir yang valid, tidak ada kelebihan kapasitas, *overlap*, atau sel kendaraan yang keluar dari kapal. Oleh karena itu, nilai *objective* akhir = *total fee*.

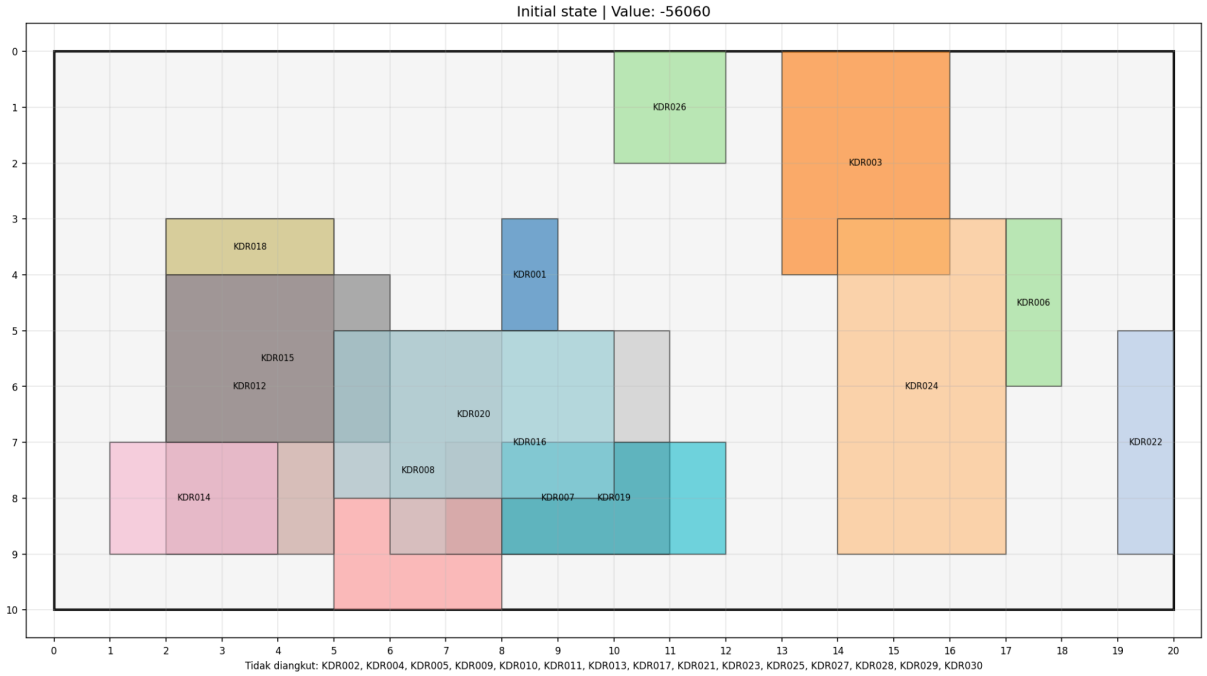
Gambar SA-1 sampai SA-6 memperlihatkan perbandingan susunan kendaraan sebelum dan sesudah pencarian untuk ketiga *seed*.



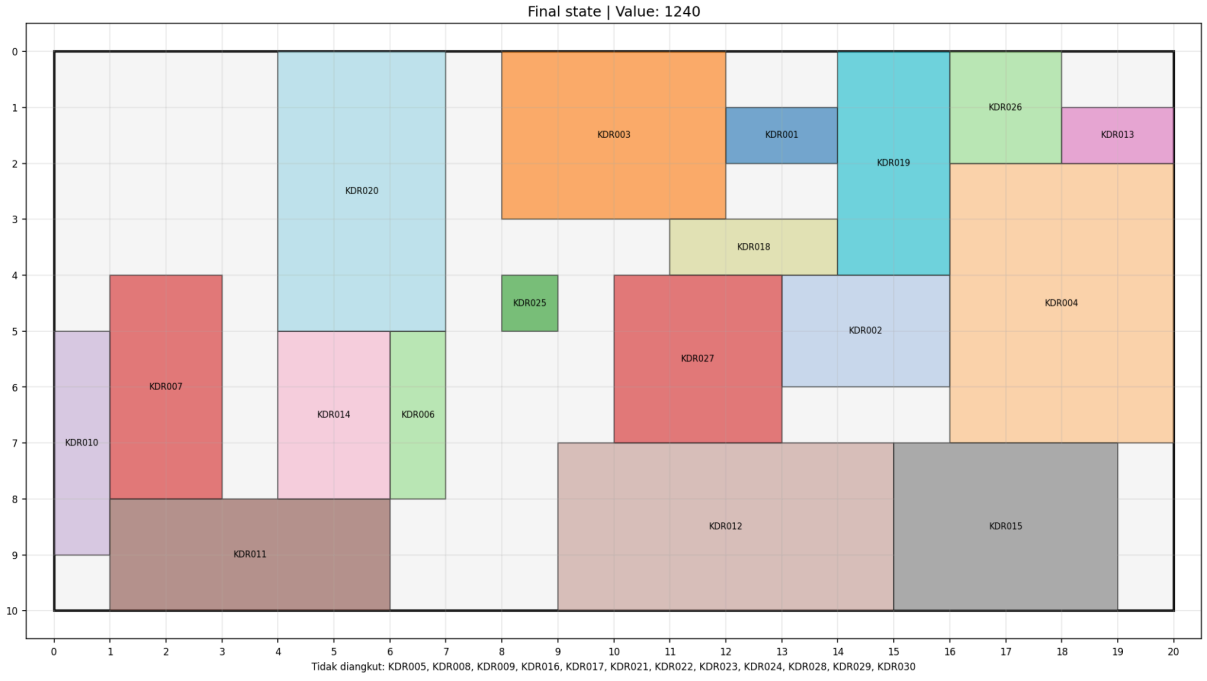
Gambar SA-1. State awal run 1, seed 21, objective −112680



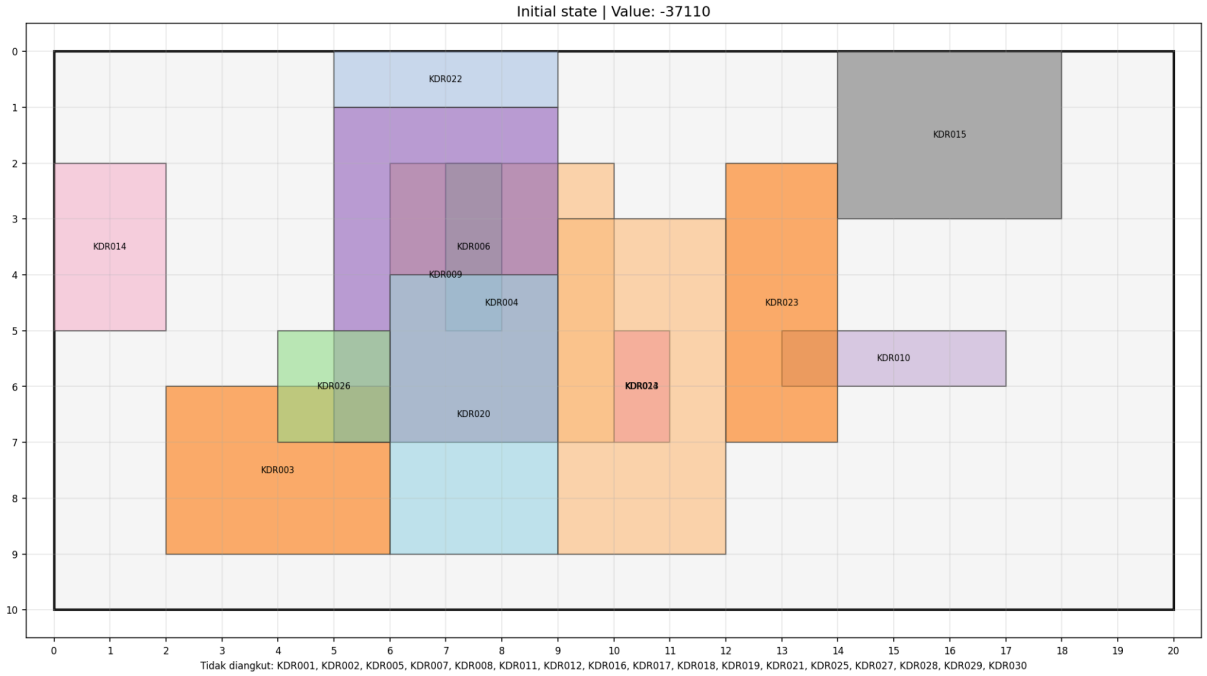
Gambar SA-2. State akhir run 1, 17 kendaraan, objective 1170



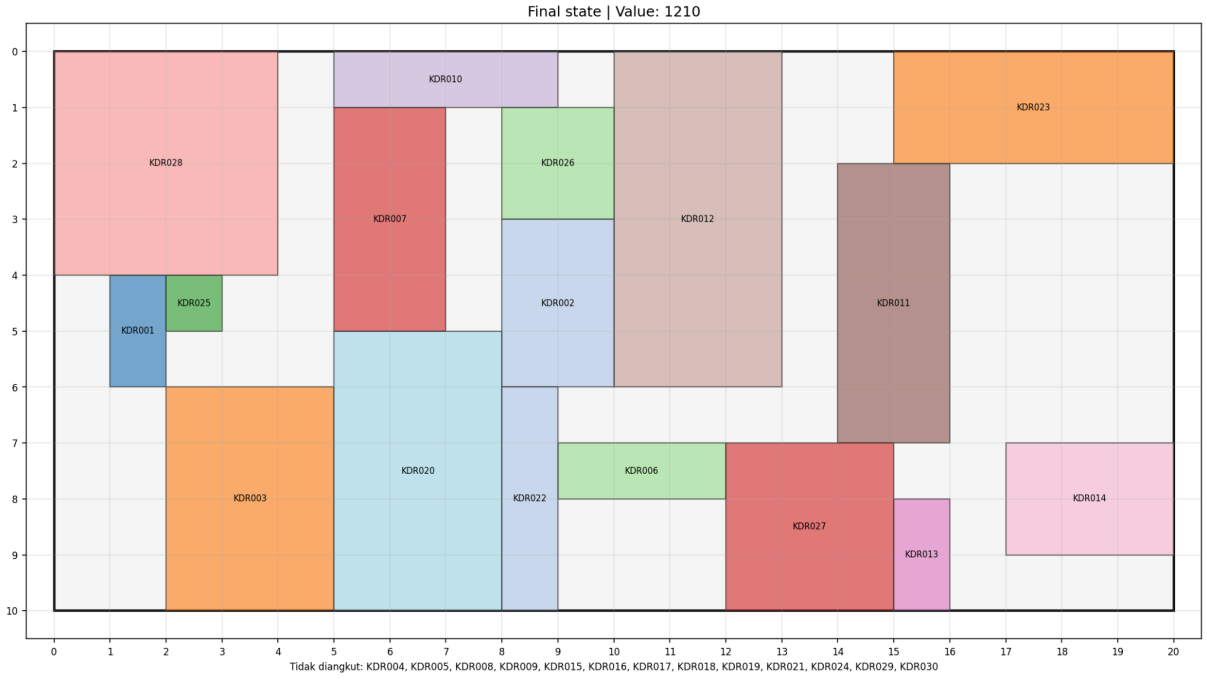
Gambar SA-3. State awal run 2, seed 42, objective −56060



Gambar SA-4. State akhir run 2, 18 kendaraan, objective 1240.

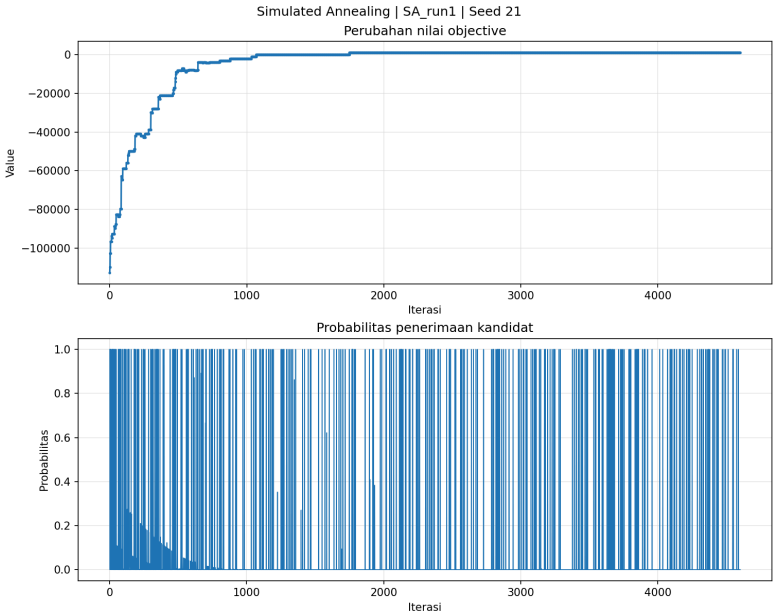


Gambar SA-5. State awal run 3, seed 63, objective −37110

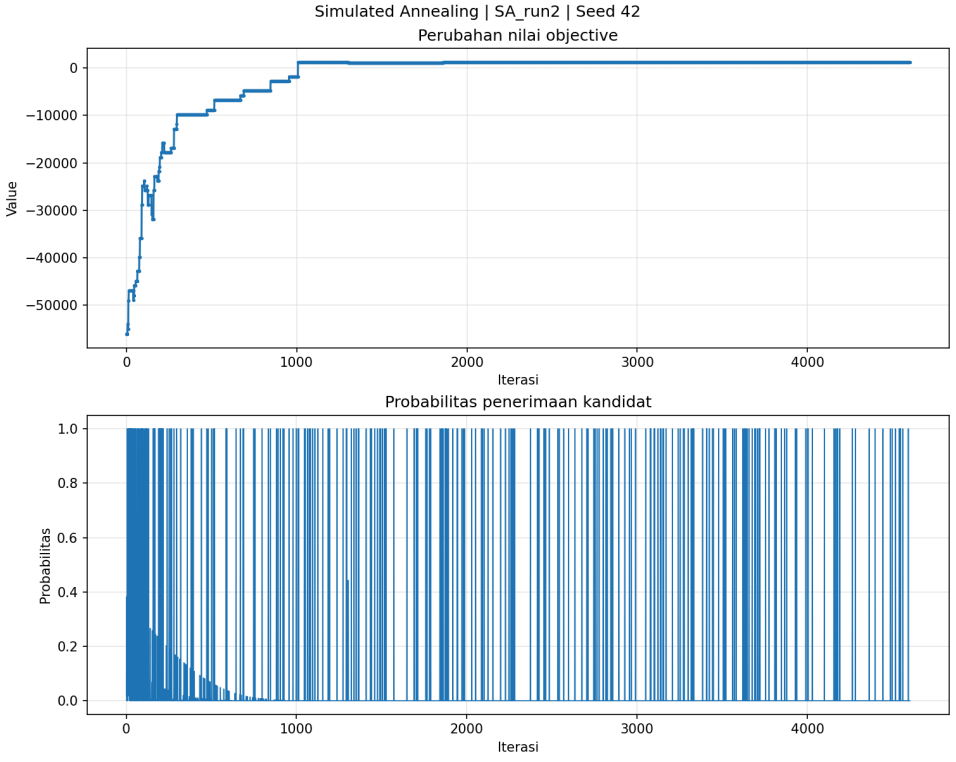


Gambar SA-6. State akhir run 3, 17 kendaraan, objective 1210

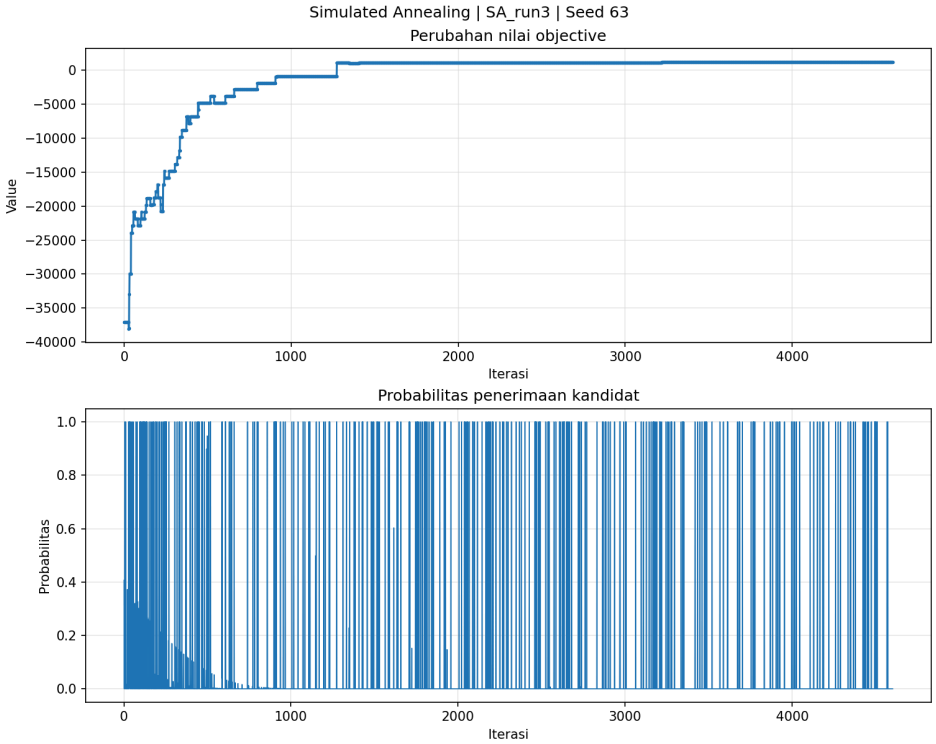
Gambar SA-7 sampai SA-9 memperlihatkan perubahan objective pada panel atas dan probabilitas penerimaan kandidat pada panel bawah untuk masing-masing percobaan.



Gambar SA-7. History objective dan probabilitas penerimaan run 1.



Gambar SA-8. History objective dan probabilitas penerimaan run 2.



Gambar SA-9. History objective dan probabilitas penerimaan run 3.

Pada *history*, nilai *objective* turun sebanyak 17 kali pada *run* 1, 11 kali pada *run* 2, dan 15 kali pada *run* 3. Penurunan ini menunjukkan bahwa SA beberapa kali menerima neighbor dengan *objective* lebih rendah karena probabilitas penerimaan masih memungkinkan, terutama pada temperatur yang lebih tinggi. Artinya, SA memang sempat menerima kandidat yang lebih buruk. Bagian kurva yang datar belum tentu berarti pencarian berhenti, karena kandidat bisa saja ditolak, atau state berpindah ke kandidat lain dengan nilai *objective* yang sama.

2.2.4 Genetic Algorithm

Eksperimen A: Jumlah populasi sebagai kontrol, banyak iterasi divariasikan

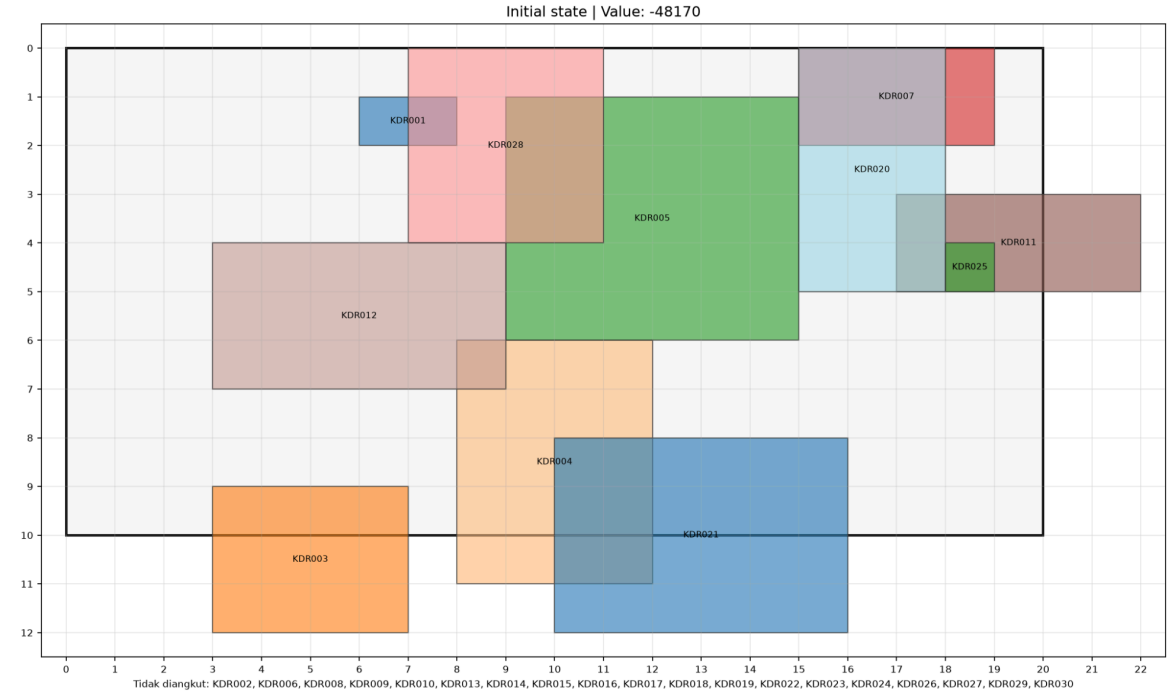
Konfigurasi	Run	Populasi	Iterasi	Objective Akhir	Durasi (s)
A1	1	30	50	1140	0.2018
A1	2	30	50	1020	0.163
A1	3	30	50	860	0.3405
A2	1	30	100	1210	0.3038

A2	2	30	100	1100	0.3294
A2	3	30	100	920	0.7615
A3	1	30	200	1210	0.6576
A3	2	30	200	1100	0.6732
A3	3	320	200	1000	0.1955

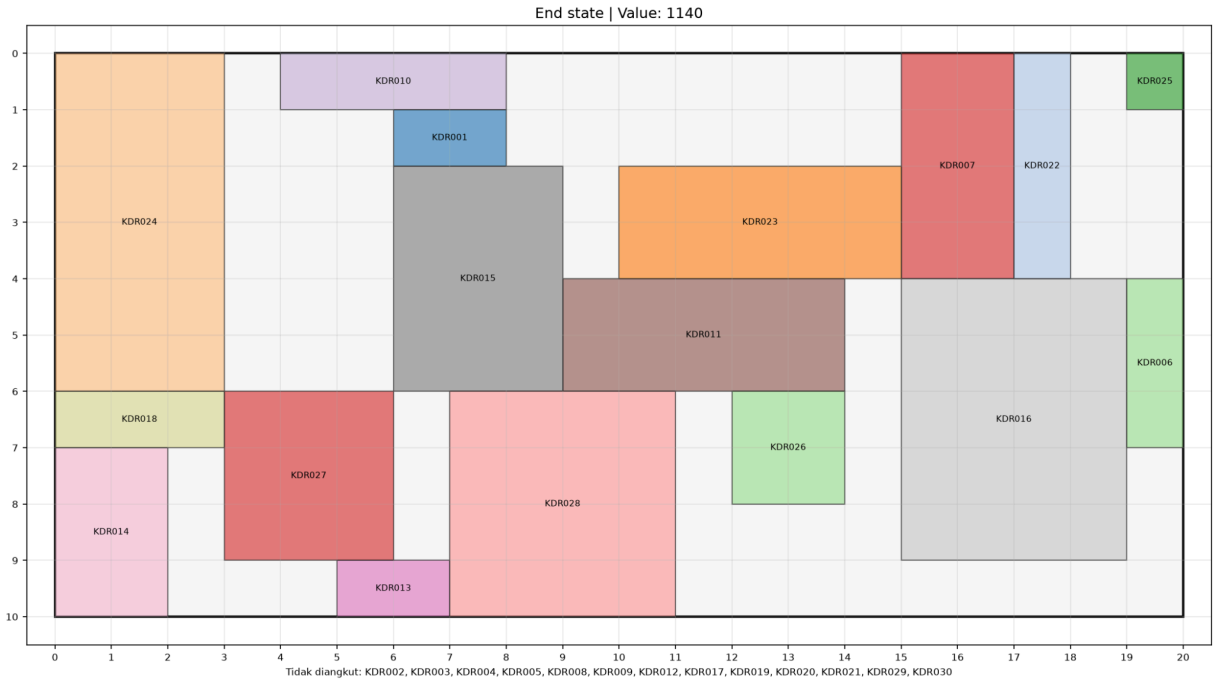
Eksperimen B: Banyak iterasi sebagai kontrol, jumlah populasi divariasikan

Konfigurasi	Run	Populasi	Iterasi	Objective Akhir	Durasi (s)
B1	1	20	100	1080	0.1955
B1	2	20	100	930	0.1864
B1	3	20	100	1160	0,1895
B2	1	50	100	1180	0.5406
B2	2	50	100	1210	0.6028
B2	3	50	100	1010	0.5395
B3	1	100	100	1110	0.0049
B3	2	100	100	1160	0.0084
B3	3	100	100	1080	0.627

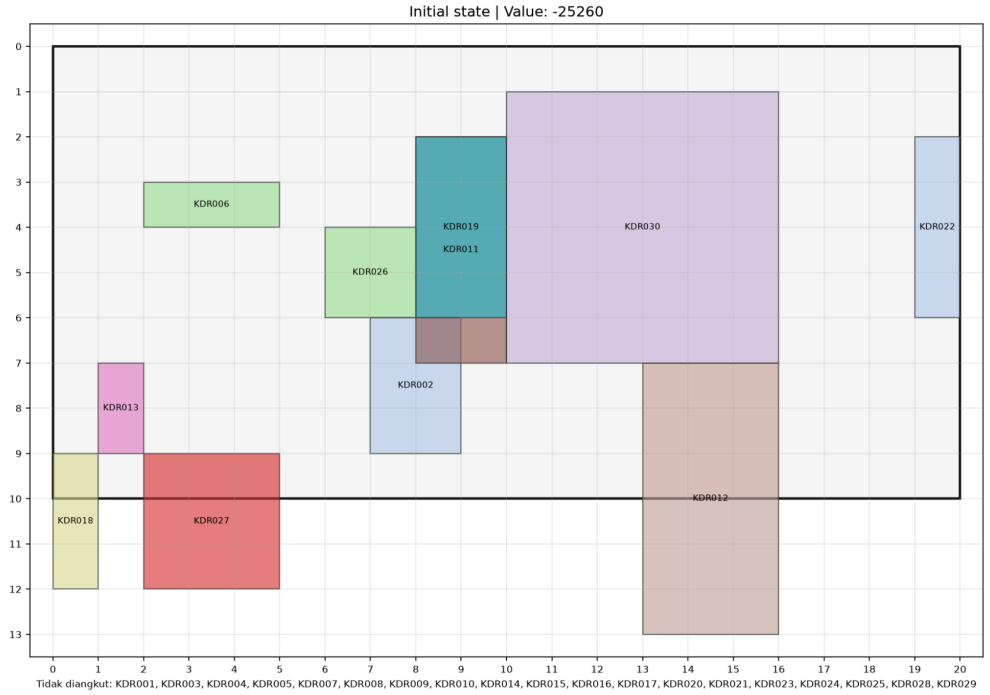
- State awal dan akhir tiap eksperimen:



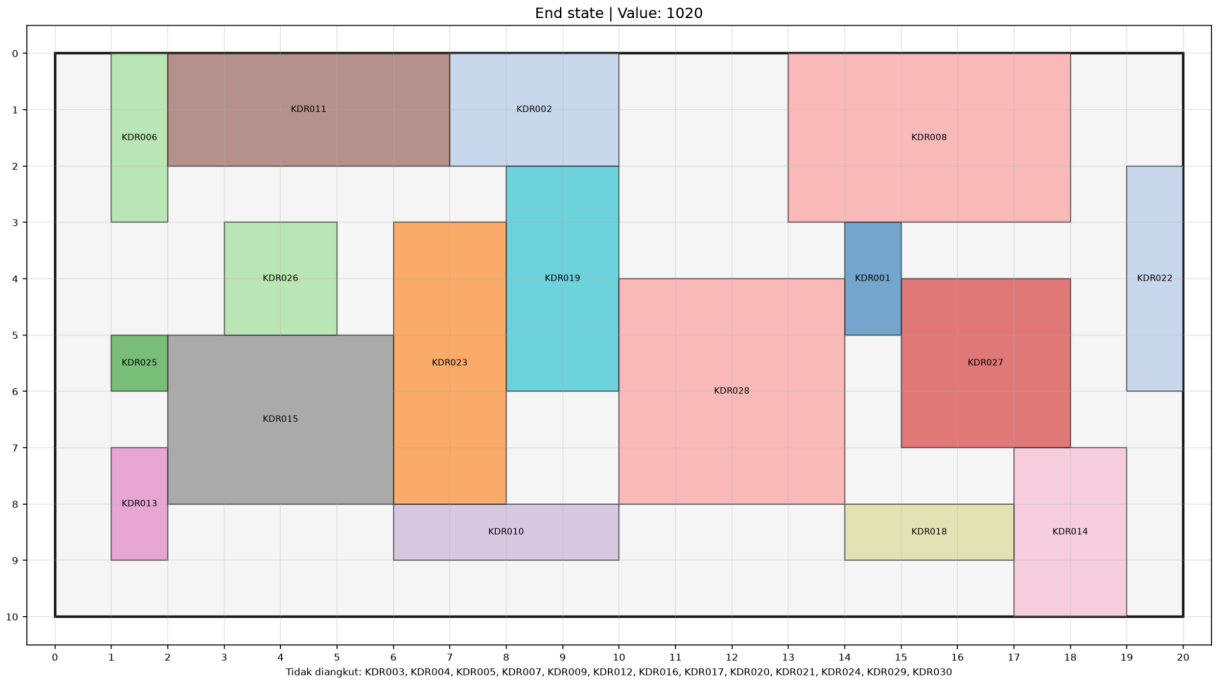
Gambar Initial State G-A1-1. Seed 5, pop 30, iter 50. Value : -48170



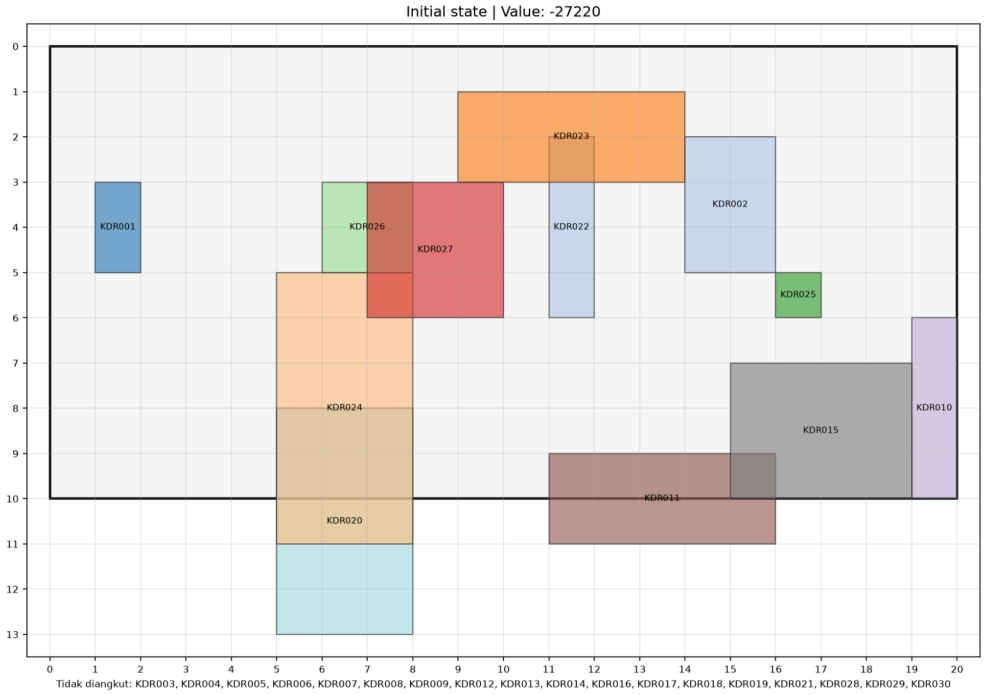
Gambar End State G-A1-1. Seed 5, pop 30, iter 50. Value : 1140



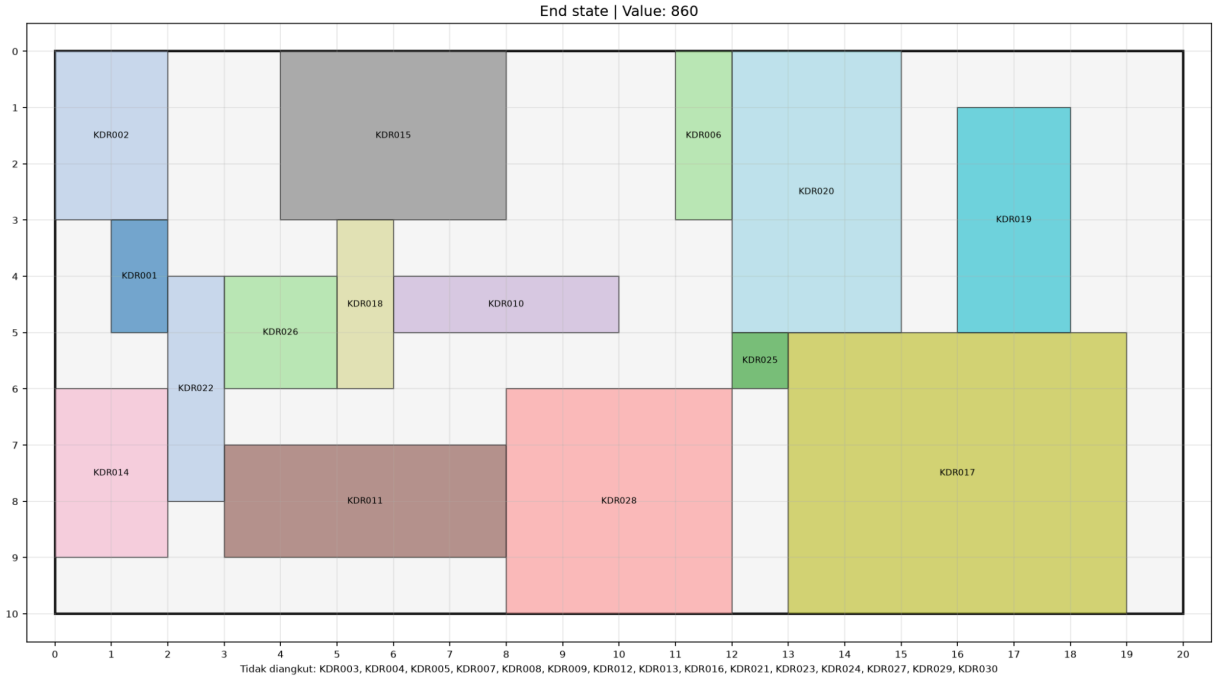
Gambar Initial State G-A1-2. Seed 6, pop 30, iter 50. Value : -25260



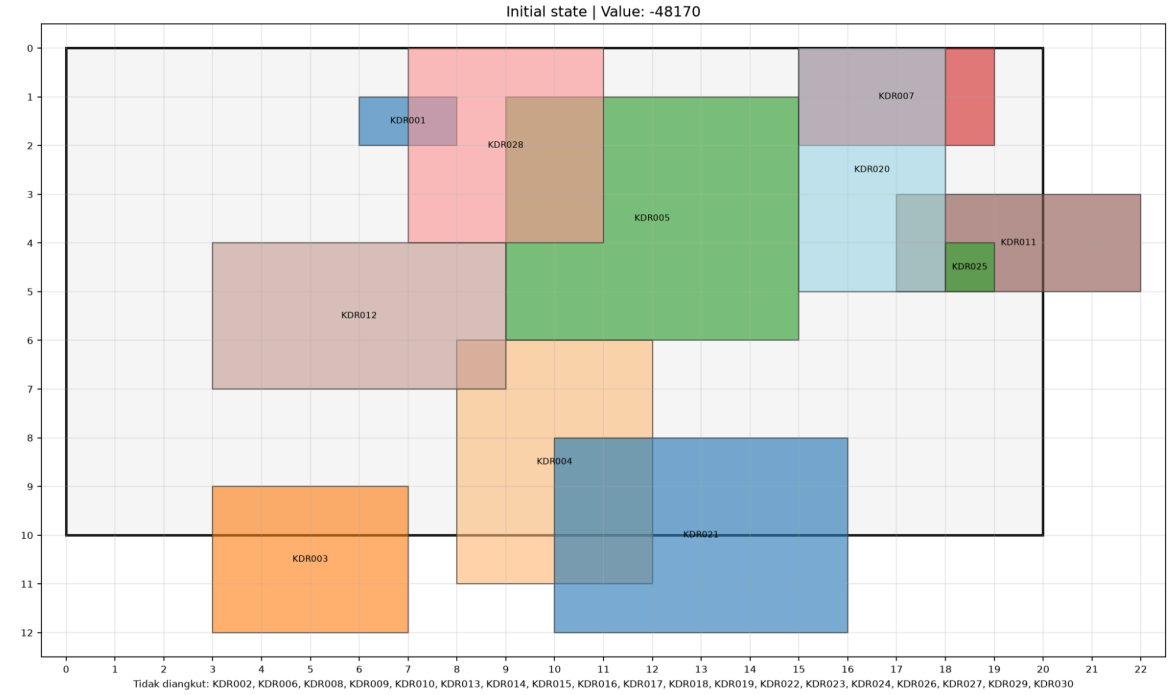
Gambar End State G-A1-2. Seed 6, pop 30, iter 50. Value : 1020



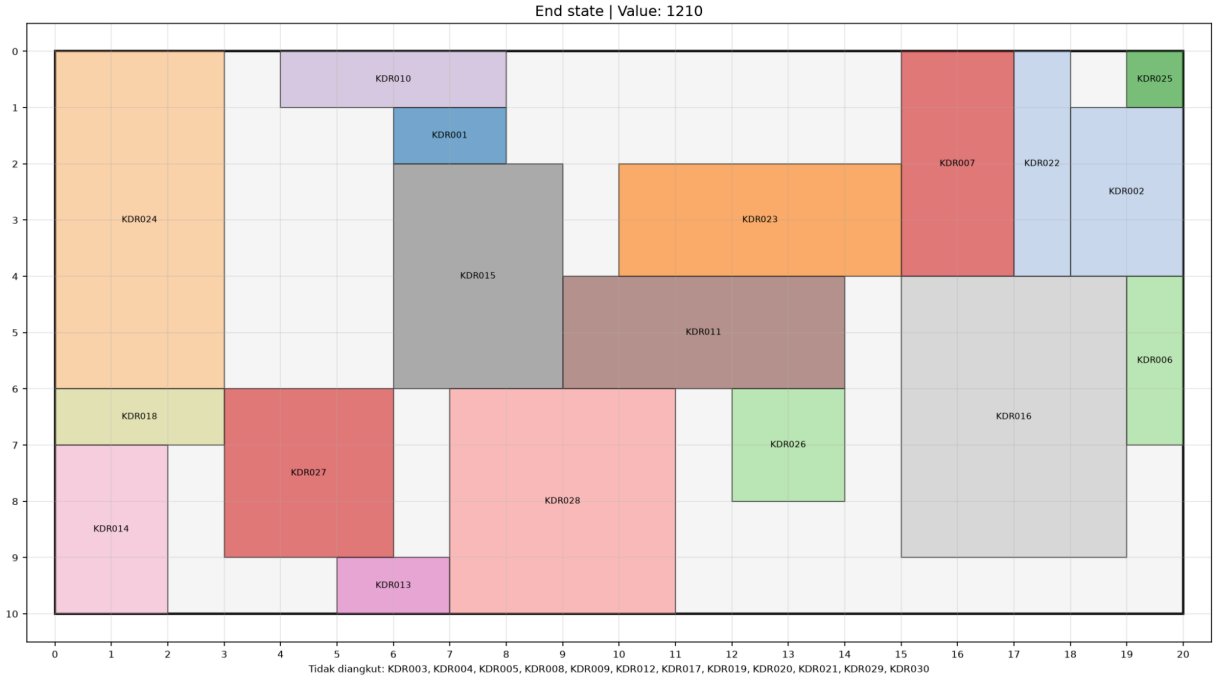
Gambar Initial State G-A1-3. Seed 7, pop 30, iter 50. Value : -27220



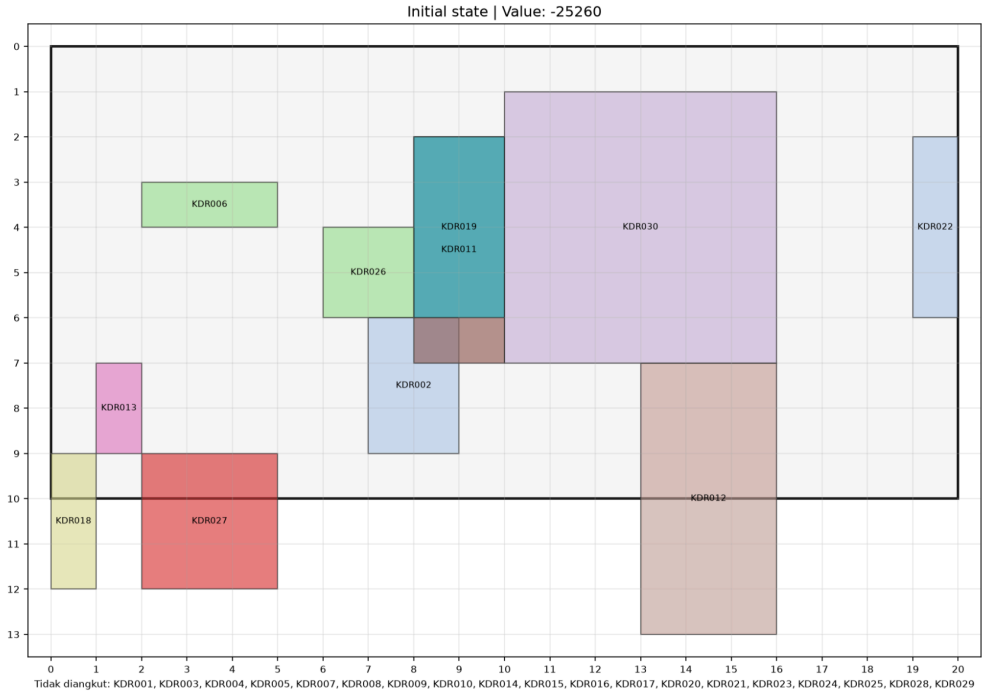
Gambar End State G-A1-3. Seed 7, pop 30, iter 0. Value : 860



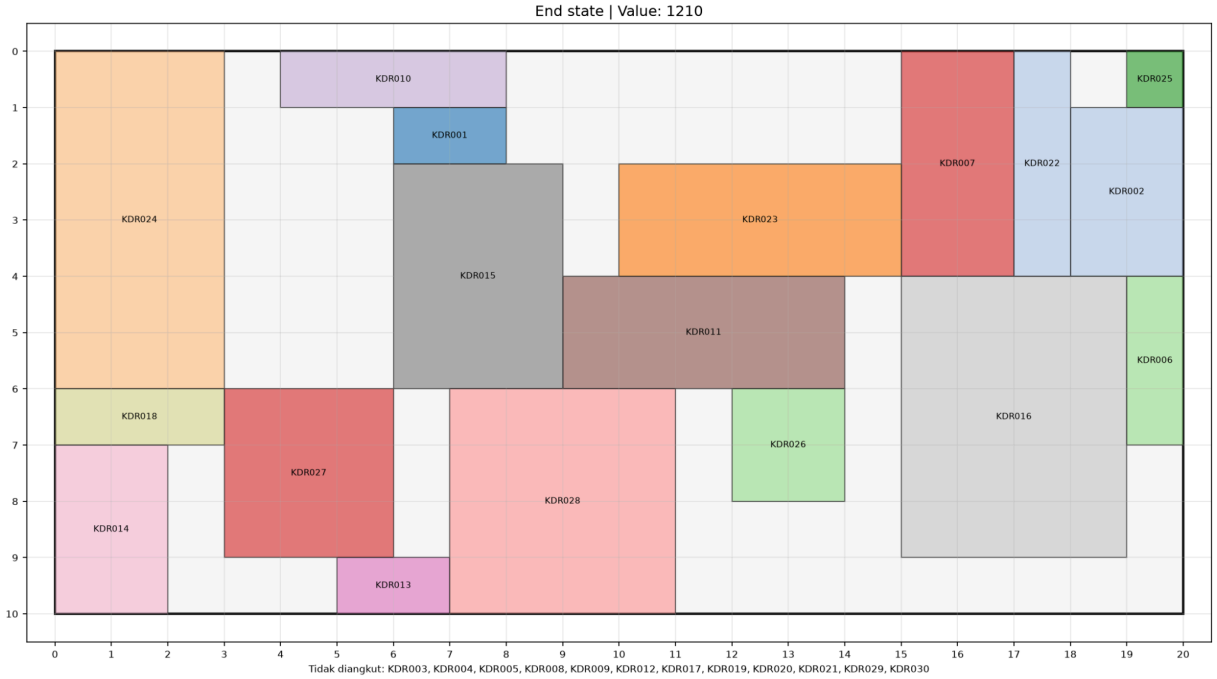
Gambar Initial State G-A2-1. Seed 5, pop 30, iter 100. Value : -48170



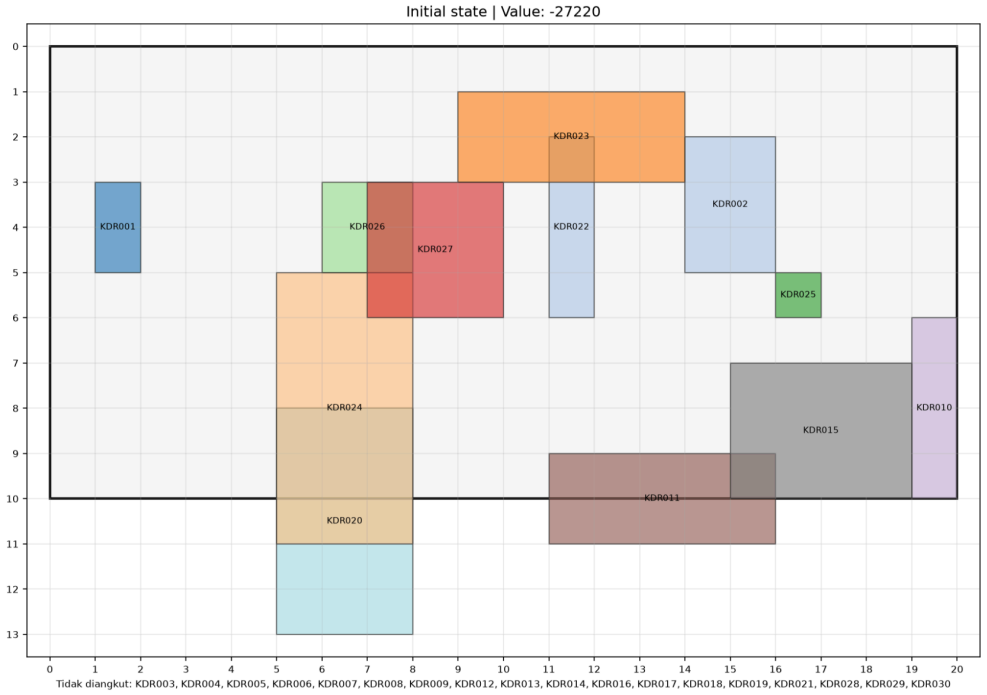
Gambar End State G-A2-1. Seed 5, pop 30, iter 100. Value : 1210



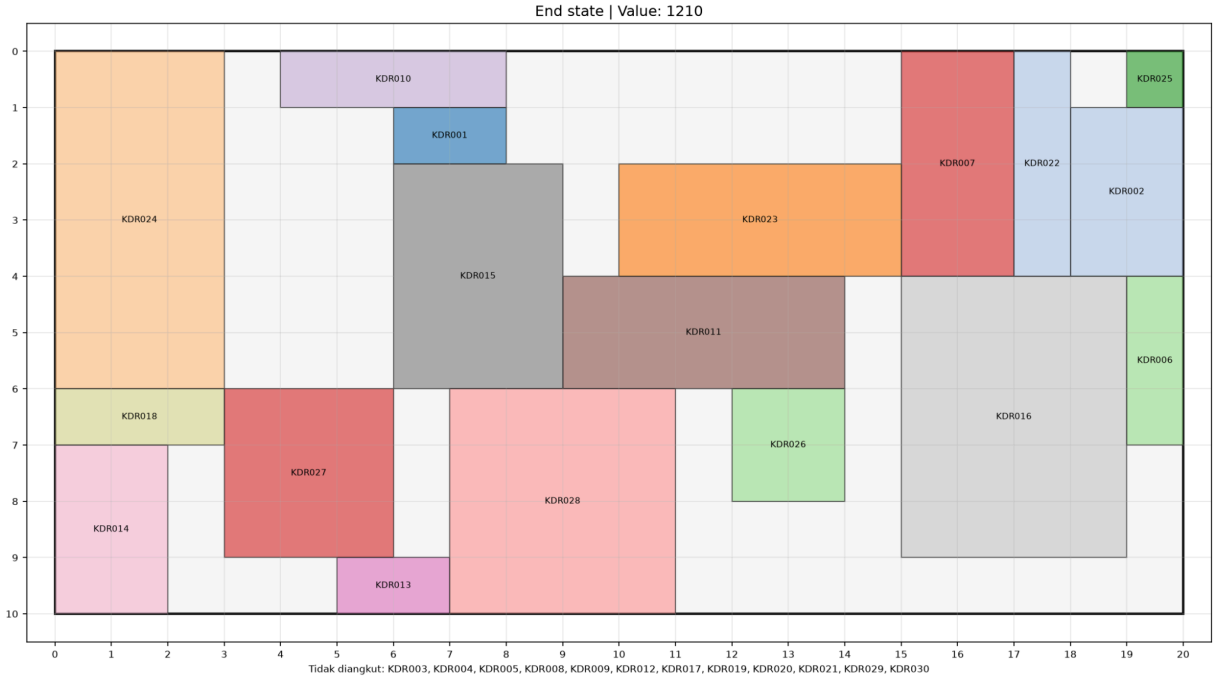
Gambar Initial State G-A2-2. Seed 6, pop 30, iter 100. Value : -25260



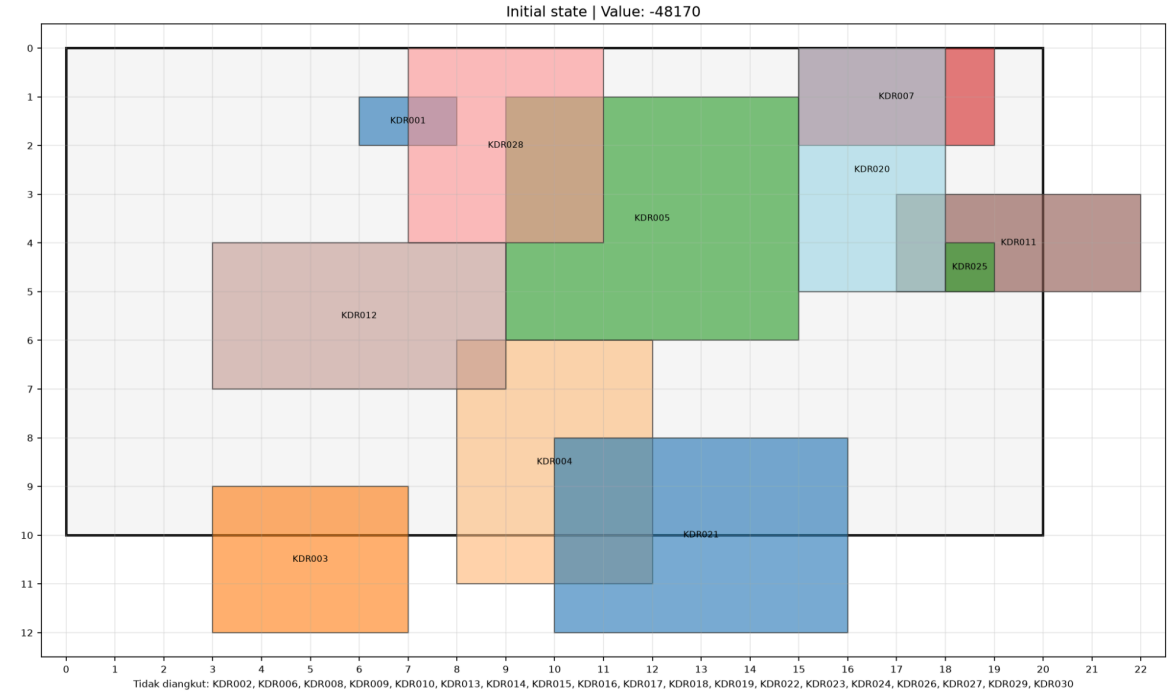
Gambar End State G-A2-2. Seed 6, pop 30, iter 100. Value : 1100



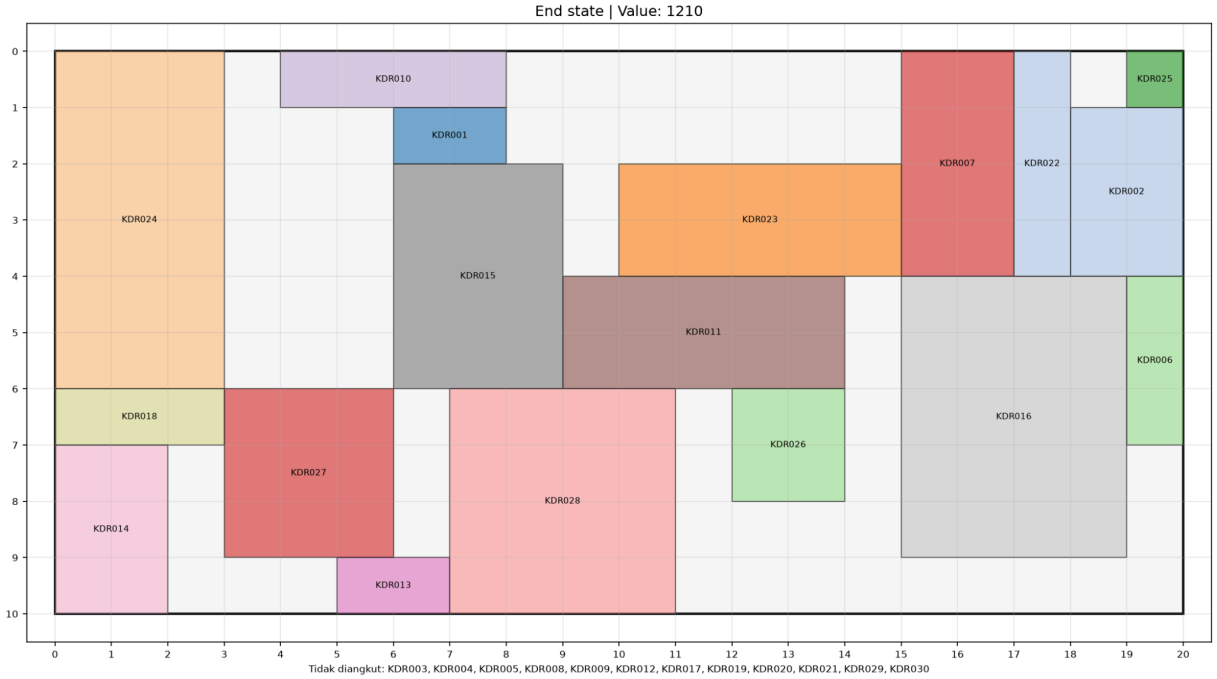
Gambar Initial State G-A2-3. Seed 7, pop 30, iter 100. Value : -27220



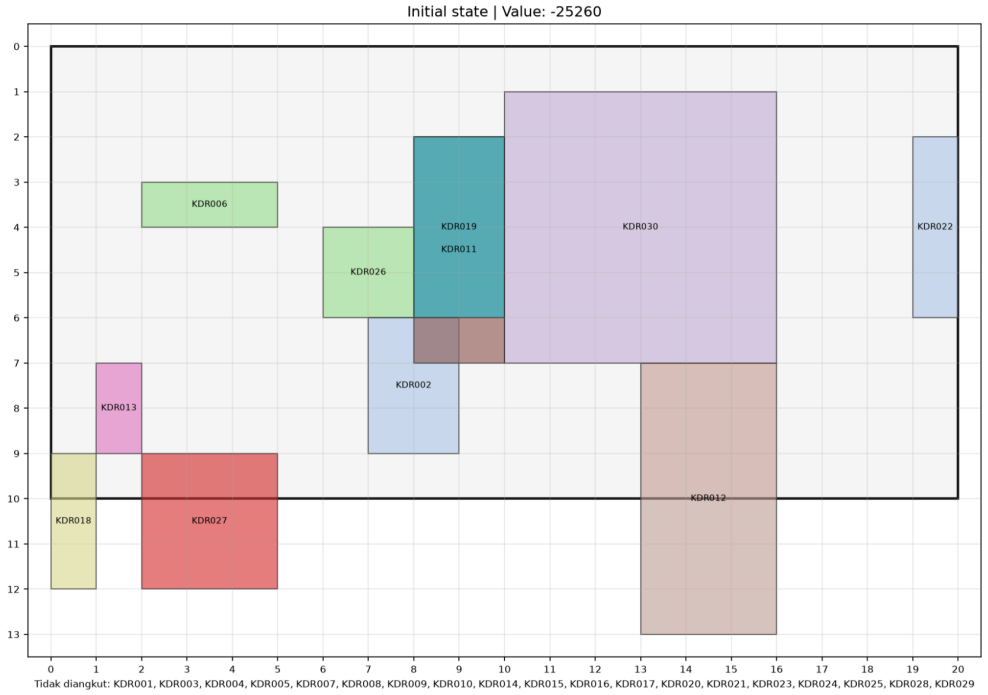
Gambar End State G-A2-3. Seed 7, pop 30, iter 100. Value : 920



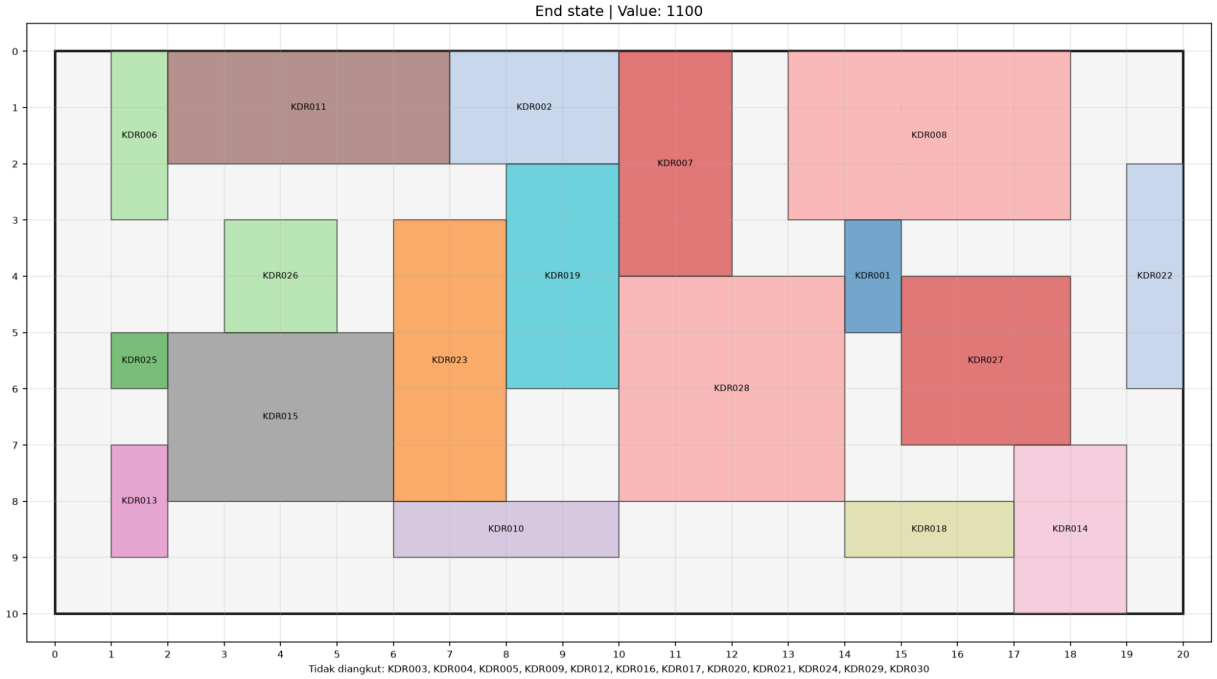
Gambar Initial State G-A3-1. Seed 5, pop 30, iter 200. Value : -48170



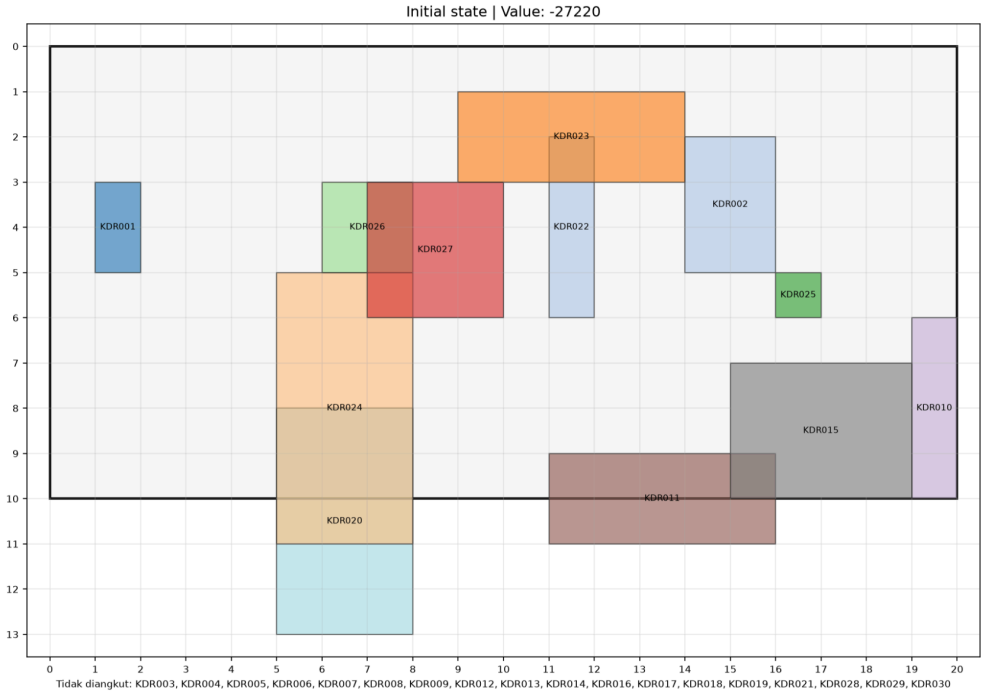
Gambar End State G-A3-1. Seed 5, pop 30, iter 200. Value : 1210



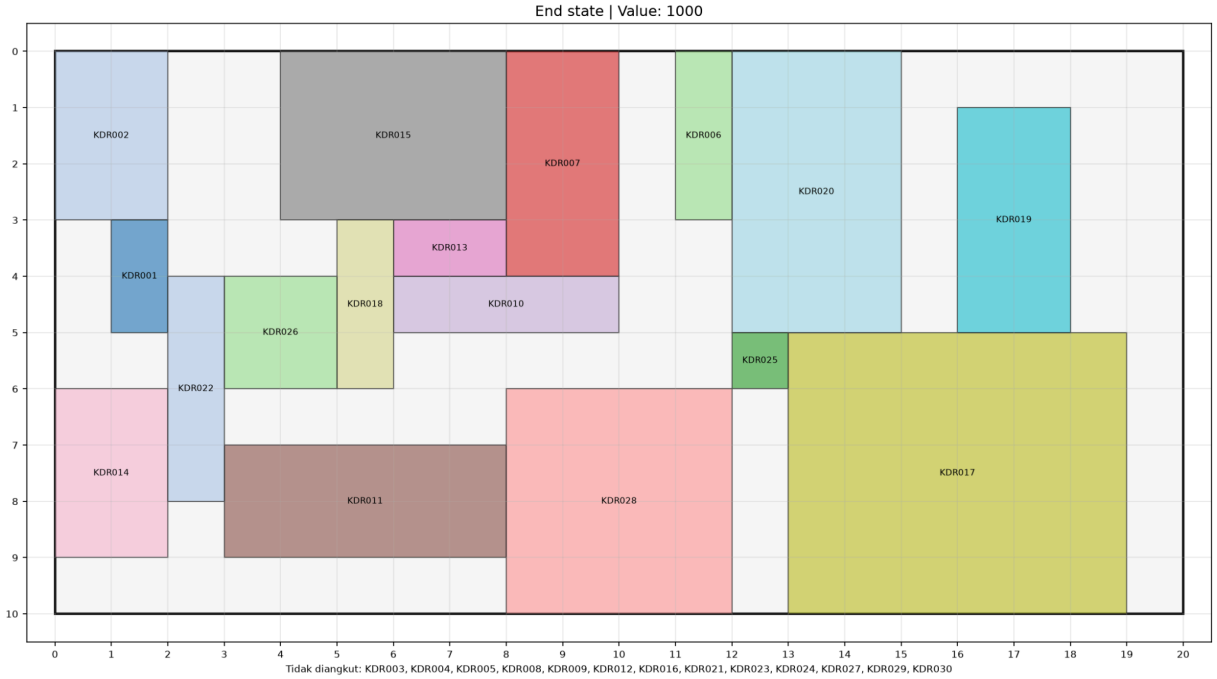
Gambar Initial State G-A3-2. Seed 6, pop 30, iter 200. Value : -25260



Gambar End State G-A3-2. Seed 6, pop 30, iter 200. Value : 1100

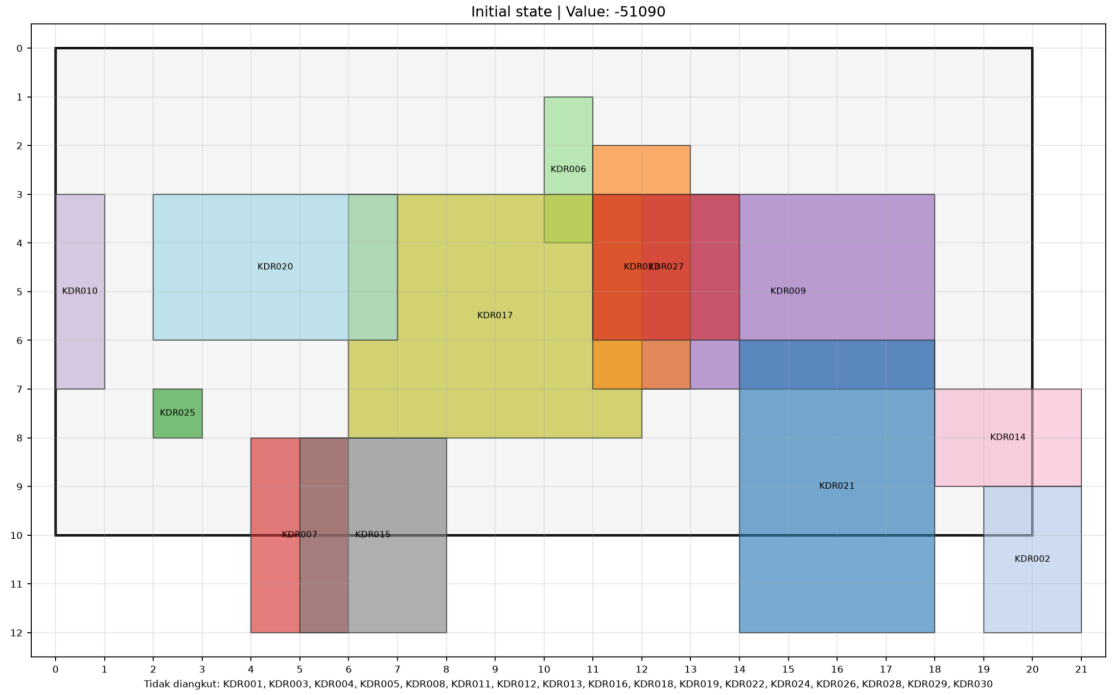


Gambar Initial State G-A3-3. Seed 7, pop 30, iter 200. Value : -27220

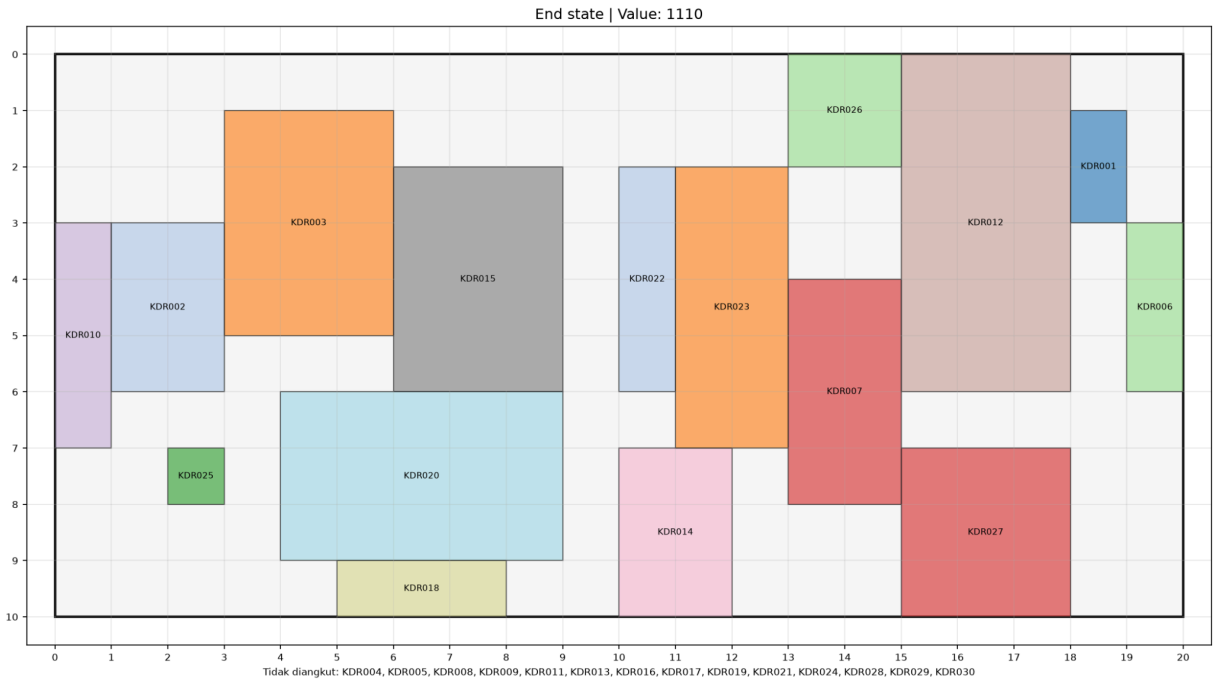


Gambar End State G-A3-3. Seed 7, pop 30, iter 200. Value : 1000

- Paramater Kontrol: Iterasi



Gambar Initial State G-B1-1. Seed 5, pop 20, iter 100. Value : -51090



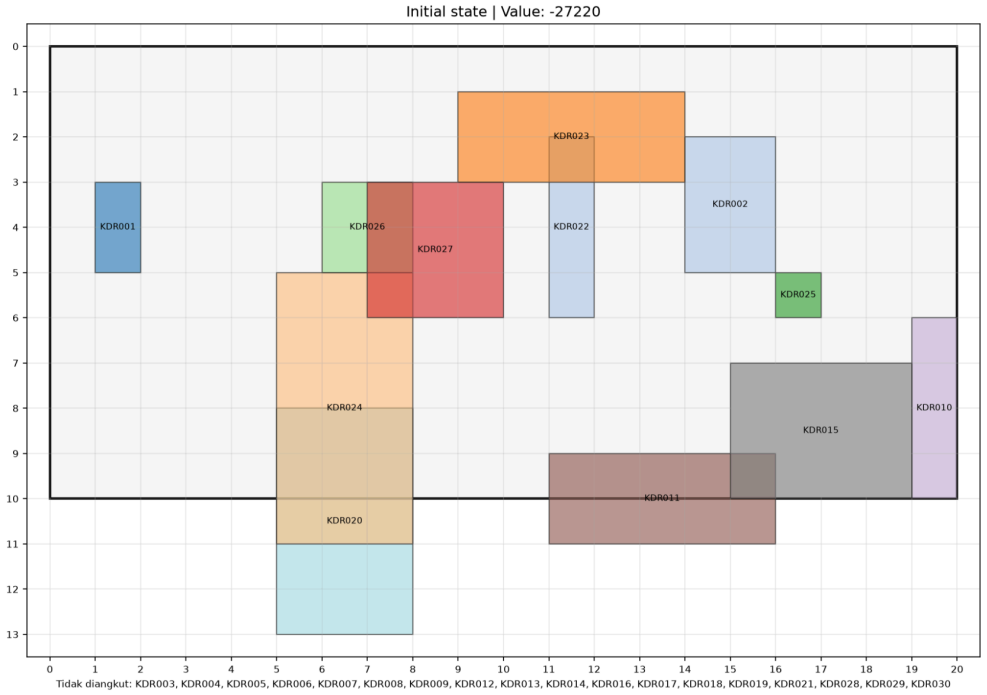
Gambar End State G-B1-1. Seed 5, pop 20, iter 100. Value : 1110

Hasil GA konsisten dalam hal kelayakan tetapi kurang konsisten dalam kualitas

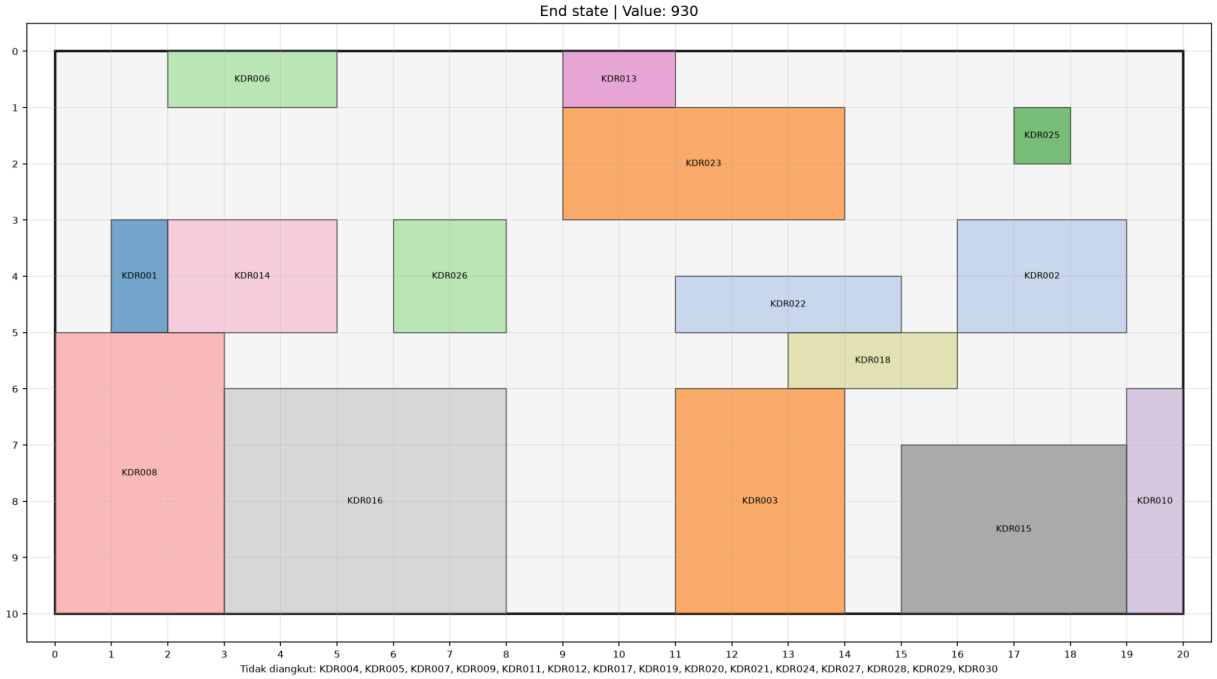
Gambar Initial State G-B1-2. Seed 6, pop 20, iter 100. Value : -33260



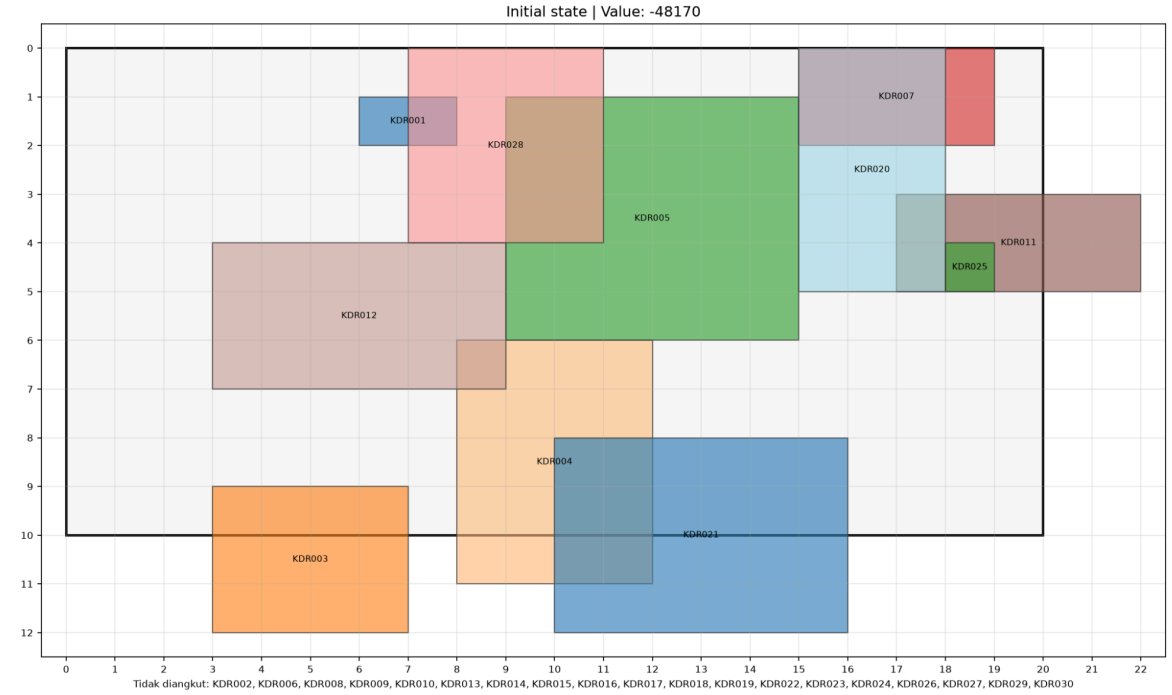
Gambar End State G-B1-2. Seed 6, pop 20, iter 100. Value : 1080



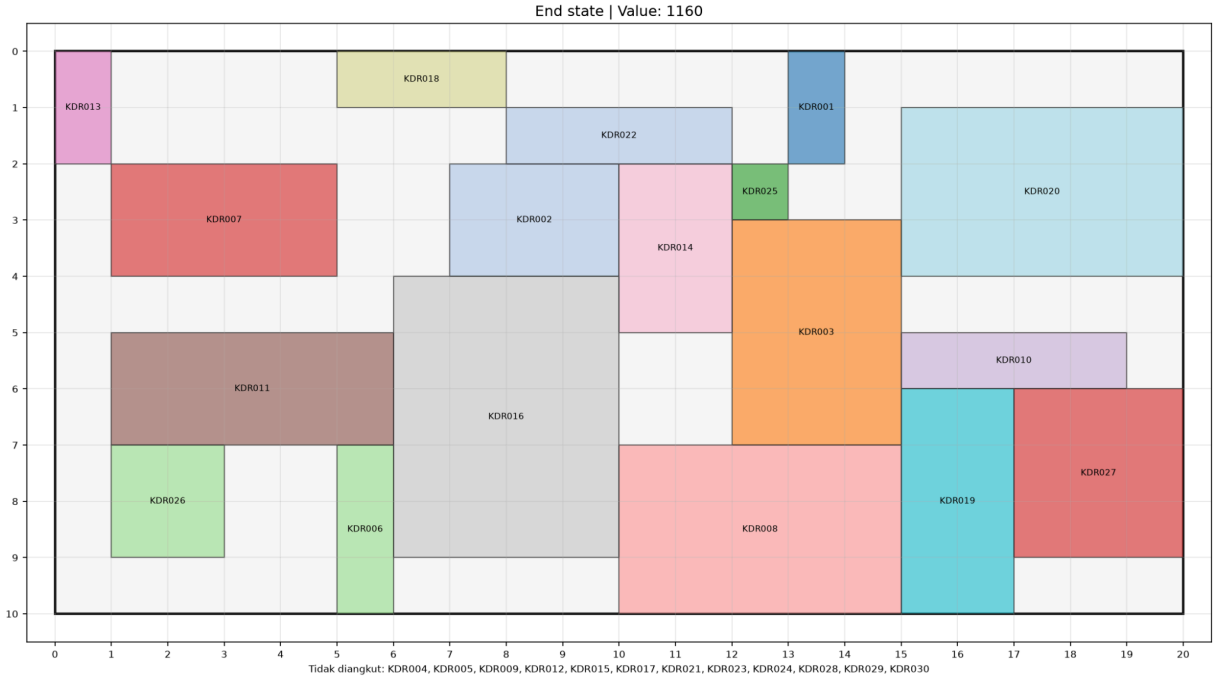
Gambar Initial State G-B1-3. Seed 7, pop 20, iter 100. Value : -27220



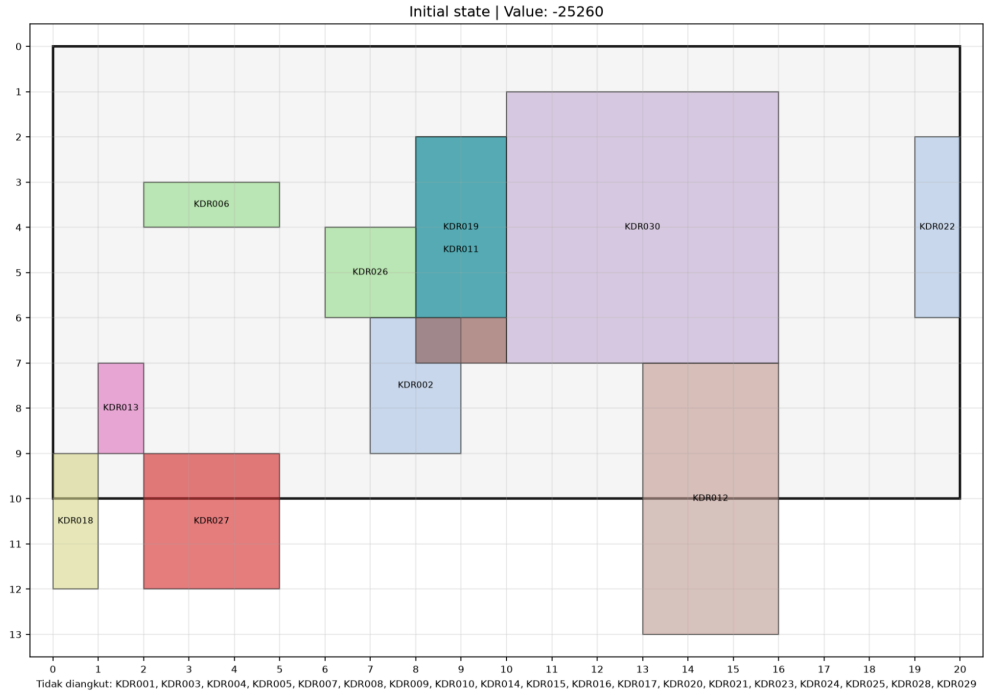
Gambar End State G-B1-3. Seed 7, pop 20, iter 100. Value : 930



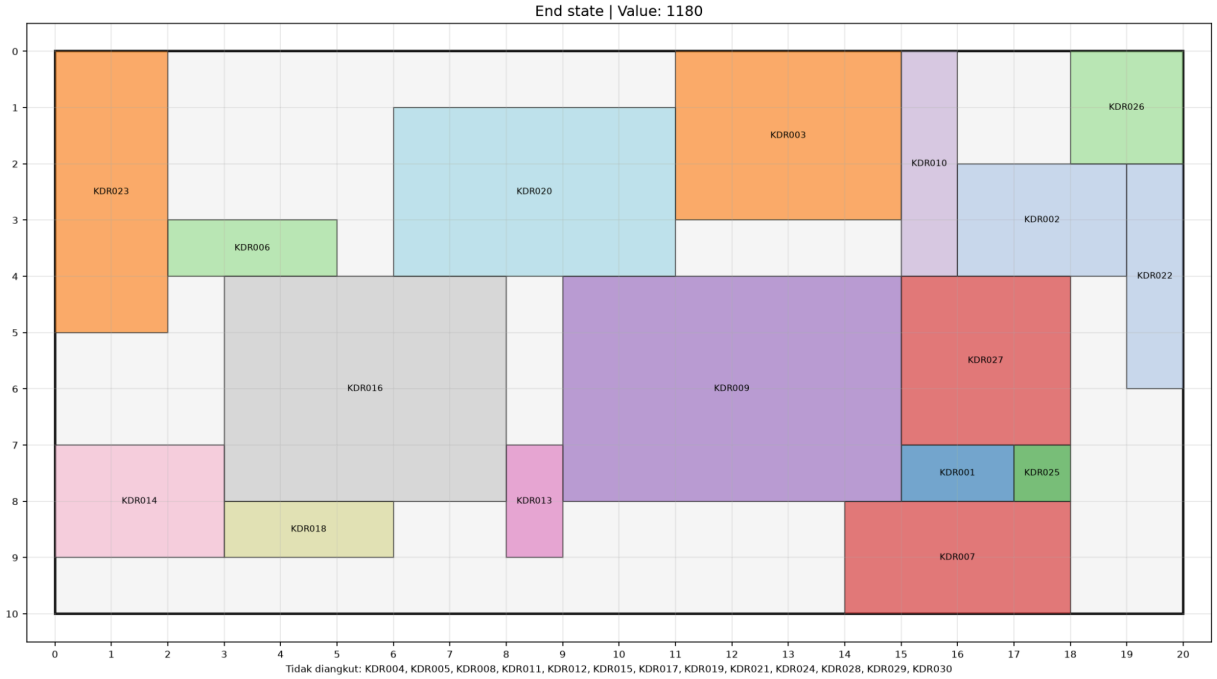
Gambar Initial State G-B2-1. Seed 5, pop 50, iter 100. Value : -48170



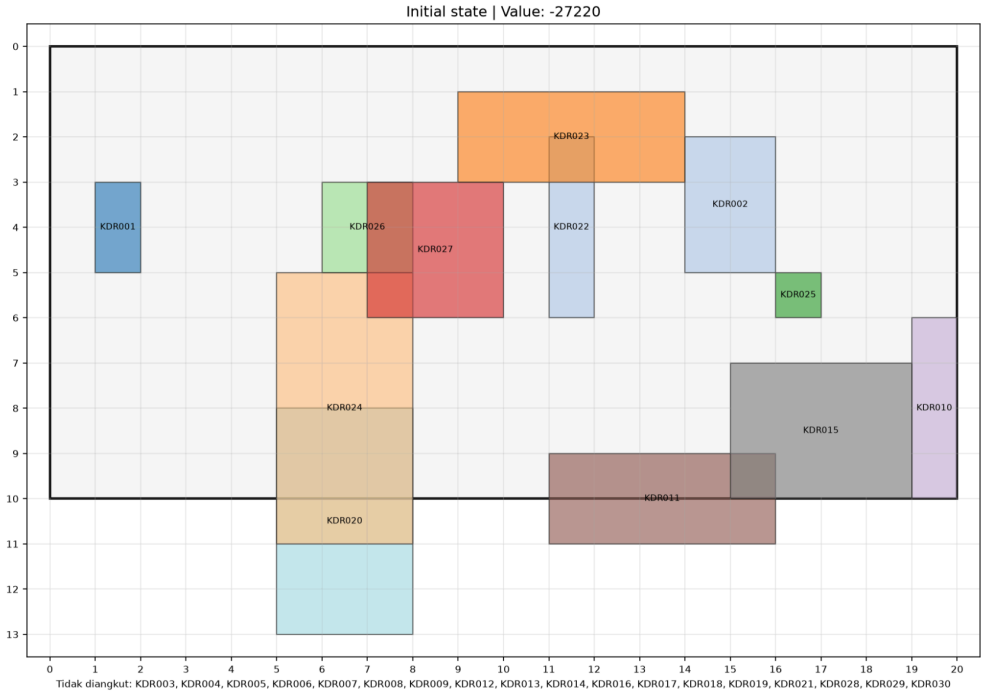
Gambar End State G-B2-1. Seed 5, pop 50, iter 100. Value : 1160



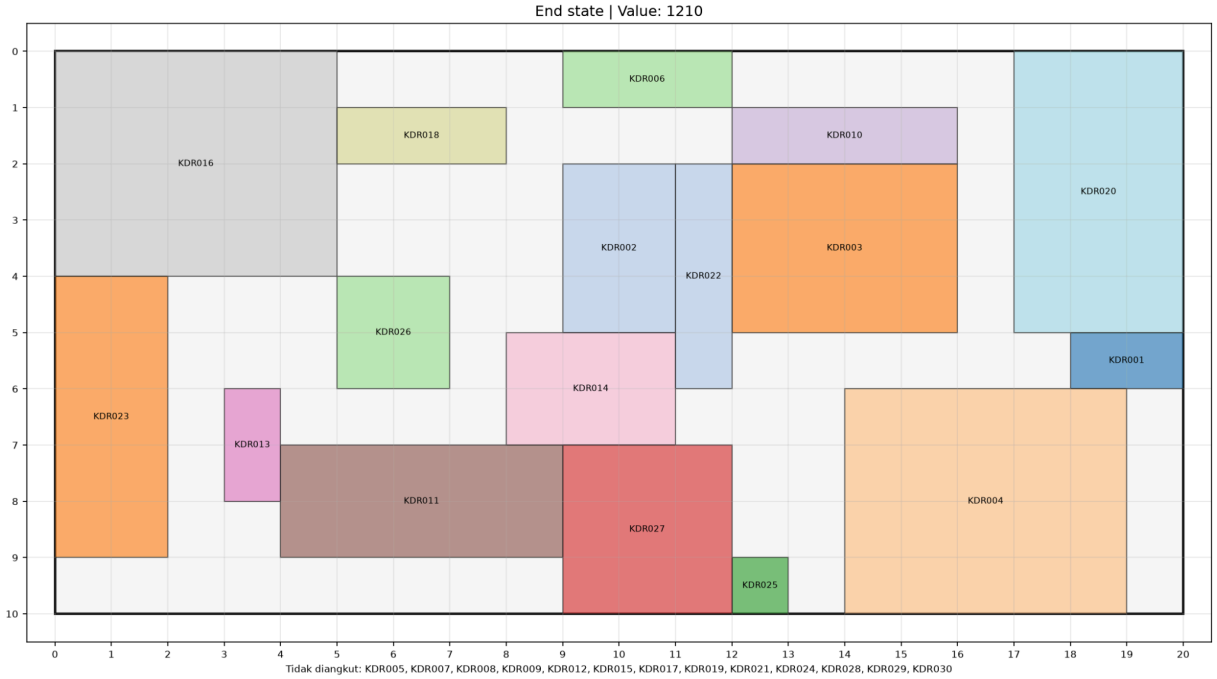
Gambar Initial State G-B2-2. Seed 6, pop 50, iter 100. Value : -25260



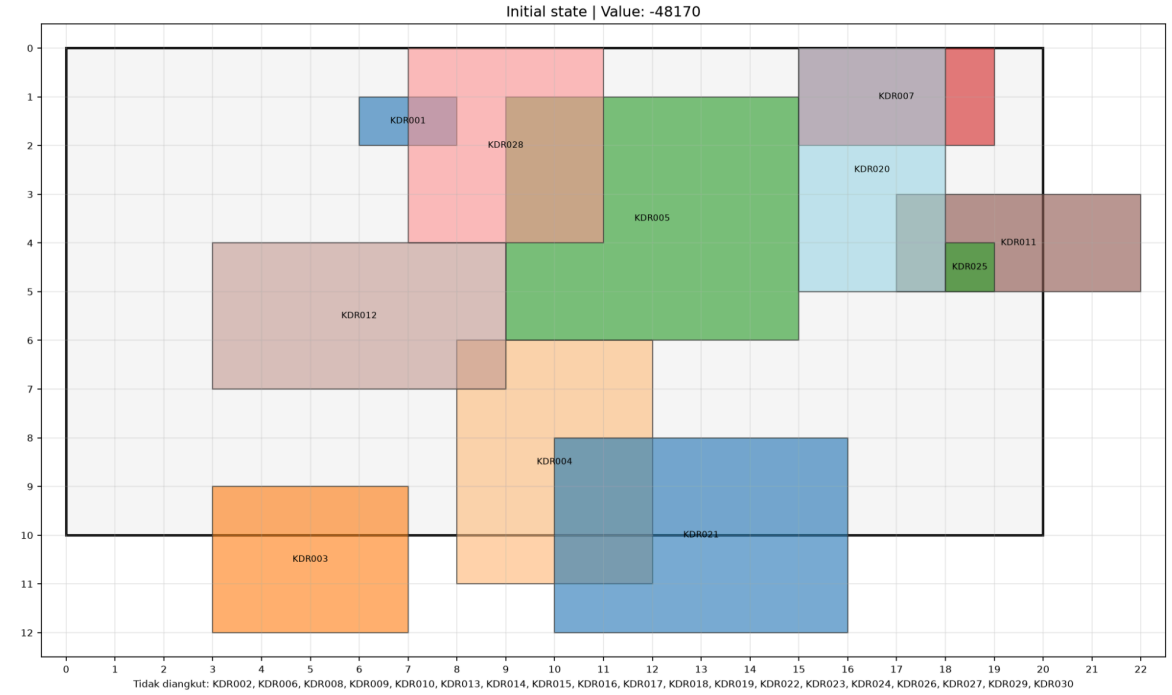
Gambar End State G-B2-2. Seed 6, pop 50, iter 100. Value : 1180



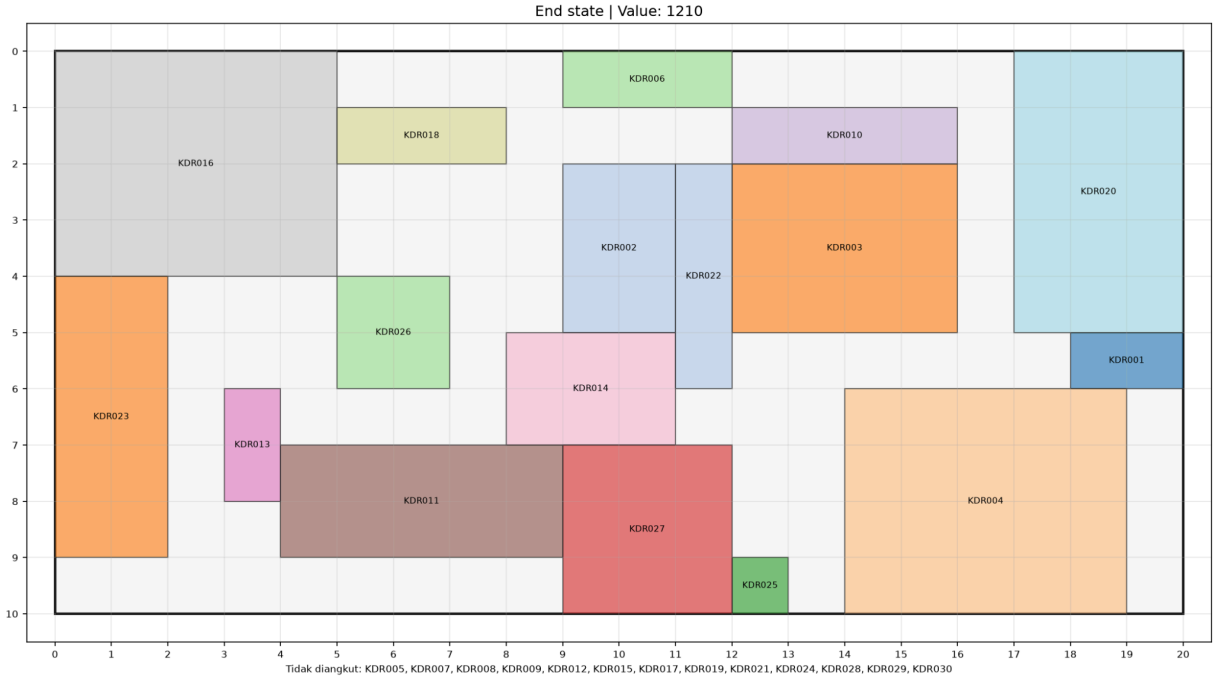
Gambar Initial State G-B2-3. Seed 7, pop 50, iter 100. Value : -27220



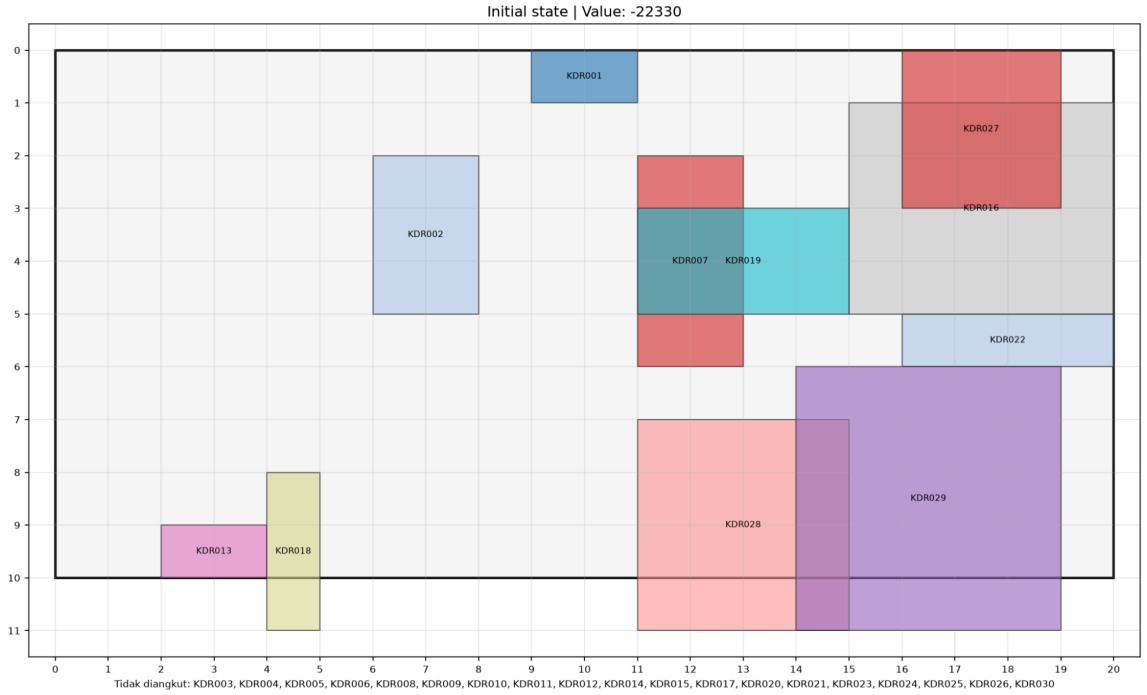
Gambar End State G-B2-3. Seed 7, pop 50, iter 100. Value : 1210



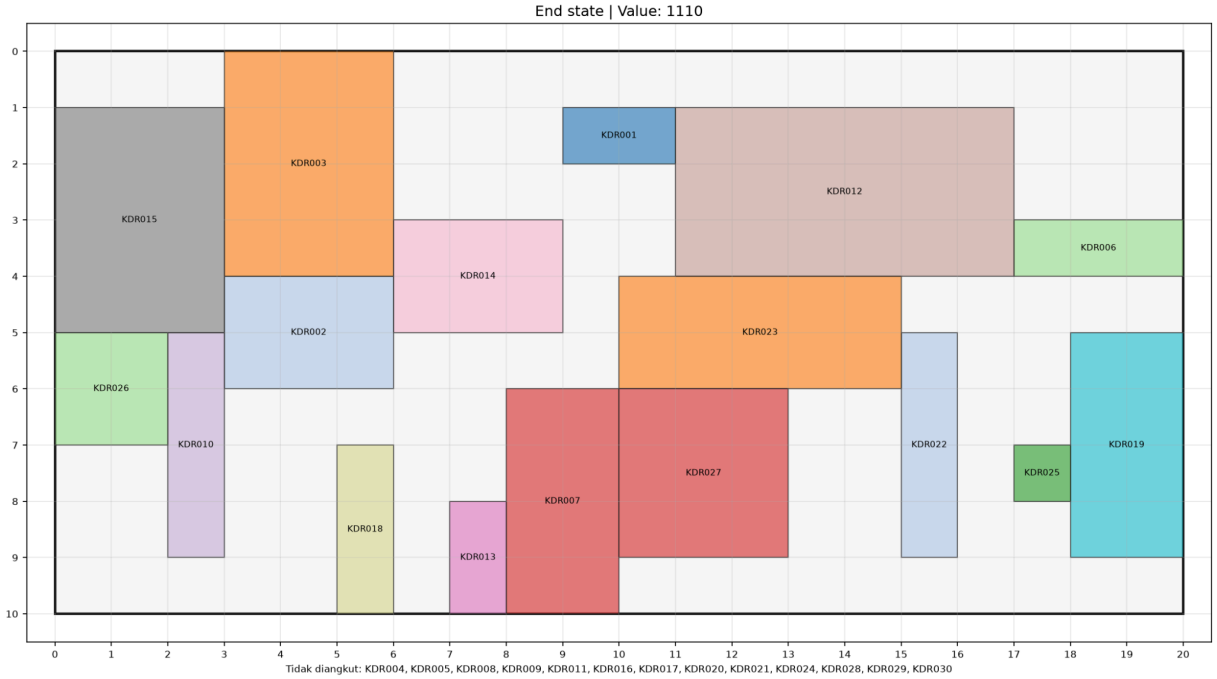
Gambar Initial State G-B3-1. Seed 5, pop 100, iter 100. Value : -48170



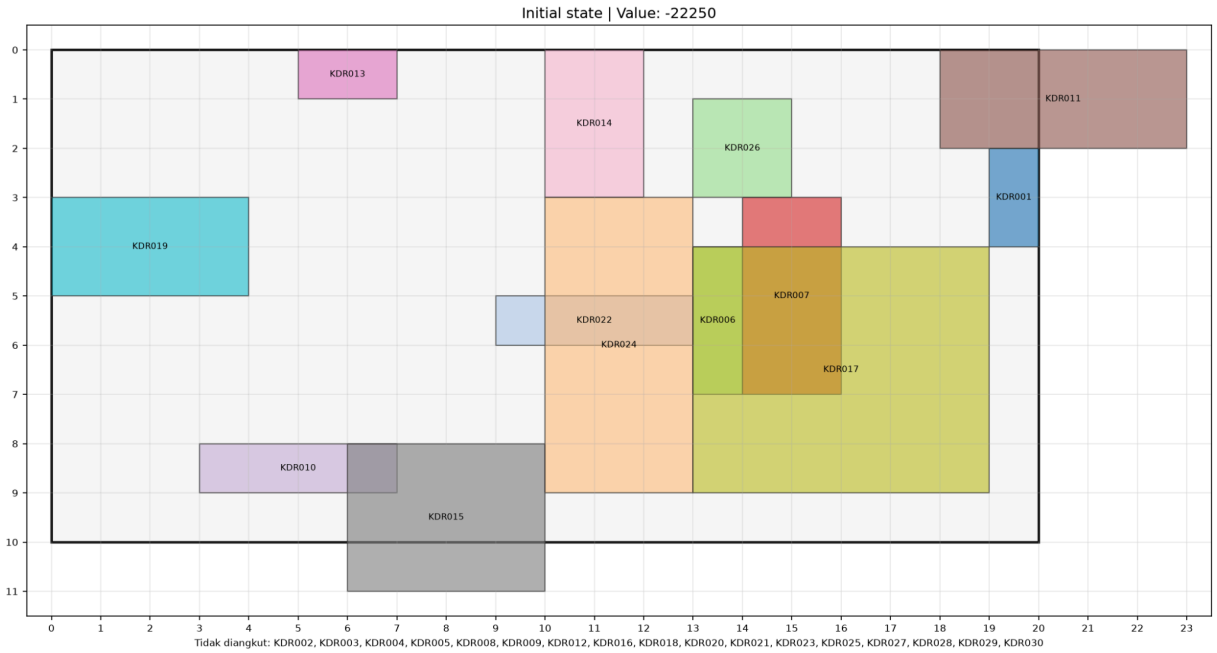
Gambar End State G-B3-1. Seed 5, pop 100, iter 100. Value : 1010



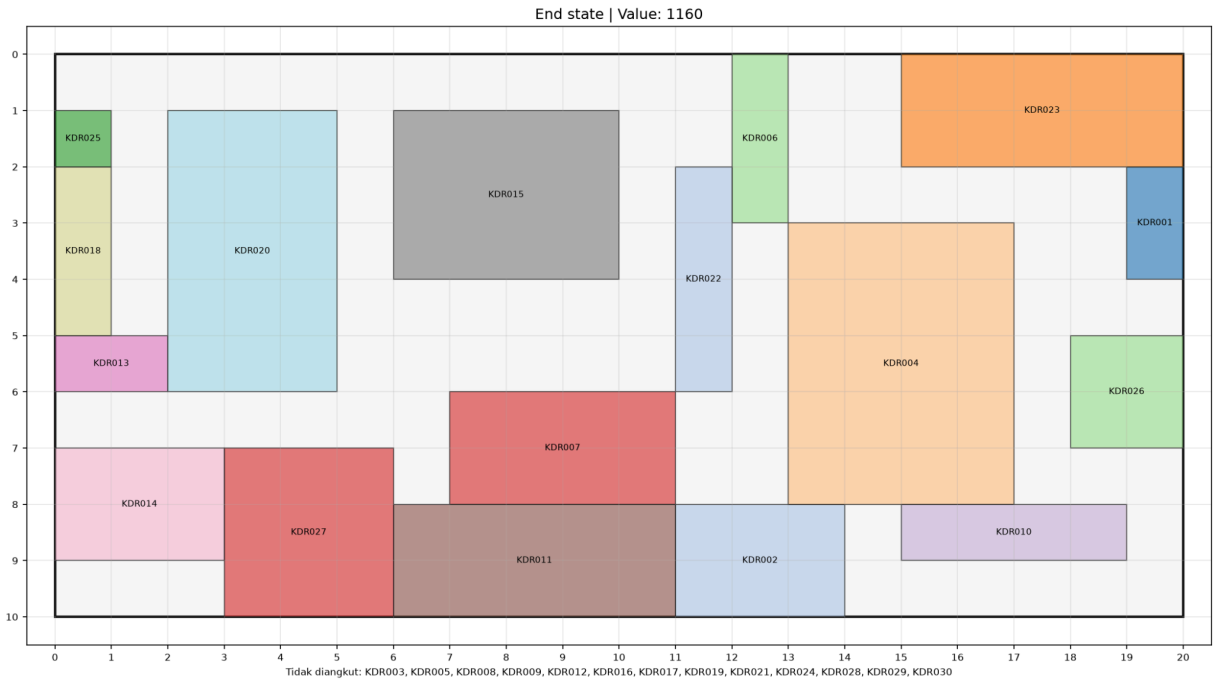
Gambar Initial State G-B3-2. Seed 6, pop 100, iter 100. Value : -22330



Gambar End State G-B3-2. Seed 6, pop 100, iter 100. Value : 1110

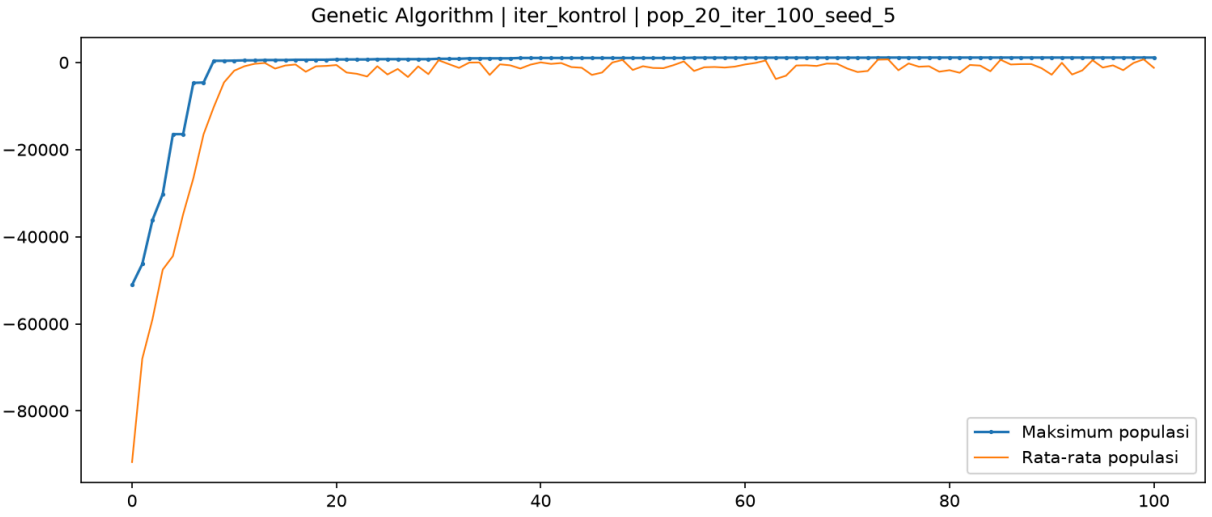


Gambar Initial State G-B3-3. Seed 7, pop 100, iter 100. Value : -22250

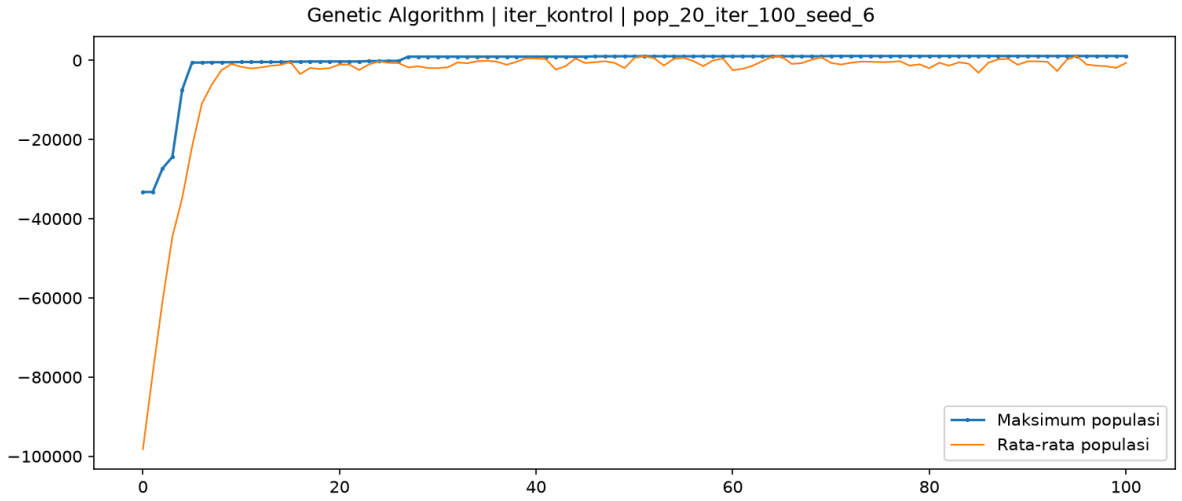


Gambar End State G-B3-3. Seed 7, pop 100, iter 100. Value : 1160

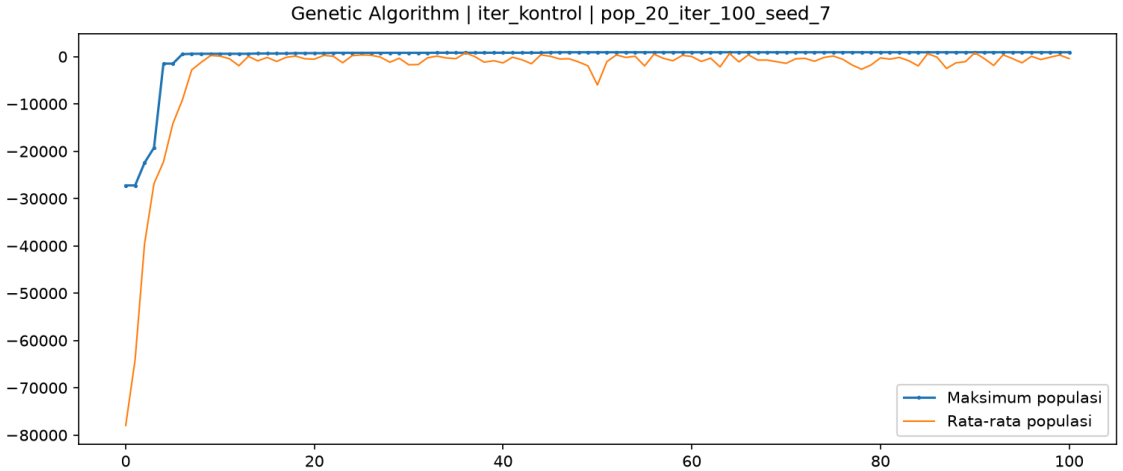
- Plot nilai objective function maksimum dan rata-rata populasi terhadap iterasi:



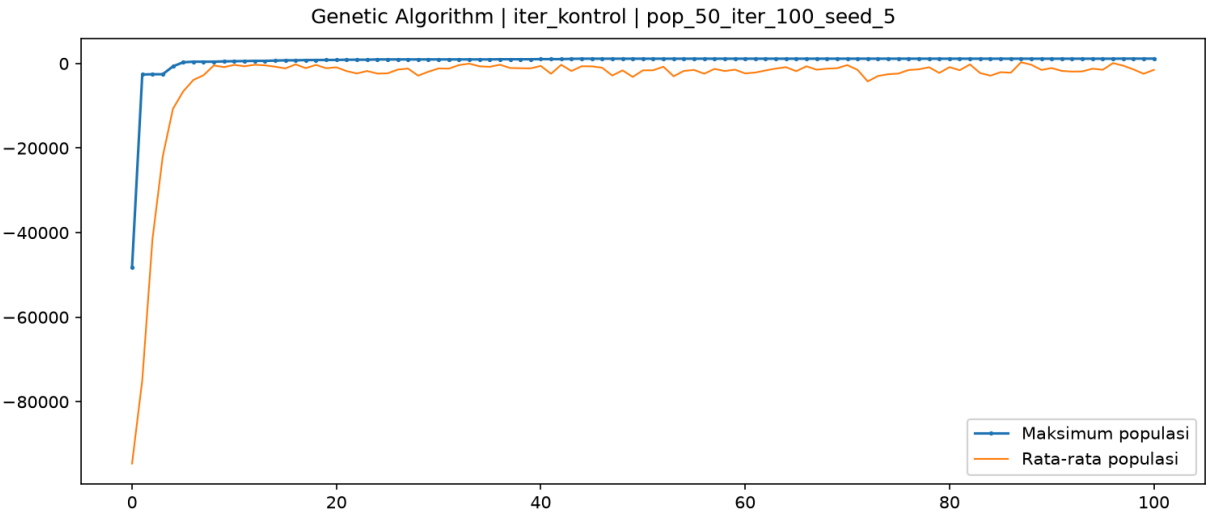
Gambar G-A1-1. Nilai objective max dan rata-rata populasi



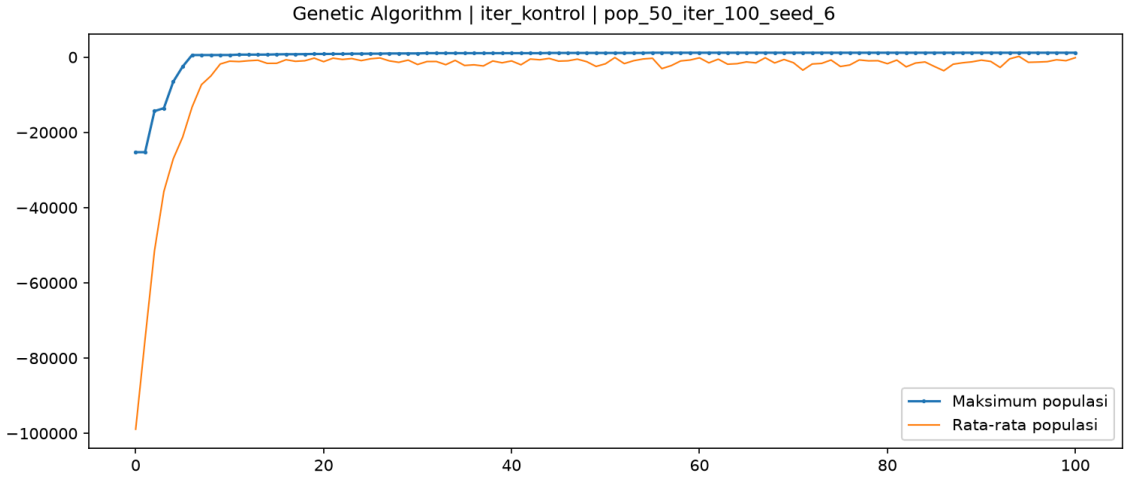
Gambar G-A1-2. Nilai objective max dan rata-rata populasi



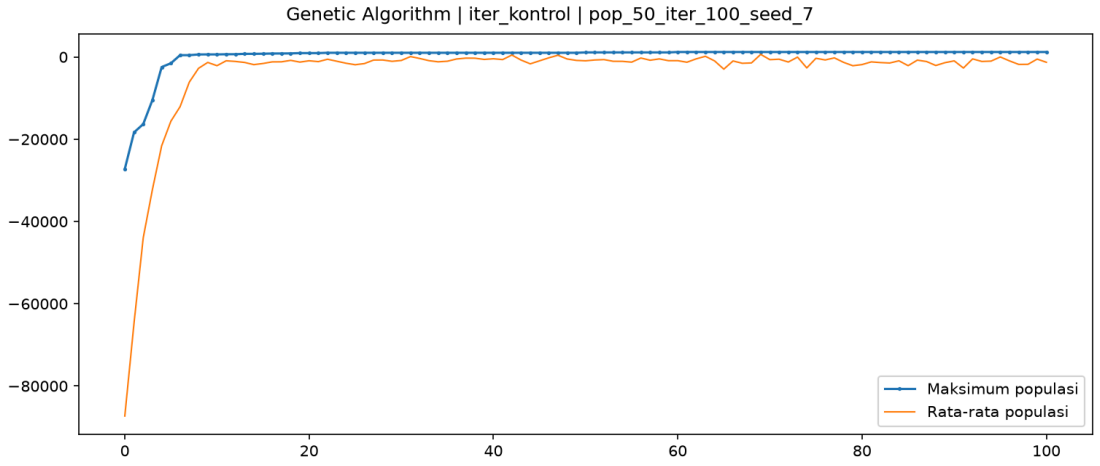
Gambar G-A1-3. Nilai objective max dan rata-rata populasi



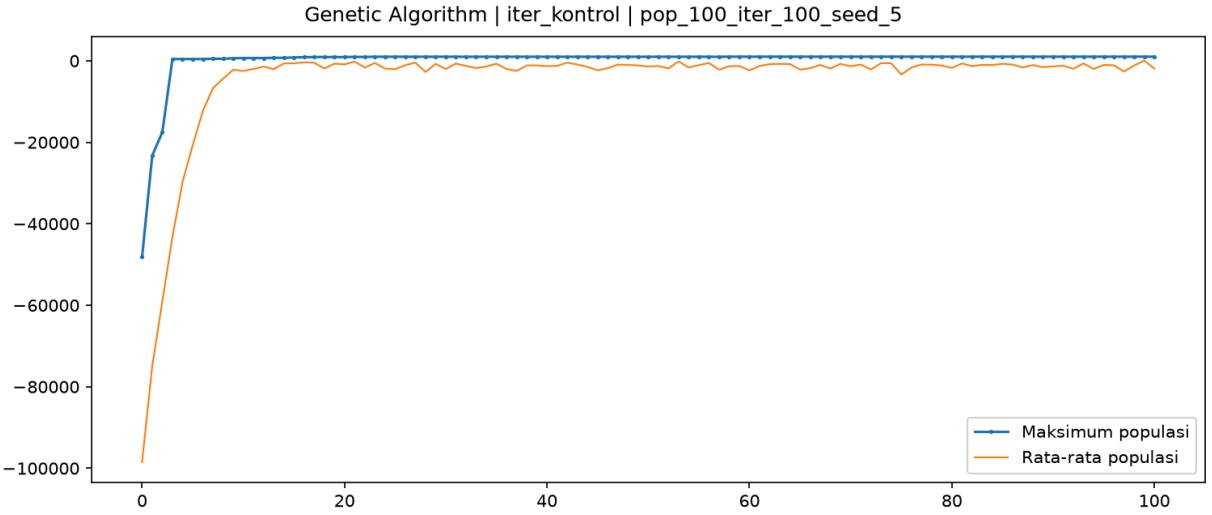
Gambar G-A2-1. Nilai objective max dan rata-rata populasi



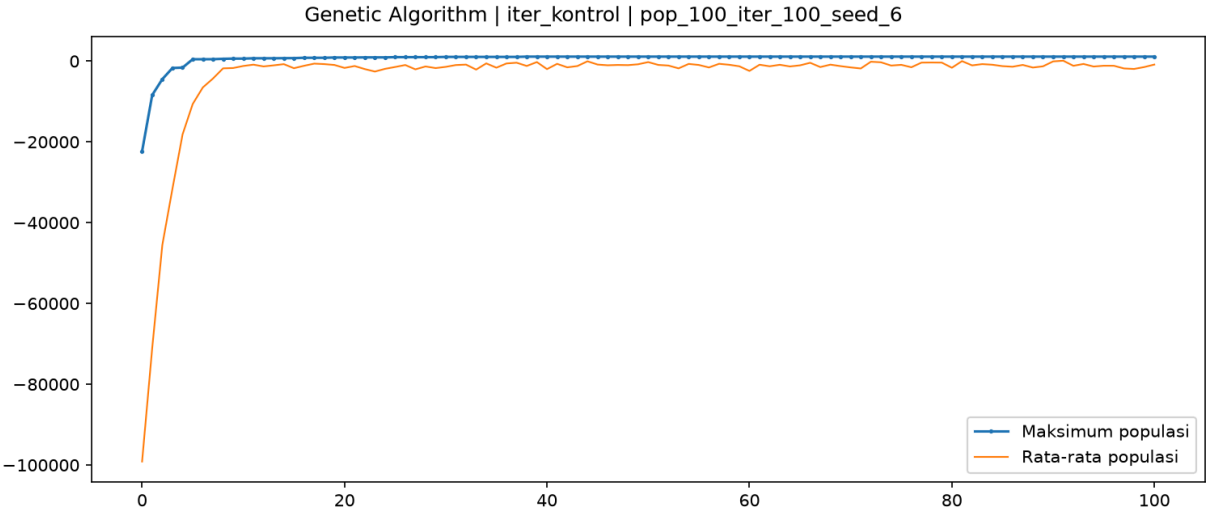
Gambar G-A2-2. Nilai objective max dan rata-rata populasi



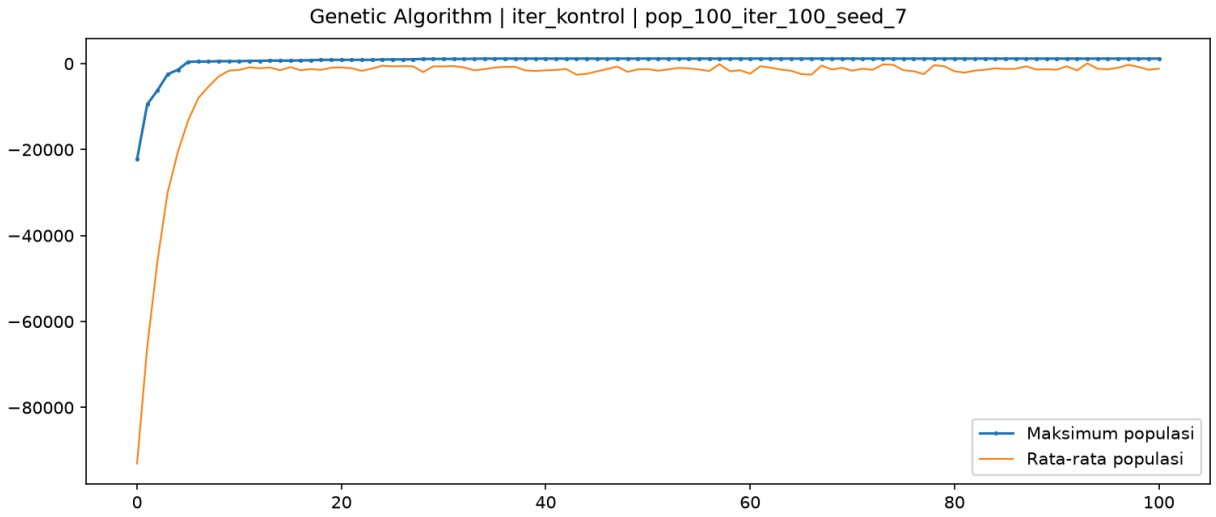
Gambar G-A2-3. Nilai objective max dan rata-rata populasi



Gambar G-A3-1. Nilai objective max dan rata-rata populasi



Gambar G-A3-2. Nilai objective max dan rata-rata populasi



Gambar G-A3-3. Nilai objective max dan rata-rata populasi

2.3 Analisis Eksperimen

2.3.1 Kedekatan terhadap Global Optima

A. Hill Climbing Steepest Ascent

Nilai terbaik dari tiga percobaan HC Steepest Ascent adalah 1310, dan seluruh *state akhir* memenuhi batasan. Namun, global optimum belum diketahui secara pasti sehingga ruang analisis sangat terbatas.

Berdasarkan run yang dilakukan, ketiga run berhenti pada nilai akhir yang berdekatan yang menandakan HC Steepest Ascent mungkin terjebak di local maximum. Selain itu, pada state akhir, total berat kendaraan yang diangkut sudah mendekati kapasitas yang berarti ruang untuk penambahan kendaraan lain sangat terbatas. Dengan demikian, meskipun dapat dilakukan optimalisasi lanjutan, solusi yang diperoleh HC Steepest Ascent mungkin hanya mendekati global optimum dan belum mencapainya.

B. SA

Nilai terbaik dari tiga percobaan SA adalah 1240 pada run 2. Nilai terendah adalah 1170 pada run 1, sedangkan rata-rata objective akhir adalah 1206,67. Namun, belum tersedia batas optimum yang cukup ketat untuk

menentukan kedekatan hasil tersebut terhadap global optimum. Total *fee* seluruh kendaraan mencapai 1970, tetapi nilai ini hanya batas atas kasar karena tidak memperhitungkan kapasitas maupun susunan kendaraan.

Tidak ada run yang menggunakan tepat seluruh kapasitas berat. Run 1 menggunakan 98/100, run 2 menggunakan 96/100, dan run 3 menggunakan 99/100. Pada run 1, kapasitas yang tersisa adalah 2 unit, sedangkan berdasarkan state akhir tidak ada overlap dan luas kendaraan yang ditempati adalah 124 dari total 200 sel geladak, sehingga terdapat 76 sel yang belum terisi. Ruang tersebut belum tentu bisa digunakan karena ukuran dan berat kendaraan yang tersisa harus sesuai dengan ruang kosong. Pada ketiga percobaan, kapasitas berat dan penataan ruang sama-sama membatasi hasil. Pada ketiga output saat ini, nilai akhir memang sama dengan nilai maksimum pada history masing-masing run: 1170, 1240, dan 1210. Namun, implementasi SA menyimpan state terakhir, bukan state terbaik sepanjang pencarian. Oleh karena itu, kesamaan tersebut merupakan hasil eksperimen ini dan tidak dijamin untuk setiap run.

C. Genetic

Pada 18 eksperimen, GA mendapati 1210 sebagai nilai terbaik (populasi 30, iterai 1000, seed 5; populasi 30, iterasi 200, seed 5; populasi 50, iterasi 100, seed7). Rata-rata value dari ke-18 eksperimen berada di 1084 dengan nilai terendah 860. Total fee seluruh kendaraan (1970) hanya batas atas kasar tetapi global optimum pada kasus ini dapat ditentukan pasti. Konfigurasi dengan rata-rata tertinggi adalah populasi 50, iterasi 100 dengan nilai 1183,3 sedangkan konfigurasi terendah adalah populasi 30, iterasi 50 dengan rata-rata 1006,7.

Kapasitas berat dan penataan ruang diperkirakan sama-sama membatasi hasil, karena menambah kendaraan pada susunan yang sudah padat berisiko menimbulkan overlap atau kelebihan besar. Mutasi GA ini hanya mengubah satu kendaraan per anak, sehingga perubahan tersebut kemungkinan besar menghasilkan child dengan nilai penalti 1000 dan

tersingkir saat seleksi. Elitism dan tournament selection juga dapat membuat populasi cepat menjadi seragam, yang terlihat pada populasi 30 seed 5 dan 6, dimana hasil iterasi 100 dan 200 identik, sehingga penambahan iterasi tidak lagi memperbaiki hasil.

2.3.2 Perbandingan Hasil Antar Algoritma

Tabel 2.3.2 Perbandingan Hasil Antar Algoritma

Algoritma	Rata-rata Objective Akhir	Terbaik	Terburuk
Hill-Climbing	1276	1310	1250
Simulated Annealing	1206,67	1240	1170
Genetic Algorithm	1084	1210	860

Ketiga algoritma mampu mengubah state awal yang sangat melanggar batasan (di mana nilainya mencapai minus ratusan ribu) menjadi susunan akhir yang valid. Namun, kualitas solusinya agak sedikit berbeda. Berdasarkan rata-rata objective akhir, urutan algoritma yang memiliki objective tertinggi hingga terendah yakni Hill-Climbing (1276,67), Simulated Annealing (1240) dan Genetic Algorithm (1084).

Ada beberapa faktor yang dapat menjelaskan perbedaan ini. Pertama, pada setiap iterasi HC mengevaluasi seluruh neighbor dan selalu memilih yang terbaik. Dengan cara ini, HC dengan cepat membersihkan pelanggaran dan bergerak menuju state yang valid secara pasti. Kedua, SA hanya mengevaluasi kandidat acak per iterasi. Dengan temperatur yang besar di awal membuat SA sering menerima kandidat yang buruk. Fase penyempurnaan SA baru efektif ketika temperaturnya sudah rendah sehingga hasil akhirnya umumnya di bawah HC. Ketiga, GA cukup rentan. Mutasi hanya mengubah satu kendaraan per anak sehingga yang melanggar batasan mudah tersingkir. Elitism dan tournament selection membuat populasi cepat seragam yang tampak pada hasil iterasi 100 dan 200 yang identik.

2.3.3 Perbandingan Durasi Pencarian

A. Hill Climbing Steepest Ascent

Durasi tiga percobaan HC Steepest Ascent yang tercatat adalah 9.11; 7.58; 7.14 detik, dengan rara-rata 7.94 detik. Durasi berbanding lurus dengan jumlah iterasi pada masing-masing run yaitu 14;13;12, jadi setiap iterasi membutuhkan waktu sekitar 0.58 hingga 0.65 detik. Biaya iterasi (wakti) cukup tinggi karena setiap iterasi pada HC Steepest Ascent akan mencari seluruh neighbor yang jumlahnya bisa mencapai ribuan state.

B. SA

Durasi tiga percobaan SA yang tercatat adalah 21,1624; 20,4528; dan 21,4031 detik, dengan rata-rata sekitar 21,0061 detik. Setiap percobaan menjalankan 4601 iterasi. Perbedaan durasi dapat dipengaruhi jumlah *neighbor* yang dibentuk pada setiap *state* dan kondisi *device* saat program dijalankan.

Pada setiap iterasi, SA mengevaluasi satu kandidat, tetapi **get_neighbors()** tetap membuat seluruh daftar *neighbor* terlebih dahulu. Proses ini membutuhkan waktu dan memori, terutama ketika pilihan posisi kendaraan cukup banyak. Perbandingan durasi dengan algoritma lain harus dilakukan pada data serta *environment* pengujian yang sama.

C. Genetic

Tabel 3.1 Perbandingan durasi, kontrol populasi vs kontrol iterasi (dari rata-rata 3 seed)

Kontrol	Config	Evaluasi (pop*iter)	Durasi	Avrg Skor
Populasi = 30	Iter 50	1.500	0.18s	1006,7
	Iter 100	3.000	0.32s	1076,7
	Iter 200	6.000	0.70s	1103,3
Iterasi = 100	Iter 20	2.000	0.19s	1040,0
	Iter 50	5.000	0.56s	1183,3
	Iter 100	10.000	1.03s	1093,3

Durasi pencarian GA naik hampir linear terhadap jumlah evaluasi (populasi * iterasi), dengan biaya sekitar 0,10-0,12 ms perevalusi, baik pada

kontrol populasi maupun kontrol iterasi. Pada kontrol populasi, iterasi 50, 100, dan 200 membutuhkan rata-rata 0,18s, 0,32s, dan 0,70. Pada kontrol iterasi, populasi 20, 50, 100 membutuhkan 0,19s, 0,56s, dan 1,03s. Karena menggandakan iterasi maupun populasi sama-sama menggandakan durasi, keduanya setara dari sisi biaya waktu dan menentukan hanyalah hasil kalinya.

Perbedaan dapat dilihat pada hasil yang diperoleh per satuan waktu. P50 * I100 (0,56s) menghasilkan rata-rata 1183,3 dan menambah iterasi dari 100 ke 200 pada populasi 30 melipatduakan durasi tetapi hanya menaikkan rata-rata sekiat 27 poin karena populasi stagnan. Durasi juga stabil antar-seed (selisih 5–20% per *config),* sehingga variasi skor akhir berasal dari faktor acak pencarian, bukan dari perbedaan waktu komputasi. Secara keseluruhan populasi 50 dan iterasi 100 memberikan keseimbangan terbaik antara durasi dan hasil.

2.3.4 Konsistensi Hasil

A. Hill Climbing Steepest Ascent

Nilai objective akhir HC Steepest Ascent berada pada rentang 1250 sampai 1310 dengan rata-rata 1276,67. Selisih antara hasil tertinggi dan terendah adalah 60, atau sekitar 4.73% dari rata-rata. Hasil pada tiga seed ini cukup konsisten. Hal ini cukup menarik karena meskipun nilai awal ketiga run berbeda (-112680;-56060;-37110), tetapi nilai akhirnya berdekatan. Jumlah iterasi yang dilakukan juga hampir sama. Dalam hal ini, HC cukup andal membersihkan pelanggaran dari state awal acak manapun, sedangkan hasil akhirnya mungkin mendekati local maximum.

B. SA

Nilai objective akhir SA berada pada rentang 1220 sampai 1260, dengan rata-rata 1240. Selisih antara hasil tertinggi dan terendah adalah 40, atau sekitar 3,23% dari rata-rata. Ketiga percobaan menghasilkan state yang valid, dengan jumlah kendaraan akhir sebanyak 17, 18, dan 17.

Hasil pada tiga seed tersebut cukup berdekatan meskipun state awal dan urutan kandidat berbeda.

C. Genetic

Hasil GA konsisten dalam hal kelayakan tetapi kurang konsisten dalam kualitas. Seluruh 18 run, yang state awalnya sangat negatif (-22.000 sampai -51.000), berakhir di rentang 860-1210, tetapi selisih terbaik dan terburuk mencapai 350 poin dan semuanya masih berada 210-560 poin di bawah nilai terbaik yang ditemukan dalam eksperimen, yaitu 1210. Konfigurasi paling konsisten adalah populasi 50, iterasi 100, sedangkan yang paling tidak konsisten adalah populasi 30, iiterasi 20 dan iterasi 100, karena pada populasi kecil hasil lebih bergantung pada populasi awal dan seed. Konsistensi juga tidak naik terus seiring ukuran populasi, karena pada populasi 100 lebih tersebar daripada populasi 50, dan urutan kualitas antar-seed berubah-ubah antara konfigurasi. Dengan hanya 3 run per konfigurasi kesimpulan ini bersifat indikatif, bukan signifikan secara statistik.

2.3.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm

Pada eksperimen A (populasi kontrol = 30), menaikkan iterasi dari 50, 100 dan 200 menaikkan rata-rata nilai akhir dari 1006,7 ke 1076,7 dan 1103,3. Peningkatannya lambat yaitu +70 pada kenaikan pertama dan +27 pada kenaikan kedua, sementara durasi naik dua kali lipat (0,32s menjadi 0,70s). Pada seed 5 dan 6, hasil iterasi 100 dan 200 identik (1210 dan 1100), yang menunjukkan populasi sudah stagnan akibat konvergensi dini (elitism dan tournament selection membuat individu cepat seragam), sehingga iterasi tambahan tidak lagi menghasilkan perbaikan.

Pada eksperimen B (Iterasi kontrol = 100), populasi 20, 50 dan 100 menghasilkan rata-rata 1040,0; 1183,3 dan 1093,3. Dari populasi 20 kke 50, hasil naik jelas dan sebarannya menyempit (rentang 10 menjadi 50), karena populasi kecil ruang beragam sehingga mudah terjebak di *local optimum*. Namun populasi 100 justru menurun, kemungkinan karena populasi yang besar membutuhkan lebih

banyak generasi untuk konvergen sedangkan iterasi dibatasi 100, meskipun dengan 3 run per konfigurasi, selisih ini bisa juga berasal dari variasi acak.

Dari kedua eksperimen, populasi memberi pengaruh lebih besar dari iterasi pada rentang yang diuji. Konfigurasi populasi 50 iterasi 100 mencapai rata-rata 1183,3 hal ini lebih baik daripada populasi 30 iterasi 200 yang hanya 1103,3 meski lebih lama. Durasi naik hampir linear terhadap populasi * iterasi, tetapi hasil tidak, sehingga memperbesar parameter di atas titik tertentu menambah waktu tanpa jaminan hasil lebih baik. Dalam percobaan ini, titik tengah (populasi 50 dan iterasi 100) memberikan keseimbangan terbaik antara kualitas, konsistensi dan durasi.

3. Kesimpulan dan Saran

3.1 Kesimpulan

1. HC Steepest Ascent menghasilkan tiga susunan akhir yang valid dengan nilai objective 1250, 1270, 1310 pada seed 21, 42, 63. Rata-rata nilai objective yaitu 1276,67, rata-rata iterasi hingga kondisi terminasi yaitu 13 iterasi, dan rata-rata durasi yaitu 7.94 detik. Algoritma berhasil menaikkan objective dari nilai awal yang sangat melanggar batasan menjadi solusi valid. Algoritma berhenti di local maximum karena kapasitas berat hampir penuh sehingga kemungkinan global optimum tidak tercapai.
2. Simulated Annealing menghasilkan tiga susunan akhir yang valid dengan objective 1170, 1240, dan 1210 pada seed 21, 42, dan 63. Rata-rata objective akhirnya adalah 1206,67, dan setiap percobaan menjalankan 4601 iterasi. Hasil ini lebih baik daripada *state* awal yang masih melanggar batasan, tetapi belum membuktikan tercapainya global optimum.
3. Genetic Algorithm menyusun muatan kapal dengan kualitas yang layak dan menguntungkan. Terbukti dari pemrosesan state awal yang negatif (-22.000 sampai -51.000), seluruh 18 percobaan berakhir di nilai positif 860-1210 dengan rata-rata 1084 dan nilai terbaik 1210. Karena batas optimum belum diketahui, kedekatan hasil terhadap global optimum tidak dapat ditentukan. Konfigurasi terbaik adalah populasi 50 dengan iterasi 100 (rata-rata 1183,3 pada rentang antar-seed 50, durasi 0,56s), dan durasi naik hampir linear

terhadap populasi * iterasi, tetapi hasilnya tidak. Hasil yang identik pada iterasi 100 dan 200 ((populasi 30, seed 5 dan 6) menunjukkan pencarian dapat stagnan, yang diduga akibat mutasi yang hanya mengubah satu kendaraan per anak serta elitism dan tournament selection yang membuat populasi cepat seragam. Bagi PT Intim Line, GA layak sebagai alat bantu.

3.2 Saran

1. HC Steepest Ascent dapat ditingkatkan dengan varian HC lainnya yang bisa melewati plateau dan local maximum, yaitu varian Sideways Move dan Random Restart Hill-Climbing. Pengujian juga sebaiknya menggunakan lebih banyak seed agar perbandingan antar algoritma lebih terlihat.
2. Pengujian SA dapat dilanjutkan dengan lebih banyak *seed* serta variasi temperatur awal dan *cooling* rate. Jika ingin mengukur kejadian *local optimum*, perlu pemeriksaan seluruh *neighbor*, bukan hanya menghitung kandidat buruk yang ditolak. *Objective* alternatif juga perlu digunakan langsung sebagai evaluator pencarian agar hasilnya dapat dibandingkan dengan *objective* berbasis *fee*.
3. Genetic Algorithm dapat ditingkatkan dengan memperkuat mutasi dengan mengubah beberapa kendaraan sekaligus dan ditambah repair operator untuk membuang kendaraan yang overlap atau menyebabkan kelebihan berat, sementara keragaman populasi dijaga, misalnya dengan menyuntikkan individu acak. Populasi sekitar 50 dengan 100 iterasi dapat dipakai sebagai konfigurasi awal karena paling seimbang pada percobaan ini. Pengembangan berikutnya dapat berupa hybrid GA dengan local search, penambahan ETA sebagai batasan deadline, dan perbandingan dengan HC dan SA pada data yang sama.

4. Pembagian Tugas

Tabel 4 Pembagian Tugas

NIM	Nama	Tugas yang Dikerjakan
18224001	Nisrina Zakiyah	Simulated Annealing, data uji, *visualization*, **bonus *objective function,*** dan laporan bagiannya
18224017	Naomi Azzahra	Genetic Algorithm, run file for genetic algorithm (experiment.py), formatting laporan, laporan pada bagian Genetic Algorithm.
18224083	Antania Hanjani Yustika Putri	HC Steepest Ascent, Data Uji, Fungsional Dasar (Representasi Masalah, Pencarian Neighbor, Objective Function Dasar), laporan bagiannya.

REFERENSI

[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video]. [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video]. [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video]. [Accessed: Sep. 16, 2026].

[6] Towards AI, "tutorials," GitHub. [Online]. Available: https://github.com/towardsai/tutorials. [Accessed: Sep. 16, 2026].

[7] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

Lampiran A. Form Penggunaan Generative AI

Link Form Penggunaan GenAI:

Pernyataan AI Tugas Besar 1 IF3170 Dasar Inteligensi Artifisial 2026_2027.pdf

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Nisrina Zakiyah/18224001,
Naomi Azzahra/18224017,
Antania Hanjani Yustika Putri/18224083

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi (STEI-K)

Kode Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Inteligensi Artifisial

Judul Tugas/Proposal : Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Gemini Google auto di docs
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Pembuatan test case.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Penjelasan algoritma *local search*. Bantuan debugging.

Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Nisrina Zakiyah
18224001



Naomi Azzahra
18224017



Antania Hanjani. Y.P
18224083

Lampiran B. Lampiran Lainnya

Link GitHub Repository:

https://github.com/bellechillguy/IF3070_G21_Antum_Intelligence